Machine Learning From Scratch

Second Edition

Hieu Nguyen

© Hieu Nguyen
hieutn@bu.edu

Preface to the Second Edition

We wrote the first version of these notes pretty quickly, mostly for ourselves, so it showed. There were typos, a couple of derivations that were wrong in ways that happened to cancel out, and a few places where we wrote "blah blah blah" instead of explaining the thing. This edition keeps the spirit and the order of the original, and it tries to fix everything else.

The philosophy hasn't changed. Every method in this book is derived from scratch and then implemented from scratch in Python with nothing fancier than NumPy. Only after we have built something ourselves do we reach for a library. We follow the classical treatments, mainly Bishop's *Pattern Recognition and Machine Learning* [3] and Hastie, Tibshirani and Friedman's *The Elements of Statistical Learning* [6], for the derivations. Where the field has moved on since those books were written, we say so and give the modern version more room. You will find mini-batch training, Adam, automatic differentiation, k-means++, double descent, soft-margin SVMs trained by subgradient descent, and Q-learning, alongside the classics they grew out of.

What You Need

We assume you are comfortable with multivariable calculus and with linear algebra at the level of eigenvalues and the SVD. A first course in probability helps a lot, and if you have seen maximum likelihood once, you are in great shape. On the programming side, you need Python 3 with `numpy` and `matplotlib`. We use `scikit-learn` now and then for convenience, and `cvxopt` for one example in Chapter 6. All listings were run while preparing this edition and the printed outputs are the real ones.

How To Use The Problems

Instead of piling all the problems up at the end of each chapter, we scattered them throughout so that each one sits right after the idea that it is about. Some are short derivations that fill a gap we skipped on purpose, and others are coding exercises, a couple of which are fairly open-ended. We strongly suggest doing at least the derivations as you go, since several later sections assume you have.

Notation

Vectors are bold lowercase ($\mathbf{x}, \mathbf{y}, \boldsymbol{\theta}$), matrices are bold uppercase ($\mathbf{X}, \boldsymbol{\Sigma}$). We have N data points and M features, and $\mathbf{X}$ is the $N \times (M + 1)$ feature matrix whose first column is all ones. Parameters are $\boldsymbol{\theta}$ (or $\mathbf{w}$ and b when the intercept is kept separate, as is common for neural networks and SVMs). A hat means an estimate or a prediction, α is a learning rate, and λ is a regularization strength or an eigenvalue, depending on context, which we will always make clear.

Hieu

Contents

Preface ii

I Supervised Learning 1

1 Linear Regression 2
- 1.1 Setting Up The Problem . . . 2
- 1.2 Least Squares . . . 3
  - 1.2.1 The Cost Function . . . 4
  - 1.2.2 Why Squares? . . . 6
- 1.3 The Normal Equations . . . 7
  - 1.3.1 The Geometry, Or Why "Normal" . . . 9
  - 1.3.2 Solving It Without Inverting Anything . . . 10
- 1.4 Gradient Descent . . . 10
  - 1.4.1 Feature Scaling . . . 14
  - 1.4.2 Stochastic And Mini-Batch Gradient Descent . . . 14
- 1.5 Regularization . . . 15
  - 1.5.1 Gradient Descent With Ridge . . . 17

2 Linear Classification 20
- 2.1 Why Not Linear Regression For Classification? . . . 20
- 2.2 Logistic Regression . . . 24
  - 2.2.1 The Likelihood . . . 25
  - 2.2.2 Cross-Entropy, The Loss Everyone Uses . . . 26
  - 2.2.3 Gradient Descent . . . 26
  - 2.2.4 Why Is This A Linear Classifier? . . . 27
  - 2.2.5 An Example . . . 28

2.2.6 Separable Data . . . 29
2.3 Multiclass Classification . . . 30
2.3.1 Measuring How Well We Did . . . 33
2.3.2 The Last Layer . . . 34

3 Basis Expansion And Regularization 36
3.1 Choosing A Basis . . . 36
3.2 Overfitting, The Burden Of Choice . . . 40
3.2.1 The Bias-Variance Tradeoff . . . 41
3.2.2 Cross-Validation, Or How To Actually Choose . . . 43
3.3 Regularization . . . 44
3.3.1 The Lasso . . . 45
3.4 A Modern Twist, Double Descent . . . 47

4 Neural Networks 49
4.1 Neural Network Architecture . . . 49
4.2 The Feed Forward Algorithm . . . 51
4.2.1 Vectorized Feed Forward . . . 52
4.3 Back Propagation . . . 53
4.4 An Example . . . 55
4.5 Training Networks The Modern Way . . . 57
4.6 Automatic Differentiation From Scratch . . . 60
4.7 Other Architectures . . . 62

II Unsupervised Learning 64

5 K-Means, Gaussian Mixtures, And PCA 65
5.1 Problem Set Up . . . 65
5.2 The K-Means Algorithm . . . 66
5.2.1 Better Starts With k-means++ . . . 68
5.2.2 Choosing k . . . 69
5.3 Gaussian Mixture Models . . . 70
5.3.1 How Do Gaussian Mixture Models Learn? . . . 72
5.3.2 The Expectation Maximization Algorithm . . . 73
5.3.3 Why EM Works . . . 76
5.4 Dimensionality Reduction And Principal Component Analysis . . . 77
5.4.1 PCA As Best Approximation . . . 78
5.4.2 PCA Is Not Regression . . . 80

5.4.3 PCA Through The SVD 80
5.4.4 A Real Example, Handwritten Digits 81

III More Tools for the Toolbox 83

6 Support Vector Machines 84
6.1 Finding The Margin 85
6.1.1 The Dual Problem 87
6.2 An Illustrative Example 88
6.3 Soft Margins 89
6.3.1 The Hinge Loss, Or SVMs Without A QP Solver 91
6.4 Kernels 92
6.4.1 Kernels Today 95

7 Bayesian Methods 97
7.1 Bayesian Statistics 97
7.1.1 The Posterior Distribution 98
7.1.2 MAP Estimation, And Where Ridge Came From 100
7.2 Bayesian Linear Regression 101
7.3 Linear Discriminant Analysis 103
7.3.1 Estimating The Parameters 105
7.3.2 Generative Versus Discriminative 106
7.4 Quadratic Discriminant Analysis 108

8 Reinforcement Learning 110
8.1 Markov Decision Processes 110
8.1.1 Returns And Discounting 111
8.2 Value Functions And The Bellman Equations 112
8.3 Value Iteration 113
8.4 Q-Learning 116
8.5 Scaling Up, Deep Q-Networks 119
8.6 Policy Gradients 120
8.6.1 From Games To Language Models 121

Part I

Supervised Learning

1

Linear Regression

"If you're the one teaching the machine, shouldn't it be called machine teaching?"

–

There is a popular misconception that machine learning is some brand new field that showed up a few years ago to pave the way for the future. It isn't new. Machine learning is a science built on the back of statistics, and a good chunk of what we will cover in this book has been known for more than half a century. The reason it blew up recently is mostly the sheer computational power of modern hardware, plus the absurd amount of data that we now collect. So we don't want to just yap about ideas that are older than our parents. We want to revisit them and build them from scratch, because we think that is the most useful way to learn them, for the reader and for us. The mathematics matters and we will do it carefully. The emphasis, though, is on implementation, since that is where the ideas become real. Every derivation in this book ends in code that you can run.

1.1 Setting Up The Problem

The point of machine learning is to find structure in data. What information can we extract? If data were perfect and there were no randomness involved, this would be trivial and the field would have been solved hundreds of years ago. That is not the world we live in. Data is noisy for all sorts of reasons (sloppy measurements, or variables we forgot to record), and we won't dwell on where the noise comes from. What we will do is find the best possible path through the noisy forest.

Let us consider the following problem. Suppose we go out and collect some data, and when we plot it on the x_1x_2 plane, we see that there seems to be a relationship between x_1 and x_2 (peep Figure 1.1). It looks like if we increase x_1, then x_2 also increases. But by how much? What is the relation? That is what we are going to answer in this chapter.

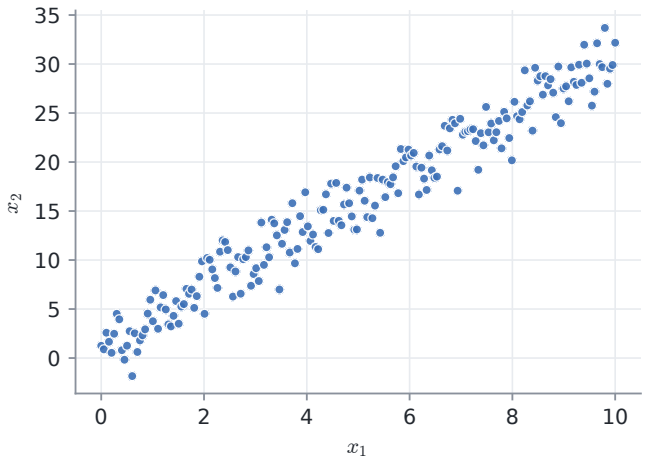


Figure 1.1: A dataset of 200 points. As x_1 goes up, x_2 seems to go up with it, but the points do not sit perfectly on any line.

1.2 Least Squares

The method that we are going to use is called least squares, and it relies on one major assumption, which is that **the relationship is linear**. Suppose there exists a line that passes through the "average" of all the data points, and we are using the word average loosely here. We want about half the points above the line and half below, so that the line tracks the average trend. If the relationship is linear, then there exists a function

$$x_2 = \theta_1 x_1 + \theta_0 \tag{1.1}$$

for some values of θ_1 and θ_0 that best capture the relationship between x_1 and x_2. From now on, we denote our input data by x_i, our outputs by y_i (so in this example y_i plays the role of x_2 for the i-th point), and our prediction by $\hat{y}_i = \theta_1 x_i + \theta_0$. The hat is statistics shorthand for "our best guess".

1.2.1 The Cost Function

We introduce a function that measures, roughly, the total squared distance between every data point and the line,

$$J(\theta_0, \theta_1) = \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{N} (y_i - \theta_1 x_i - \theta_0)^2 . \tag{1.2}$$

Each data point x_i has an output y_i, and we want the slope and intercept that make this as small as possible. Why squared? If we just added up $y_i - \hat{y}_i$, points above the line would cancel points below the line and we could get a cost of zero for a terrible fit. Squaring makes every term nonnegative. Another way to think about it is that we care about the distance, not the displacement. (We could use absolute values instead, and people do, but the square is smooth and gives us clean derivatives. There is also a much deeper reason that we will see in a moment.)

To find the minimum, we take derivatives and set them equal to zero. Starting with the slope,

$$\begin{aligned} \frac{\partial J}{\partial \theta_1} &= \sum_{i=1}^{N} \frac{\partial}{\partial \theta_1} (y_i - \theta_1 x_i - \theta_0)^2 \\ &= -2 \sum_{i=1}^{N} x_i (y_i - \theta_1 x_i - \theta_0) = 0. \end{aligned} \tag{1.3}$$

Dividing by -2 and distributing the sum, we get

$$\sum_{i=1}^{N} x_i y_i - \theta_1 \sum_{i=1}^{N} x_i^2 - \theta_0 \sum_{i=1}^{N} x_i = 0. \tag{1.4}$$

This still has the intercept θ_0 floating around in it, so we do the same thing for θ_0,

$$\begin{aligned} \frac{\partial J}{\partial \theta_0} &= -2 \sum_{i=1}^{N} (y_i - \theta_1 x_i - \theta_0) = 0 \\ \implies \quad \theta_0 &= \frac{1}{N} \sum_{i=1}^{N} y_i - \theta_1 \frac{1}{N} \sum_{i=1}^{N} x_i = \bar{y} - \theta_1 \bar{x}, \end{aligned} \tag{1.5}$$

where $\bar{x}$ and $\bar{y}$ are the sample means. That is a cute result on its own. It says that the best line always passes through the point $(\bar{x}, \bar{y})$, which is exactly the "average" that we were hand-waving about earlier.

Now we have two equations and two unknowns. In the first edition of these notes we threw

the system at Mathematica, but it is honestly not bad by hand. Substituting (1.5) into (1.4),

$$\begin{aligned} 0 &= \sum_i x_i y_i - \theta_1 \sum_i x_i^2 - (\bar{y} - \theta_1 \bar{x}) N \bar{x} \\ &= \Big(\sum_i x_i y_i - N \bar{x} \bar{y} \Big) - \theta_1 \Big(\sum_i x_i^2 - N \bar{x}^2 \Big) \\ &= \sum_i (x_i - \bar{x})(y_i - \bar{y}) - \theta_1 \sum_i (x_i - \bar{x})^2, \end{aligned} \tag{1.6}$$

which gives

$$\theta_1 = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^N (x_i - \bar{x})^2}, \qquad \theta_0 = \bar{y} - \theta_1 \bar{x}. \tag{1.7}$$

The slope is the sample covariance of x and y divided by the sample variance of x. If you expand everything out in raw sums, you get the gnarly version that you may have seen before,

$$\theta_1 = \frac{N \sum x_i y_i - (\sum x_i)(\sum y_i)}{N \sum x_i^2 - (\sum x_i)^2}, \qquad \theta_0 = \frac{\sum x_i^2 \sum y_i - \sum x_i \sum x_i y_i}{N \sum x_i^2 - (\sum x_i)^2}. \tag{1.8}$$

Same numbers, uglier shape. We prefer (1.7), both because it is easier to remember and because subtracting means first is numerically safer when the x_i are large.

> **Problem 1.1:**
>
> Show that the two forms of θ_1 above are equal. Then show that if we include an intercept, the residuals $e_i = y_i - \hat{y}_i$ of the least squares line always satisfy $\sum_i e_i = 0$ and $\sum_i x_i e_i = 0$. What goes wrong with the first identity if we force $\theta_0 = 0$?

To check proof of concept, we will use these formulas on our dataset. Here is the code that generates the data and fits the line.

```
import numpy as np

rng = np.random.default_rng(0)

# 200 points scattered around the line y = 3x + 1
x = np.linspace(0, 10, 200)
y = 3 * x + 1 + rng.normal(0, 2, size=x.size)

# closed-form least squares, the centered version
```

```
x_bar, y_bar = x.mean(), y.mean()
theta_1 = np.sum((x - x_bar) * (y - y_bar)) / np.sum((x - x_bar) ** 2)
theta_0 = y_bar - theta_1 * x_bar

print(f"theta_1 = {theta_1:.4f}, theta_0 = {theta_0:.4f}")
```

which prints

```
theta_1 = 2.9605, theta_0 = 1.2280
```

The true slope and intercept were 3 and 1, so we are pretty close. Peep Figure 1.2. We did not get exactly 3 and 1 because we only have 200 noisy points, and a different random seed would give us slightly different numbers. That wiggle from sample to sample is going to be a huge deal in Chapter 3.

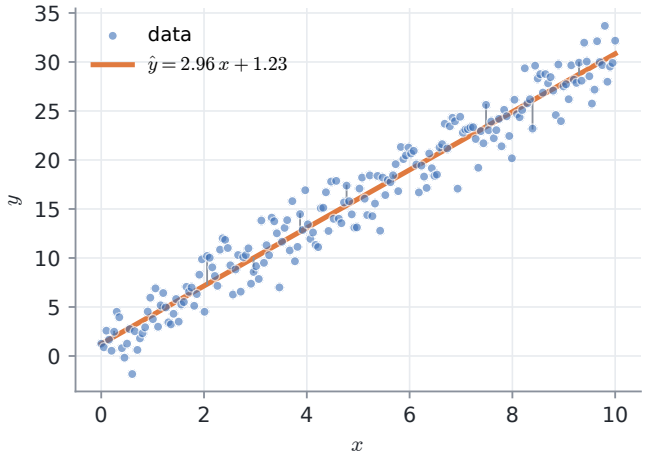


Figure 1.2: The least squares line from (1.7). The grey vertical segments are some of the residuals $y_i - \hat{y}_i$, and least squares makes the sum of their squares as small as possible.

1.2.2 Why Squares?

We said there was a deeper reason for squaring. Here it is, and it is going to come back over and over again in this book. Suppose the data really is generated by a line plus some Gaussian noise,

$$y_i = \theta_1 x_i + \theta_0 + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2) \text{ independent.} \tag{1.9}$$

Then each y_i, given x_i, is a Gaussian with mean $\theta_1 x_i + \theta_0$ and variance σ^2. The probability of seeing all of our data (the **likelihood**) is the product of the individual densities, and taking a log turns the product into a sum,

$$\begin{aligned}\log L(\theta_0, \theta_1) &= \sum_{i=1}^{N} \log \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \theta_1 x_i - \theta_0)^2}{2\sigma^2} \right) \right] \\ &= -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{N} (y_i - \theta_1 x_i - \theta_0)^2. \end{aligned} \tag{1.10}$$

The first term doesn't care about θ. So maximizing the likelihood is *exactly* the same as minimizing the sum of squares. Least squares is the maximum likelihood estimator when the noise is Gaussian. This is the bridge between "machine learning" and "statistics" that people like to pretend doesn't exist. Bishop builds his whole book on this idea [3], and we will lean on it heavily in Chapter 2 when the outputs stop being continuous.

> Problem 1.2:
>
> Under the Gaussian noise model above, find the maximum likelihood estimate of σ^2. Then redo the argument assuming the noise is Laplace distributed, $p(\varepsilon) = \frac{1}{2b} e^{-|\varepsilon|/b}$. Which cost function do you get, and why might it be less sensitive to outliers?

1.3 The Normal Equations

This is really great, we found a way to do this for two-dimensional data sets! But in real life, data is almost never two-dimensional like this. Modern models are trained on millions of data points living in spaces with thousands (sometimes millions) of dimensions. Computing derivatives one slope at a time like we just did is impractical. We must resort to vectorization.

Instead of computing derivatives one at a time, we will do them all simultaneously in any dimension. Suppose each data point now has M features, $x_{i1}, x_{i2}, \ldots, x_{iM}$, and our model is

$$\hat{y}_i = \theta_0 + \theta_1 x_{i1} + \theta_2 x_{i2} + \cdots + \theta_M x_{iM}. \tag{1.11}$$

Geometrically, our fit is no longer a line. It is an M-dimensional hyperplane sitting inside

the $(M+1)$-dimensional space of inputs and output. We stack everything into matrices,

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1M} \\ 1 & x_{21} & \cdots & x_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N1} & \cdots & x_{NM} \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}, \qquad \boldsymbol{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_M \end{bmatrix}, \tag{1.12}$$

so that all N predictions at once are just $\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$. The matrix $\mathbf{X}$ is called the **feature matrix** (or design matrix). Each row is one data point, and each column is one feature. Note the extra column of ones. Look back at (1.1), there was no data multiplying the intercept, so we give it a column of ones to multiply instead. Forgetting that column is the single most common bug in this whole chapter.

If we think about what the sum of squares is really doing, it is taking the dot product of the residual vector $\mathbf{y} - \hat{\mathbf{y}}$ with itself. So the cost function becomes

$$J(\boldsymbol{\theta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2. \tag{1.13}$$

J is still a scalar, it just eats a vector now. To minimize it, first expand the quadratic,

$$J(\boldsymbol{\theta}) = \mathbf{y}^T\mathbf{y} - \mathbf{y}^T\mathbf{X}\boldsymbol{\theta} - \boldsymbol{\theta}^T\mathbf{X}^T\mathbf{y} + \boldsymbol{\theta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\theta}. \tag{1.14}$$

Since $\mathbf{y}^T\mathbf{X}\boldsymbol{\theta}$ is a scalar, it equals its own transpose, $\mathbf{y}^T\mathbf{X}\boldsymbol{\theta} = (\mathbf{y}^T\mathbf{X}\boldsymbol{\theta})^T = \boldsymbol{\theta}^T\mathbf{X}^T\mathbf{y}$, and the two middle terms combine,

$$J(\boldsymbol{\theta}) = \mathbf{y}^T\mathbf{y} - 2\boldsymbol{\theta}^T\mathbf{X}^T\mathbf{y} + \boldsymbol{\theta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\theta}. \tag{1.15}$$

Now we take the gradient term by term. The two matrix calculus facts that we need are $\nabla_{\boldsymbol{\theta}}(\boldsymbol{\theta}^T\mathbf{a}) = \mathbf{a}$ and $\nabla_{\boldsymbol{\theta}}(\boldsymbol{\theta}^T\mathbf{A}\boldsymbol{\theta}) = (\mathbf{A} + \mathbf{A}^T)\boldsymbol{\theta}$, which is $2\mathbf{A}\boldsymbol{\theta}$ when $\mathbf{A}$ is symmetric. (If these feel like magic, write them out with indices once. They are just the product rule.) So

$$\nabla_{\boldsymbol{\theta}} J = -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\boldsymbol{\theta}. \tag{1.16}$$

Setting it equal to zero gives us

$$\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} = \mathbf{X}^T\mathbf{y}, \tag{1.17}$$

and if $\mathbf{X}^T\mathbf{X}$ is invertible, we can left multiply by its inverse,

$$\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}. \tag{1.18}$$

Equation (1.17) is called **the normal equations**. For $M = 1$, it reduces exactly to the two

equations we solved by hand in the previous section, so nothing new happened. We just stopped writing indices.

1.3.1 The Geometry, Or Why "Normal"

In the first edition we waved our hands and said the name comes from a normal projection, "blah blah blah". It is worth doing properly, because the picture is beautiful. Rewrite (1.17) as

$$\mathbf{X}^T(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) = \mathbf{0}. \tag{1.19}$$

This says the residual vector $\mathbf{e} = \mathbf{y} - \mathbf{X}\boldsymbol{\theta}$ is orthogonal to every column of $\mathbf{X}$. Think of $\mathbf{y}$ as a single vector in $\mathbb{R}^N$ (one coordinate per data point). The set of all possible predictions $\mathbf{X}\boldsymbol{\theta}$ is the column space of $\mathbf{X}$, a flat subspace of dimension $M+1$. Least squares picks the point in that subspace closest to $\mathbf{y}$, and the closest point is the one you get by dropping a perpendicular, that is, a *normal* line (Figure 1.3). The matrix that does this,

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T, \qquad \hat{\mathbf{y}} = \mathbf{H}\mathbf{y}, \tag{1.20}$$

is the orthogonal projection onto the column space. Statisticians call it the **hat matrix** because it puts the hat on $\mathbf{y}$. We used to prefer this geometric derivation, then we came around to calculus because it is systematic and generalizes to costs that aren't quadratic. Honestly, it is best to keep both pictures in your head.

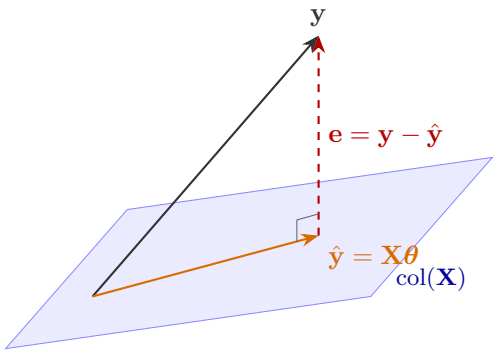


Figure 1.3: The geometry of least squares. The vector of outputs $\mathbf{y}$ lives in $\mathbb{R}^N$. The best prediction $\hat{\mathbf{y}}$ is its orthogonal projection onto the column space of $\mathbf{X}$, so the residual is normal to that plane.

Problem 1.3:

(1) Show that $\mathbf{X}^T\mathbf{X}$ is always symmetric and positive semi-definite, and that it is invertible if and only if the columns of $\mathbf{X}$ are linearly independent.
(2) Show that the hat matrix satisfies $\mathbf{H}^2 = \mathbf{H}$ and $\mathbf{H}^T = \mathbf{H}$. Interpret both identities geometrically.
(3) Conclude that J is convex, so the stationary point (1.18) is a global minimum and not just a critical point.

1.3.2 Solving It Without Inverting Anything

Equation (1.18) is what every textbook writes down, but in practice we should basically never compute it the way it is written. Explicitly forming $(\mathbf{X}^T\mathbf{X})^{-1}$ is slow and numerically fragile, since the condition number of $\mathbf{X}^T\mathbf{X}$ is the *square* of the condition number of $\mathbf{X}$. If two features are nearly collinear, you can lose half of your floating point digits before you start. Modern libraries solve least squares with a QR or SVD factorization of $\mathbf{X}$ directly, which is what `np.linalg.lstsq` does. If you really want the normal equations, use `np.linalg.solve`, which solves the linear system without forming the inverse.

```
# build the feature matrix: a column of ones, then the features
X = np.c_[np.ones_like(x), x]          # shape (N, 2)

# three ways to get theta, from worst to best
theta_inv   = np.linalg.inv(X.T @ X) @ X.T @ y       # textbook, avoid
theta_solve = np.linalg.solve(X.T @ X, X.T @ y)      # fine for small M
theta_lstsq, *_ = np.linalg.lstsq(X, y, rcond=None)  # QR/SVD, preferred

print("Theta:", theta_lstsq)
```

```
Theta: [1.22803035 2.96049919]
```

Same numbers as before, et voilà! The power of statistics. The beauty is that this exact code works for any M. Add more columns to `X` and nothing else changes.

1.4 Gradient Descent

We vectorized because our data can live in very high dimensional spaces. Even so, solving (1.17) costs roughly $O(NM^2)$ to form $\mathbf{X}^T\mathbf{X}$ plus $O(M^3)$ to factor it. When M is in the millions, M^3 is a number that makes computers cry. We bypass this using **gradient descent**.

Think back to multivariable calculus. The gradient of a function points in the direction of steepest ascent, so the negative gradient points in the direction of steepest descent. Our cost function J is a convex bowl in $(M+1)$-dimensional parameter space (Problem 1.3). If we keep taking small steps against the gradient, we will roll down the bowl and eventually land at the global minimum. The algorithm goes like this.

Algorithm 1: Gradient descent

Start at some initial guess $\boldsymbol{\theta}^{(0)}$ (zeros, or small random numbers);
for $t = 0, 1, 2, \ldots$ *until converged* **do**
  Compute the gradient $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^{(t)})$;
  Step downhill, $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}^{(t)})$;
end

The scaling factor α is called the **learning rate**, which just sets how big of a step we take. Our job now is to find the gradient. We already did it in matrix form in (1.16), but let us do it once with indices so the structure is clear. Write the cost as

$$J(\boldsymbol{\theta}) = \sum_{i=1}^{N} \Delta_i^2, \qquad \Delta_i = y_i - \theta_0 - \sum_{j=1}^{M} x_{ij}\theta_j, \tag{1.21}$$

where M is the number of features and N is the number of data points. The Δ_i are just the residuals. Taking a partial derivative with respect to one particular θ_k,

$$\begin{aligned} \frac{\partial J}{\partial \theta_k} &= \sum_{i=1}^{N} 2\Delta_i \frac{\partial \Delta_i}{\partial \theta_k} \\ &= \sum_{i=1}^{N} 2\Delta_i \left(-\sum_{j=1}^{M} x_{ij}\delta_{jk} \right) \\ &= -2\sum_{i=1}^{N} \Delta_i \, x_{ik}. \end{aligned} \tag{1.22}$$

The Kronecker delta δ_{jk} shows up because $\partial\theta_j/\partial\theta_k$ is 1 when the indices match and 0 otherwise, and it kills the sum over j. Notice there is exactly one free index left, k, so the gradient is a vector, as it should be. The intercept works the same way except the thing multiplying θ_0 is a 1,

$$\frac{\partial J}{\partial \theta_0} = -2\sum_{i=1}^{N} \Delta_i. \tag{1.23}$$

If we use the convention $x_{i0} = 1$ (the column of ones!), the intercept formula is just the

$k = 0$ case of the general one, and stacking them up gives

$$\begin{aligned}\nabla J &= \begin{bmatrix} \partial J/\partial \theta_0 \\ \partial J/\partial \theta_1 \\ \vdots \\ \partial J/\partial \theta_M \end{bmatrix} \\ &= -2 \begin{bmatrix} \sum_i \Delta_i \\ \sum_i x_{i1} \Delta_i \\ \vdots \\ \sum_i x_{iM} \Delta_i \end{bmatrix} \\ &= -2\, \mathbf{X}^T (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}), \end{aligned} \tag{1.24}$$

which matches (1.16). So the update rule is

$$\boldsymbol{\theta}^{\text{new}} = \boldsymbol{\theta}^{\text{old}} + 2\alpha\, \mathbf{X}^T (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}^{\text{old}}). \tag{1.25}$$

In practice we almost always divide the cost by N (use the *mean* squared error instead of the sum). That doesn't move the minimum at all, but it makes the size of the gradient independent of how much data we have, which means a good learning rate for 100 points is still a good learning rate for a million.

How do we know when to stop? A common rule is to stop when $\|\boldsymbol{\theta}^{(t+1)} - \boldsymbol{\theta}^{(t)}\|$ or the change in J is smaller than some tolerance that we pick. Another is to just run a fixed number of iterations and look at the curve. In practice, libraries ship optimizers and do the heavy lifting for us, so writing gradient descent by hand is a bit like reinventing the wheel. It is a wheel worth reinventing exactly once, though, because every model in this book (and every neural network you will ever train) is fit by some descendant of this loop.

```python
def gradient_descent(X, y, alpha=0.02, iterations=5000):
    """Least squares by batch gradient descent on the MSE."""
    N = len(y)
    theta = np.zeros(X.shape[1])        # start at the origin
    history = []
    for _ in range(iterations):
        residual = X @ theta - y          # this is minus Delta
        history.append(np.mean(residual ** 2))
        gradient = (2 / N) * X.T @ residual
        theta -= alpha * gradient
    return theta, np.array(history)

theta_gd, J_hist = gradient_descent(X, y)
```

```
print("Theta:", theta_gd)
```

```
Theta: [1.22803035 2.96049919]
```

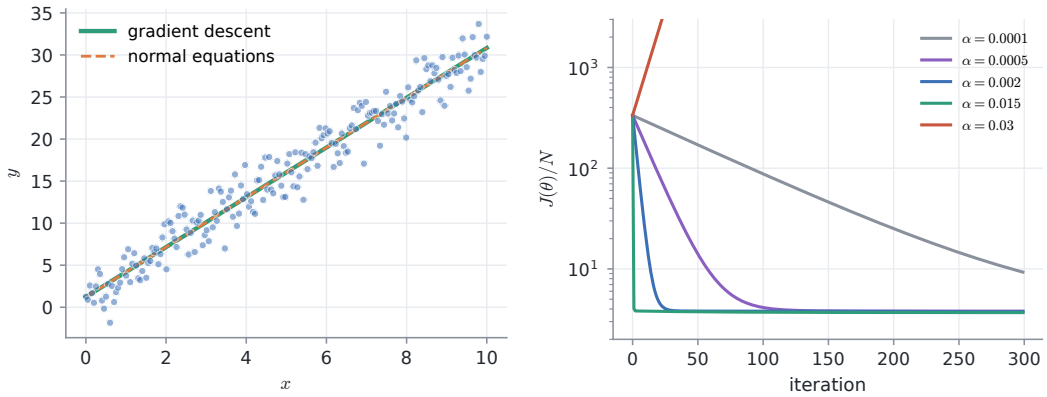


Figure 1.4: Left, the gradient descent fit lands right on top of the normal equations. Right, the mean squared error per iteration for several learning rates. Small α crawls, a good α converges in a few dozen steps, and $\alpha = 0.03$ is just past the stability limit and blows up.

We plot the results in Figure 1.4. The right panel is the more interesting one. For a fixed number of iterations, a smaller α gives a worse answer because each step is tiny and we haven't reached the bottom yet. Make α too big, though, and we overshoot the minimum, land on the other side of the bowl higher than where we started, and the whole thing diverges. For a quadratic cost we can say exactly where that cliff is. The MSE has Hessian $\frac{2}{N}\mathbf{X}^T\mathbf{X}$, and gradient descent is stable if and only if

$$\alpha < \frac{2}{\lambda_{\max}\left(\frac{2}{N}\mathbf{X}^T\mathbf{X}\right)}, \tag{1.26}$$

where $\lambda_{\max}$ is the largest eigenvalue. For our data this works out to about 0.029, which is why 0.03 explodes.

Problem 1.4:

Let $\mathbf{A} = \frac{2}{N}\mathbf{X}^T\mathbf{X}$ and let $\boldsymbol{\theta}^\star$ be the least squares solution. Show that the gradient descent error $\mathbf{e}^{(t)} = \boldsymbol{\theta}^{(t)} - \boldsymbol{\theta}^\star$ satisfies $\mathbf{e}^{(t+1)} = (\mathbf{I} - \alpha\mathbf{A})\mathbf{e}^{(t)}$. By diagonalizing $\mathbf{A}$, prove the stability condition above, and show that the slowest direction converges at rate $|1 - \alpha\lambda_{\min}|$ per step. Check the bound 0.029 numerically for our dataset.

1.4.1 Feature Scaling

Problem 1.4 has a sneaky consequence. The fastest we can go is limited by the *largest* eigenvalue, but the slowest direction converges at a rate set by the *smallest* one. If the ratio $\lambda_{\max}/\lambda_{\min}$ (the condition number) is big, the cost surface is a long skinny valley. Gradient descent bounces between the steep walls while crawling along the flat floor. Look at the left panel of Figure 1.5. Our feature x runs from 0 to 10, so the column of x values is much "bigger" than the column of ones, and the valley is stretched.

The fix is embarrassingly simple. Standardize every feature, $x \to (x - \bar{x})/s_x$, before training. Now every direction has roughly the same curvature, the bowl is round, and the negative gradient points straight at the minimum (right panel). This is why `StandardScaler` shows up at the top of practically every sklearn pipeline, and it is a big part of why tricks like batch normalization help deep networks train. Same idea, fancier clothes.

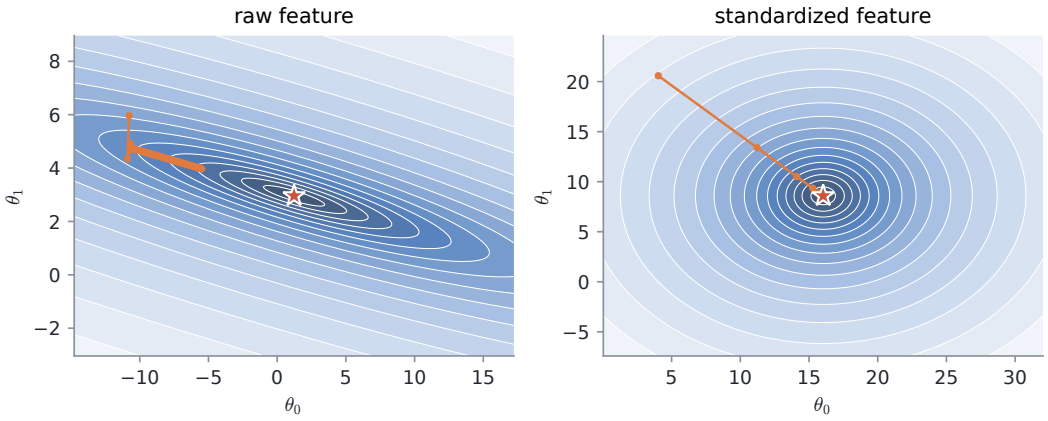


Figure 1.5: Contours of the cost with the gradient descent path (orange) and the minimum (star). With the raw feature the valley is long and narrow, and the path zig-zags. After standardizing the feature the contours are circles and descent heads straight for the bottom.

1.4.2 Stochastic And Mini-Batch Gradient Descent

There is one more problem with the loop above. Each step uses all N data points to compute the gradient. If N is a billion, a single step is expensive, and we need thousands of steps. Here is the modern trick that makes everything from linear regression to large language models trainable. The gradient is a sum over data points, so a random subset gives an unbiased estimate of it. Pick a random **mini-batch** B of, say, 32 points, and use

$$\nabla J \approx \frac{2}{|B|} \sum_{i \in B} x_{ik} \, (\hat{y}_i - y_i). \tag{1.27}$$

So now each step is noisy, but it is also $N/|B|$ times cheaper, and since the noise averages out over many steps, we come out ahead. When $|B| = 1$ this is classic **stochastic gradient descent** (SGD). One pass through the whole dataset is called an **epoch**. In practice the noise even helps, since it jiggles the parameters out of bad spots in non-convex problems, which will matter a lot once we get to neural networks.

```
def minibatch_sgd(X, y, alpha=0.01, epochs=50, batch_size=32, seed=0):
    rng = np.random.default_rng(seed)
    N = len(y)
    theta = np.zeros(X.shape[1])
    for epoch in range(epochs):
        order = rng.permutation(N)              # reshuffle every epoch
        for start in range(0, N, batch_size):
            idx = order[start:start + batch_size]
            residual = X[idx] @ theta - y[idx]
            theta -= alpha * (2 / len(idx)) * X[idx].T @ residual
    return theta

# standardize first, then fit (see feature scaling above)
x_std = (x - x.mean()) / x.std()
X_std = np.c_[np.ones_like(x_std), x_std]
print("Theta (standardized units):", minibatch_sgd(X_std, y))
```

```
Theta (standardized units): [15.99218101  8.55474328]
```

These are in standardized units, so the slope is θ_1/s_x in the original units. Converting back recovers the slope of roughly 2.96 (try it).

> Problem 1.5:
>
> Write the formulas that convert (θ_0, θ_1) fitted on standardized x back to the original units. Then run `minibatch_sgd` with batch sizes 1, 8, 32 and 200, keeping the number of epochs fixed, and plot the MSE after each epoch. Which batch size reaches a given error with the fewest *gradient evaluations of single data points*? Which reaches it with the fewest *parameter updates*?

1.5 Regularization

So far we have quietly assumed that $\mathbf{X}^T\mathbf{X}$ is invertible. For larger datasets with many features, this can fail. Two features might be highly correlated (height in inches and height

in centimeters is the silly example, the number of bedrooms and the square footage of a house is a realistic one), which makes two *columns* of the feature matrix nearly parallel. Or we may simply have more features than data points, $M + 1 > N$. In either case $\mathbf{X}^T\mathbf{X}$ has eigenvalues at or near zero. Since the inverse divides by those eigenvalues, the solution blows up, and tiny changes in the data cause wild swings in $\boldsymbol{\theta}$.

What we will do is **ridge regression**, which means adding a penalty $\lambda \sum_{k=1}^{M} \theta_k^2$ that punishes θ values that are very large. Begin with the cost function

$$J(\boldsymbol{\theta}) = \sum_{i=1}^{N} \left(y_i - \theta_0 - \sum_{k=1}^{M} x_{ik}\theta_k \right)^2 + \lambda \sum_{k=1}^{M} \theta_k^2. \tag{1.28}$$

Note that the penalty sum starts at $k = 1$. We don't penalize the intercept, since shifting all the y values up by a constant shouldn't change the slopes. In matrix form, with $\mathbf{I}_0$ the identity matrix but with a zero in the top left corner,

$$J(\boldsymbol{\theta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) + \lambda \boldsymbol{\theta}^T \mathbf{I}_0 \boldsymbol{\theta}. \tag{1.29}$$

(If you center $\mathbf{y}$ and the features first, the intercept drops out and $\mathbf{I}_0$ becomes the plain identity, which is what most derivations quietly assume.) Expanding and taking the gradient exactly as before,

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} J &= -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} + 2\lambda \mathbf{I}_0 \boldsymbol{\theta} = 0 \\ \implies \quad (\mathbf{X}^T\mathbf{X} + \lambda \mathbf{I}_0)\boldsymbol{\theta} &= \mathbf{X}^T\mathbf{y}, \end{aligned} \tag{1.30}$$

so that

$$\boldsymbol{\theta}_{\text{ridge}} = (\mathbf{X}^T\mathbf{X} + \lambda \mathbf{I}_0)^{-1}\mathbf{X}^T\mathbf{y}. \tag{1.31}$$

The effect of the penalty is to add λ to the diagonal. If $\mathbf{X}^T\mathbf{X}$ had eigenvalues $\sigma_1^2, \ldots, \sigma_M^2$ (some possibly zero), the regularized matrix has eigenvalues $\sigma_k^2 + \lambda$, which are all at least $\lambda > 0$. So the matrix is always invertible, and the inversion is stable. The price is that we no longer get the unbiased least squares answer. We are deliberately pulling the coefficients toward zero, trading a bit of bias for a lot less variance. That trade is the whole story of Chapter 3.

Ridge also has a Bayesian reading that we will make precise in Chapter 7. Remember that least squares is maximum likelihood under Gaussian noise. If, on top of that, we put a Gaussian prior on the weights, $\theta_k \sim \mathcal{N}(0, \tau^2)$, then the most probable $\boldsymbol{\theta}$ after seeing the data is exactly the ridge solution with $\lambda = \sigma^2/\tau^2$. The penalty is a belief that weights are probably small.

Problem 1.6:

(1) (Ridge as fake data.) Assume centered data so there is no intercept. Show that ridge regression is ordinary least squares on an augmented dataset, where we stack $\sqrt{\lambda}\,\mathbf{I}$ under $\mathbf{X}$ and M zeros under $\mathbf{y}$.

(2) (Shrinkage.) Let $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$ be the thin SVD with singular values s_k. Show that

$$\boldsymbol{\theta}_{\text{ridge}} = \sum_k \frac{s_k}{s_k^2 + \lambda} (\mathbf{u}_k^T \mathbf{y})\, \mathbf{v}_k.$$

Directions with small singular values (the ones causing trouble) get shrunk the most. What happens as $\lambda \to 0$ and as $\lambda \to \infty$?

1.5.1 Gradient Descent With Ridge

For the same reasons we wanted gradient descent for plain least squares, we want it for ridge. The only thing that changes is the gradient of the penalty term,

$$\frac{\partial}{\partial \theta_k} \left(\lambda \sum_{j=1}^{M} \theta_j^2 \right) = 2\lambda\theta_k, \qquad k \geq 1, \tag{1.32}$$

and everything else we already computed. So

$$\nabla J = \begin{bmatrix} \partial J/\partial \theta_0 \\ \partial J/\partial \theta_1 \\ \vdots \\ \partial J/\partial \theta_M \end{bmatrix} = \begin{bmatrix} -2\sum_i \Delta_i \\ -2\sum_i x_{i1}\Delta_i + 2\lambda\theta_1 \\ \vdots \\ -2\sum_i x_{iM}\Delta_i + 2\lambda\theta_M \end{bmatrix}. \tag{1.33}$$

Plug this into the update and look at what the penalty does on its own,

$$\theta_k \leftarrow (1 - 2\alpha\lambda)\,\theta_k + 2\alpha \sum_i x_{ik}\Delta_i. \tag{1.34}$$

Every step, each weight first gets multiplied by a number slightly less than one, and then gets the usual data-driven nudge. The weights literally decay toward zero unless the data keeps pushing them up. That is why, in the deep learning world, ridge goes by the name **weight decay**, and it is still a default ingredient when training huge models (usually in the decoupled form of AdamW [9], which we will meet in Chapter 4). The modified code is below, with both the closed form and the gradient version.

```python
def ridge_regression(X, y, lambda_reg):
    """
    Closed-form ridge regression.
    X should NOT include the column of ones, we add it here.
    Returns coefficients with the intercept first.
    """
    n_samples, n_features = X.shape
    X_int = np.c_[np.ones(n_samples), X]

    I0 = np.eye(n_features + 1)
    I0[0, 0] = 0                    # do not penalize the intercept

    A = X_int.T @ X_int + lambda_reg * I0
    return np.linalg.solve(A, X_int.T @ y)


def ridge_gd(X, y, lambda_reg, alpha=0.01, iterations=5000):
    """Ridge by gradient descent on (1/N)||y - X theta||^2 + (lambda/N)||theta||^2."""
    N = len(y)
    X_int = np.c_[np.ones(N), X]
    theta = np.zeros(X_int.shape[1])
    for _ in range(iterations):
        grad = (2 / N) * X_int.T @ (X_int @ theta - y)
        grad[1:] += (2 * lambda_reg / N) * theta[1:]   # weight decay, skip intercept
        theta -= alpha * grad
    return theta

# two almost identical features: plain least squares goes haywire
rng = np.random.default_rng(3)
z = rng.normal(size=50)
X2 = np.c_[z, z + 1e-3 * rng.normal(size=50)]
y2 = 2 * z + rng.normal(0, 0.1, size=50)

print("least squares:", ridge_regression(X2, y2, 0.0))
print("ridge, lam=1 :", ridge_regression(X2, y2, 1.0))
print("ridge, GD    :", ridge_gd(X2, y2, 1.0))
```

```
least squares: [ 0.01559706 -9.50441085 11.51216271]
ridge, lam=1 : [0.01442336 0.99576506 0.99642634]
ridge, GD    : [0.01442336 0.99580261 0.9963888 ]
```

Look at that. Least squares assigns -9.5 to one copy of the feature and $+11.5$ to the other, two huge numbers that nearly cancel. Technically it fits the training data, but it is a terrible, fragile model. Ridge notices the two features carry the same information and splits the credit evenly, about 1 each, for a total effect of 2. That is exactly the truth.

Problem 1.7:

Using the collinear dataset above, plot each ridge coefficient as a function of λ on a log scale from 10^{-6} to 10^{3} (this is called a **regularization path**). At roughly what λ do the coefficients stabilize? Now refit on a fresh random draw of the data (change the seed) for both $\lambda = 0$ and $\lambda = 1$, and compare how much each set of coefficients moved.

We only used two-dimensional examples in this chapter for visualization purposes. Fear not, everything here was written for general M, so feel free to throw more columns into $\mathbf{X}$ and watch it still work. Next chapter we keep the linear machinery but change the question. Instead of predicting a number, we will predict a category.

2

Linear Classification

"Behold! A Man!"

– Diogenes, holding a plucked chicken

Linear regression was about using linear methods (namely linear algebra) to fit curves to continuous data. Now, what if the outputs were strictly categorical? Does this image show a 0 or a 1? Is this a dog or a cat? Is this a man or not a man (thanks, Diogenes)? This is **classification**, and it is probably the single most common task in applied machine learning. Let us set up a fake problem, shown in Figure 2.1.

Each point has two input features, x_1 and x_2, and a label telling us which cluster it came from. We want an algorithm that looks at a new point and says "class 0" or "class 1".

2.1 Why Not Linear Regression For Classification?

We built a whole toolbox for regression in the last chapter. Can we not use those same tools to do classification? The premature answer is yes, sort of. Let us encode the label as a number, $y_i \in \{0, 1\}$ (left or right, black or white, it doesn't matter as long as we can represent it as a binary number), and just run least squares, minimizing $\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$ as before. Our predictions $\hat{y}$ will still be continuous, since that is what linear regression produces. We force them to be binary by thresholding, so if $\hat{y} \geq 0.5$ we say class 1, and

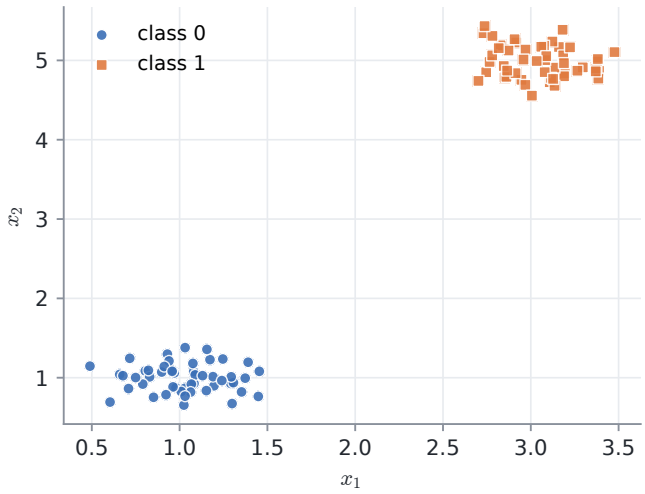


Figure 2.1: Two clusters of points, one centered at $(1, 1)$ and the other at $(3, 5)$. We want a rule that tells us which cluster a new point belongs to.

class 0 otherwise. Since this is just regression, the normal equations hand us the answer,

$$\boldsymbol{\theta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}. \tag{2.1}$$

Being the good data scientists that we are, we have to prepare the data before stabbing it with least squares. First we build the feature matrix $\mathbf{X}$. The first column is the intercept column of all ones, the second column holds the x_1 coordinates, and the third holds the x_2 coordinates,

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} \\ 1 & x_{21} & x_{22} \\ \vdots & \vdots & \vdots \\ 1 & x_{N1} & x_{N2} \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 1 \end{bmatrix}, \tag{2.2}$$

where 0 means the cluster at $(1, 1)$ and 1 means the cluster at $(3, 5)$. In the first edition we called the second coordinate y, which was a crime, since y is already the label. From here on the inputs are always x's.

Before doing anything, we split the data into a **training set** and a **testing set**. We learn $\boldsymbol{\theta}$ on the training set only, then use it to predict the test set and compare with the true labels. Evaluating on data the model has never seen is the only honest way to measure how good it is, and this habit will save us in Chapter 3. Here is the code.

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split

np.random.seed(0)

# two clusters, around (1, 1) and (3, 5)
x1_a = np.random.normal(loc=1, scale=0.2, size=50)
x2_a = np.random.normal(loc=1, scale=0.2, size=50)
x1_b = np.random.normal(loc=3, scale=0.2, size=50)
x2_b = np.random.normal(loc=5, scale=0.2, size=50)

x1 = np.concatenate([x1_a, x1_b])
x2 = np.concatenate([x2_a, x2_b])
labels = np.array([0] * 50 + [1] * 50)

# feature matrix with intercept column
X = np.c_[np.ones(x1.size), x1, x2]

X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.3, random_state=0)

# least squares on the training set
theta, *_ = np.linalg.lstsq(X_train, y_train, rcond=None)

def predict_ls(X, theta):
    return (X @ theta >= 0.5).astype(int)

y_pred = predict_ls(X_test, theta)
print("test accuracy:", np.mean(y_pred == y_test))
```

```
test accuracy: 1.0
```

The accuracy is 100%! That's pretty good, right? (Figure 2.2.) Yes, but this problem is easy. The clusters are far apart and nicely round. There are two big problems hiding here.

The first is conceptual. The outputs $\hat{y}$ can be any real number, like 1.7 or -0.3, which don't mean anything for a yes or no question. It would be much nicer if the output were a probability.

The second is practical and a lot nastier. Least squares punishes points for being far from the target value, *even when they are on the correct side*. Suppose we add some class-1 points that are very obviously class 1, way out in the corner. They are easy to classify, so they should not change anything. But each of them has $\hat{y}$ much larger than 1, which least squares sees as a big error, so it tilts the whole boundary to reduce those "errors". Peep Figure 2.3. Adding *easy* points made least squares misclassify a bunch of points it used to get right.

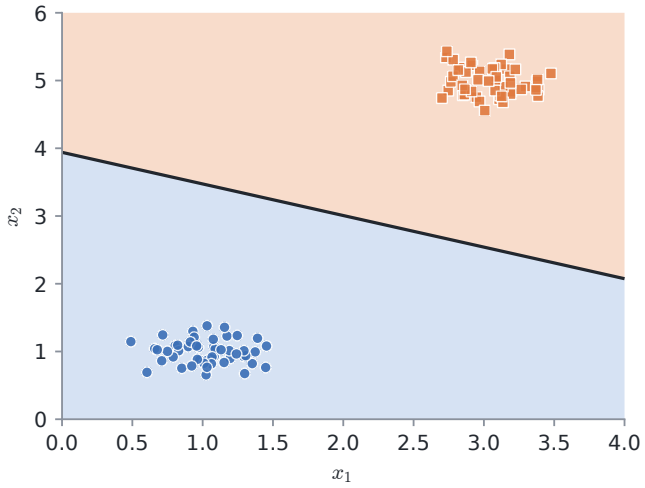


Figure 2.2: The least squares classifier. The black line is where $\mathbf{x}^T\boldsymbol{\theta} = 0.5$, and the shading shows which class each region of the plane is assigned to.

(This is the example Bishop uses in Section 4.1.3 of [3], and it convinced us for life.) Least squares also struggles with more than two classes, since one line can only split the plane into two regions. We need a better idea.

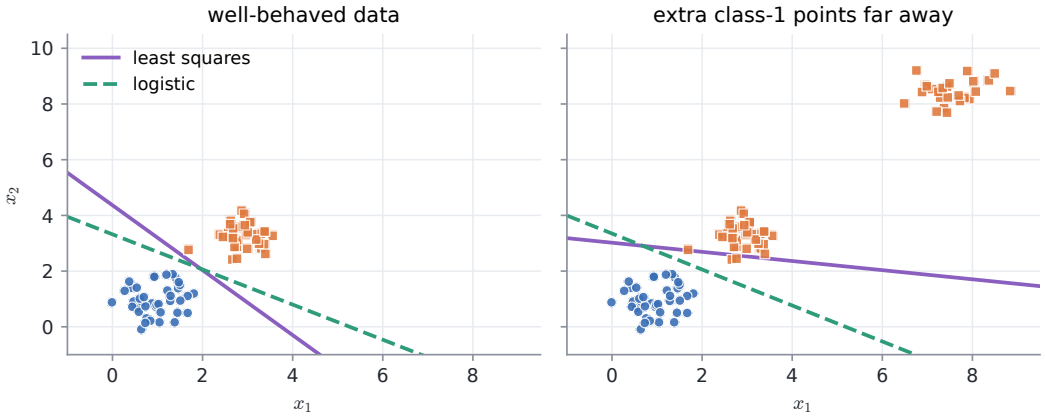


Figure 2.3: Left, on clean data least squares (purple) and logistic regression (green, dashed) both separate the classes. Right, we add class-1 points that are far from the boundary and completely unambiguous. Least squares tilts its boundary toward them and starts misclassifying orange points. Logistic regression barely moves.

2.2 Logistic Regression

The problem with least squares classification is that the linear output is unbounded. We want to squash the output so that it lives between 0 and 1 and can be read as a probability. A function that does this is the **logistic function** (also called the sigmoid),

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \tag{2.3}$$

which we plot in Figure 2.4. When z is very negative, e^{-z} is huge and $\sigma(z) \approx 0$. When z is very positive, $e^{-z} \approx 0$ and $\sigma(z) \approx 1$. At $z = 0$ we get exactly $1/2$.

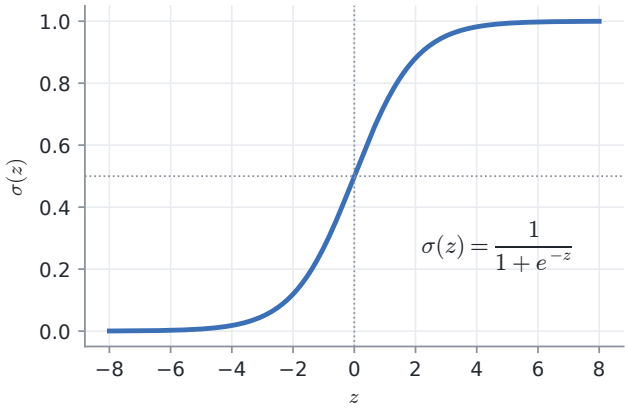


Figure 2.4: The logistic function smoothly maps the whole real line into $(0, 1)$.

We use this as the conditional probability that a point $\mathbf{x}$ belongs to class 1,

$$\mathbb{P}(y = 1 \mid \mathbf{x}) = \sigma(z), \qquad \mathbb{P}(y = 0 \mid \mathbf{x}) = 1 - \sigma(z), \tag{2.4}$$

where $z = \theta_0 + \theta_1 x_1 + \cdots + \theta_M x_M = \boldsymbol{\theta}^T \mathbf{x}$ (with the usual $x_0 = 1$ hiding the intercept inside $\boldsymbol{\theta}$). So we still compute a linear function of the inputs, we just pass it through σ at the end. If you invert the sigmoid, you get

$$z = \log \frac{\mathbb{P}(y = 1 \mid \mathbf{x})}{\mathbb{P}(y = 0 \mid \mathbf{x})}, \tag{2.5}$$

so the linear part is modeling the **log-odds**. Statisticians call z the **logit**, and deep learning people kept the name for "whatever number goes into the final sigmoid or softmax".

Problem 2.1:

Prove the following properties of the logistic function. They are used constantly, so it is worth having them at your fingertips.
(1) $1 - \sigma(z) = \sigma(-z)$.
(2) $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$.
(3) $\log \sigma(z) = -\log(1 + e^{-z})$ and $\log(1 - \sigma(z)) = -z - \log(1 + e^{-z})$.

2.2.1 The Likelihood

In Chapter 1 we saw that least squares is secretly maximum likelihood with Gaussian noise. Now we do maximum likelihood on purpose. A label $y \in \{0, 1\}$ is a coin flip whose probability of heads is $\sigma(\boldsymbol{\theta}^T \mathbf{x})$, that is, a Bernoulli random variable. We can compact the two conditional probabilities into one expression,

$$\mathbb{P}(y \mid \mathbf{x}, \boldsymbol{\theta}) = \sigma(\boldsymbol{\theta}^T \mathbf{x})^y \big(1 - \sigma(\boldsymbol{\theta}^T \mathbf{x})\big)^{1-y}. \tag{2.6}$$

Check it. If $y = 1$ the second factor is raised to the zero and disappears, leaving σ. If $y = 0$ the first factor disappears, leaving $1 - \sigma$. If we assume that the data points are independent of each other, the probability of the whole dataset is the product

$$L(\boldsymbol{\theta}) = \prod_{i=1}^{N} \sigma(\boldsymbol{\theta}^T \mathbf{x}_i)^{y_i} \big(1 - \sigma(\boldsymbol{\theta}^T \mathbf{x}_i)\big)^{1-y_i}. \tag{2.7}$$

This is called the **likelihood function** of $\boldsymbol{\theta}$. Products of probabilities are really annoying, analytically (product rule everywhere) and computationally (multiply a thousand numbers smaller than one and your computer rounds the answer to zero). So we take the log, which turns products into sums. Writing $z_i = \boldsymbol{\theta}^T \mathbf{x}_i$ and using Problem 2.1,

$$\begin{aligned} \ell(\boldsymbol{\theta}) = \log L(\boldsymbol{\theta}) &= \sum_{i=1}^{N} \Big[y_i \log \sigma(z_i) + (1 - y_i) \log \big(1 - \sigma(z_i)\big) \Big] \\ &= \sum_{i=1}^{N} \Big[-y_i \log(1 + e^{-z_i}) + (1 - y_i)\big(-z_i - \log(1 + e^{-z_i})\big) \Big] \\ &= \sum_{i=1}^{N} \Big[-\log(1 + e^{-z_i}) - (1 - y_i)\, z_i \Big]. \end{aligned} \tag{2.8}$$

(The first edition had a sign error on that last term. It happened to be fixed by a second sign error two lines later, which is the most physics thing that has ever happened in a machine

learning book.) Now use the identity $-\log(1+e^{-z}) = z - \log(1+e^{z})$, which you get by multiplying the inside of the log by e^{z}/e^{z}. Substituting,

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{N} \Big[y_i \, \boldsymbol{\theta}^T \mathbf{x}_i - \log\left(1 + e^{\boldsymbol{\theta}^T \mathbf{x}_i}\right) \Big]. \tag{2.9}$$

This is the **log-likelihood**. What exactly are we trying to do? We want the $\boldsymbol{\theta}$ that best fits the data, which is the same thing as finding the **maximum likelihood estimator** of $\boldsymbol{\theta}$. One problem though. Setting the derivative to zero gives a transcendental equation (the θ's are trapped inside exponentials), so unlike least squares there is no closed form. Our only hope is iterative optimization.

2.2.2 Cross-Entropy, The Loss Everyone Uses

Optimizers like to minimize, so in practice we flip the sign and average. The negative log-likelihood per data point,

$$J(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^{N} \Big[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \Big], \qquad \hat{p}_i = \sigma(\boldsymbol{\theta}^T \mathbf{x}_i), \tag{2.10}$$

is called the **binary cross-entropy**. It is the same thing as $-\ell/N$, just written in terms of predicted probabilities. Look at what it does to a single point with $y_i = 1$. The cost is $-\log \hat{p}_i$. If we are confident and right ($\hat{p}_i \to 1$), the cost goes to zero. If we are confident and *wrong* ($\hat{p}_i \to 0$), the cost goes to infinity. And for points that are already correct with high confidence, the cost is basically zero no matter how far they sit from the boundary. That's exactly why logistic regression shrugs off the faraway points in Figure 2.3, while least squares freaks out.

2.2.3 Gradient Descent

Let us find the gradient of the log-likelihood (2.9), going term by term. The first term is linear in $\boldsymbol{\theta}$,

$$\nabla_{\boldsymbol{\theta}} \sum_{i=1}^{N} y_i \, \boldsymbol{\theta}^T \mathbf{x}_i = \sum_{i=1}^{N} y_i \, \mathbf{x}_i. \tag{2.11}$$

For the second term we use the chain rule,

$$\nabla_{\boldsymbol{\theta}} \left[-\sum_{i=1}^{N} \log\left(1 + e^{\boldsymbol{\theta}^T \mathbf{x}_i}\right) \right] = -\sum_{i=1}^{N} \frac{e^{\boldsymbol{\theta}^T \mathbf{x}_i}}{1 + e^{\boldsymbol{\theta}^T \mathbf{x}_i}} \, \mathbf{x}_i = -\sum_{i=1}^{N} \sigma(\boldsymbol{\theta}^T \mathbf{x}_i) \, \mathbf{x}_i, \tag{2.12}$$

where we simplified with $e^z/(1+e^z) = 1/(1+e^{-z}) = \sigma(z)$. Combining both terms,

$$\nabla_{\boldsymbol{\theta}}\, \ell(\boldsymbol{\theta}) = \sum_{i=1}^{N} \big(y_i - \sigma(\boldsymbol{\theta}^T \mathbf{x}_i)\big)\, \mathbf{x}_i = \mathbf{X}^T(\mathbf{y} - \hat{\mathbf{p}}). \tag{2.13}$$

Stare at this for a second, because it is gorgeous. Compare it with the least squares gradient, $\mathbf{X}^T(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})$. It has the exact same shape, "features times (truth minus prediction)", the only change being that the prediction is now a probability. This is not a coincidence. Both are members of the family of generalized linear models with their natural link function, and they all share this form. We will see it a third time in a few pages.

Since we are maximizing ℓ, we walk *up* the gradient (gradient ascent), or equivalently walk down the gradient of the cross-entropy,

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \sum_{i=1}^{N} \big(y_i - \sigma(\boldsymbol{\theta}^T \mathbf{x}_i)\big)\mathbf{x}_i. \tag{2.14}$$

If we were very fancy, we would slap on an L_2 penalty $\frac{\lambda}{2}\|\boldsymbol{\theta}\|^2$ like ridge regression (again, skipping the intercept), and the update becomes

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \left(\sum_{i=1}^{N} \big(y_i - \sigma(\boldsymbol{\theta}^T \mathbf{x}_i)\big)\mathbf{x}_i - \lambda \boldsymbol{\theta} \right). \tag{2.15}$$

Once we have the optimal $\boldsymbol{\theta}$, we classify new points using (2.6). If $\mathbb{P}(y = 1 \mid \mathbf{x}) \geq 0.5$ we choose class 1, else class 0.

> Problem 2.2:
>
> Show that the Hessian of the cross-entropy is $\nabla^2 J = \frac{1}{N}\mathbf{X}^T\mathbf{S}\mathbf{X}$ with $\mathbf{S} = \operatorname{diag}\big(\hat{p}_i(1-\hat{p}_i)\big)$. Conclude that J is convex, so gradient descent cannot get stuck in a bad local minimum. Then implement **Newton's method**, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - (\nabla^2 J)^{-1}\nabla J$, for logistic regression. Statisticians call this iteratively reweighted least squares (IRLS) because each step solves a weighted least squares problem. How many iterations does it need on our data compared with gradient descent?

2.2.4 Why Is This A Linear Classifier?

It is a bit weird to discuss a model with an exponential in it under the heading "linear classification", isn't it? Not at all! What makes a classifier linear is that its **decision**

boundary is linear. We predict class 1 when $\sigma(z) \geq 0.5$, and since σ is increasing with $\sigma(0) = 1/2$, that happens exactly when

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_M x_M \geq 0. \tag{2.16}$$

So the boundary is the set where $\boldsymbol{\theta}^T \mathbf{x} = 0$, which is a hyperplane. The sigmoid only changes how confident we are as we move away from it. The vector $(\theta_1, \ldots, \theta_M)$ is perpendicular to that hyperplane, and its length controls how sharply the probability goes from 0 to 1 as we cross it. Keep that picture in mind for support vector machines in Chapter 6.

2.2.5 An Example

We all love proofs of concept, so let us do one. We build the feature matrix as usual (don't forget the bias column!), create the labels, split, and train with gradient descent and L_2 regularization. One modern detail is that we never compute $\log \sigma(z)$ by literally taking the log of the sigmoid. For $z = -800$, `1/(1+np.exp(800))` overflows. Every serious library computes the loss directly from the logits with the stable identity $\log(1+e^z) = \max(z, 0) + \log(1+e^{-|z|})$, which `np.logaddexp(0, z)` does for us.

```
from sklearn.metrics import accuracy_score

np.random.seed(0)
x1_a = np.random.normal(1, 0.3, 50); x2_a = np.random.normal(1, 0.3, 50)
x1_b = np.random.normal(3, 0.5, 50); x2_b = np.random.normal(5, 0.5, 50)
X = np.c_[np.ones(100), np.r_[x1_a, x1_b], np.r_[x2_a, x2_b]]
labels = np.array([0] * 50 + [1] * 50)

X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.3, random_state=0)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def bce_from_logits(z, y):
    # mean of log(1 + e^z) - y z, computed without overflow
    return np.mean(np.logaddexp(0, z) - y * z)

def logistic_regression_l2(X, y, learning_rate=0.1, iterations=1000, lambda_=0.1):
    N, n = X.shape
    theta = np.zeros(n)
    for _ in range(iterations):
        p_hat = sigmoid(X @ theta)
        gradient = X.T @ (p_hat - y) / N     # gradient of the cross-entropy
        penalty = (lambda_ / N) * theta
```

```
        penalty[0] = 0.0                    # do not regularize the intercept
        theta -= learning_rate * (gradient + penalty)
    return theta

theta = logistic_regression_l2(X_train, y_train)
p_test = sigmoid(X_test @ theta)
print("theta:", theta)
print("test loss:", bce_from_logits(X_test @ theta, y_test))
print(f"test accuracy: {accuracy_score(y_test, p_test >= 0.5) * 100:.2f}%")
```

```
theta: [-5.02823691 -0.15071673  2.03829596]
test loss: 0.026512646371911253
test accuracy: 100.00%
```

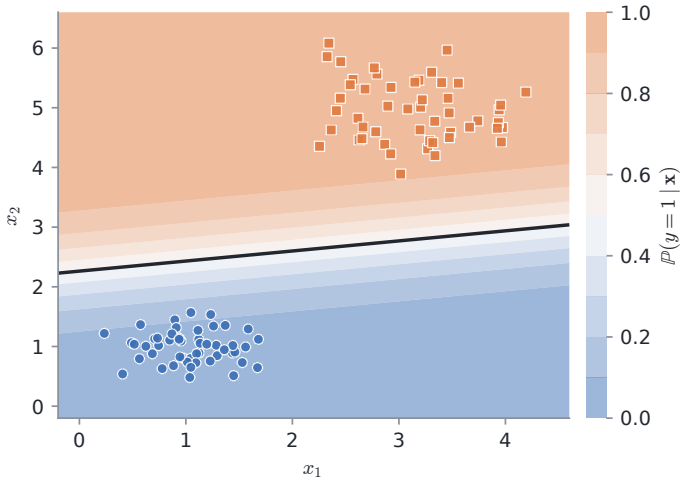


Figure 2.5: Logistic regression with L_2 regularization on the same kind of data as before. The black line is the decision boundary $\boldsymbol{\theta}^T \mathbf{x} = 0$, and the shading is the predicted probability of class 1, which changes smoothly instead of jumping.

We plot the predicted probabilities in Figure 2.5. Compare it with Figure 2.2. The boundary is similar, but now every point in the plane comes with a confidence, and points near the line get probabilities near one half. That calibrated uncertainty is often worth more than the label itself.

2.2.6 Separable Data

Here is something sneaky. Our two clusters are *perfectly* separable, there is a line with all the blue points on one side and all the orange on the other. Take any such separating $\boldsymbol{\theta}$ and multiply it by 2. The boundary doesn't move, but every probability gets pushed

closer to 0 or 1, so every term in the likelihood gets closer to 1. Multiply by 10, even better. The likelihood keeps increasing forever as $\|\boldsymbol{\theta}\| \to \infty$, which means the maximum likelihood estimator does not exist! Unregularized gradient descent on separable data never converges, it just slowly grows $\boldsymbol{\theta}$ (in a fixed direction, it turns out, the max-margin direction from Chapter 6). The L_2 penalty fixes this by making huge weights expensive. This is one more reason why regularization is a default and not an afterthought.

> Problem 2.3:
>
> Run `logistic_regression_l2` with `lambda_=0` for 10^3, 10^4 and 10^5 iterations and print $\|\boldsymbol{\theta}\|$ and the training loss each time. Then do the same with `lambda_=0.1`. Explain what you see using the argument above. Finally, normalize $\boldsymbol{\theta}/\|\boldsymbol{\theta}\|$ in the unregularized runs. Does the *direction* settle down?

2.3 Multiclass Classification

Once we understand logistic regression with two classes, doing K classes is a generalization of the two-class case. We encode the label as a **one-hot** vector $\mathbf{y}_i \in \{0,1\}^K$, with a single 1 in the position of the true class. So with $K = 3$, class 2 is $(0, 1, 0)$. Each class k gets its own parameter vector $\boldsymbol{\theta}_k$, and therefore its own logit $z_k = \boldsymbol{\theta}_k^T \mathbf{x}$. To turn K logits into K probabilities, we exponentiate (so everything is positive) and normalize (so they sum to one),

$$\mathbb{P}(y = k \mid \mathbf{x}) = \frac{\exp(\boldsymbol{\theta}_k^T \mathbf{x})}{\sum_{j=1}^{K} \exp(\boldsymbol{\theta}_j^T \mathbf{x})} \equiv p_k(\mathbf{x}). \tag{2.17}$$

This is called the **softmax** function, and it basically plays the same role that $\sigma(z)$ played in logistic regression. The name comes from the fact that it is a smooth version of "put all the probability on the largest logit". Multiply all the logits by a large number and softmax turns into a hard argmax. It is also exactly the Boltzmann distribution from statistical mechanics, with $-z_k$ playing the role of energy, which is a fun thing to notice if you are a physicist and a confusing one otherwise.

The probability of a single observation is the multiclass version of (2.6),

$$\mathbb{P}(\mathbf{y} \mid \mathbf{x}) = \prod_{k=1}^{K} p_k(\mathbf{x})^{y_k}, \tag{2.18}$$

since only the factor for the true class survives (everything else is raised to the zero).

Assuming independence again, the log-likelihood of the whole dataset is

$$\begin{aligned}\ell(\boldsymbol{\theta}) &= \sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log p_k(\mathbf{x}_i) \\ &= \sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\left(\boldsymbol{\theta}_k^T\mathbf{x}_i - \log\sum_{j=1}^{K} e^{\boldsymbol{\theta}_j^T\mathbf{x}_i}\right) \\ &= \sum_{i=1}^{N}\left(\sum_{k=1}^{K} y_{ik}\,\boldsymbol{\theta}_k^T\mathbf{x}_i - \log\sum_{j=1}^{K} e^{\boldsymbol{\theta}_j^T\mathbf{x}_i}\right), \end{aligned} \tag{2.19}$$

where in the last step we used $\sum_k y_{ik} = 1$ (exactly one class is the true one). The negative of this, divided by N, is the **categorical cross-entropy**. Let us take the gradient with respect to one class's parameters $\boldsymbol{\theta}_k$. The first term gives $y_{ik}\mathbf{x}_i$. The log-sum-exp gives

$$\nabla_{\boldsymbol{\theta}_k}\log\sum_{j=1}^{K} e^{\boldsymbol{\theta}_j^T\mathbf{x}_i} = \frac{e^{\boldsymbol{\theta}_k^T\mathbf{x}_i}}{\sum_j e^{\boldsymbol{\theta}_j^T\mathbf{x}_i}}\,\mathbf{x}_i = p_k(\mathbf{x}_i)\,\mathbf{x}_i, \tag{2.20}$$

so

$$\nabla_{\boldsymbol{\theta}_k}\ell = \sum_{i=1}^{N}\big(y_{ik} - p_k(\mathbf{x}_i)\big)\,\mathbf{x}_i. \tag{2.21}$$

And there it is, the third time. Features times (truth minus prediction). If we stack the $\boldsymbol{\theta}_k$ as columns of a matrix $\boldsymbol{\Theta}$, the one-hot labels as rows of $\mathbf{Y}$ and the probabilities as rows of $\mathbf{P}$, all K gradients at once are $\mathbf{X}^T(\mathbf{Y} - \mathbf{P})$, which is one line of NumPy. As with logistic regression, there is no closed form, so we use gradient descent.

> Problem 2.4:
>
> (1) Show that adding the same vector $\mathbf{c}$ to every $\boldsymbol{\theta}_k$ leaves all the softmax probabilities unchanged. So softmax regression has one redundant parameter vector, and the regularizer is what picks a unique solution.
> (2) Use part 1 to set $\boldsymbol{\theta}_2 = \mathbf{0}$ when $K = 2$, and show that softmax regression reduces exactly to logistic regression with $\boldsymbol{\theta} = \boldsymbol{\theta}_1$.
> (3) Use part 1 with $\mathbf{c}$ chosen so the largest logit becomes zero, and explain why subtracting `z.max()` before exponentiating (the trick in the code below) changes nothing mathematically but prevents overflow.

Let us do a quick example with three clusters, shown in Figure 2.6.

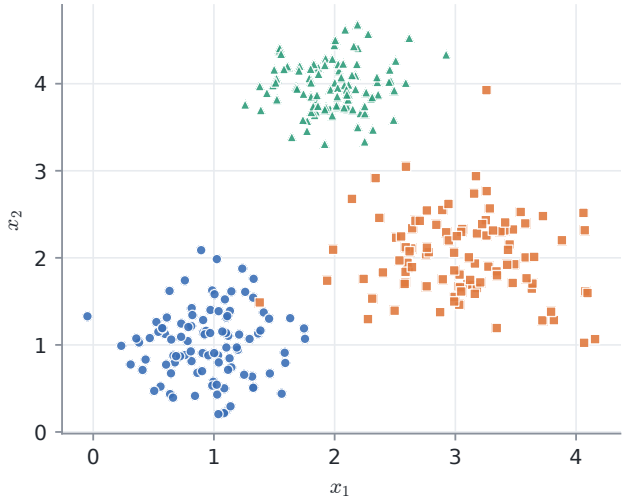


Figure 2.6: Three randomly generated clusters, centered at $(1, 1)$, $(3, 2)$ and $(2, 4)$.

```python
from sklearn.preprocessing import StandardScaler

# three clusters
np.random.seed(42)
centers = np.array([[1, 1], [3, 2], [2, 4]])
X3 = np.vstack([centers[0] + 0.4 * np.random.randn(100, 2),
                centers[1] + 0.5 * np.random.randn(100, 2),
                centers[2] + 0.3 * np.random.randn(100, 2)])
y3 = np.repeat([0, 1, 2], 100)

X_train, X_test, y_train, y_test = train_test_split(
    X3, y3, test_size=0.2, random_state=42)

# standardize (fit on train only!), then add the intercept column
scaler = StandardScaler().fit(X_train)
add_bias = lambda A: np.c_[np.ones(len(A)), A]
X_train_s = add_bias(scaler.transform(X_train))
X_test_s = add_bias(scaler.transform(X_test))

class SoftmaxRegressionGD:
    def __init__(self, learning_rate=0.1, n_iterations=1000, regularization=0.01):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.regularization = regularization
        self.theta = None

    @staticmethod
    def softmax(Z):
```

```
        Z = Z - Z.max(axis=1, keepdims=True)      # stability (Problem 2.4)
        expZ = np.exp(Z)
        return expZ / expZ.sum(axis=1, keepdims=True)

    def fit(self, X, y):
        N, n = X.shape
        K = len(np.unique(y))
        Y = np.eye(K)[y]                          # one-hot labels, shape (N, K)
        self.theta = np.zeros((n, K))             # column k is theta_k
        for _ in range(self.n_iterations):
            P = self.softmax(X @ self.theta)
            grad = X.T @ (P - Y) / N + self.regularization * self.theta
            self.theta -= self.learning_rate * grad
        return self

    def predict_proba(self, X):
        return self.softmax(X @ self.theta)

    def predict(self, X):
        return np.argmax(X @ self.theta, axis=1)

model = SoftmaxRegressionGD().fit(X_train_s, y_train)
y_pred = model.predict(X_test_s)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

```
Accuracy: 0.967
```

Peep Figure 2.7 for the decision regions. Each boundary is still a line, so this is still a linear classifier, but the three lines meet at a point and carve the plane into three wedges. Two of the test points sit in the overlap between the orange and green clusters, and no linear rule gets those right. We will come back to them.

2.3.1 Measuring How Well We Did

So far we have only been reporting accuracy, which seems like the obvious thing to do, but it turns out that accuracy can be pretty misleading. If 99% of emails are not spam, the classifier that always says "not spam" is 99% accurate and completely useless. A better habit is to look at the **confusion matrix**, whose (j, k) entry counts the test points of true class j that we predicted as class k. The diagonal is what we got right, and everything off the diagonal tells us *which* classes get mixed up.

```
def confusion_matrix(y_true, y_pred, K):
    C = np.zeros((K, K), dtype=int)
```

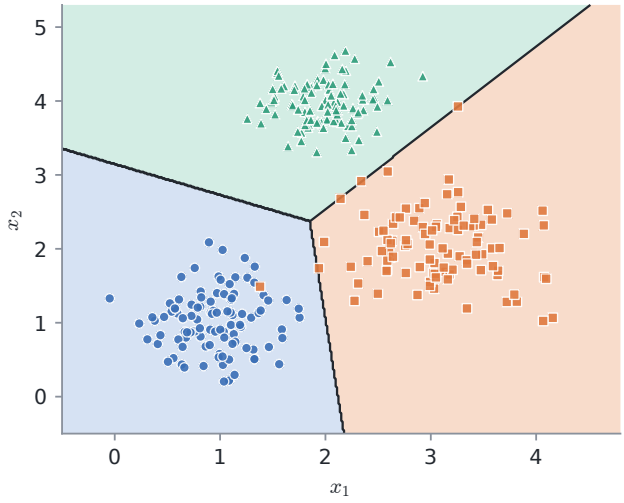


Figure 2.7: Softmax regression on the data of Figure 2.6. Every pair of classes is separated by a straight line, because the boundary between classes j and k is where $(\boldsymbol{\theta}_j - \boldsymbol{\theta}_k)^T \mathbf{x} = 0$.

```
    for t, p in zip(y_true, y_pred):
        C[t, p] += 1
    return C

print(confusion_matrix(y_test, y_pred, 3))
```

```
[[22  0  0]
 [ 0 14  2]
 [ 0  0 22]]
```

All the mistakes live in one cell. Two orange points (class 1) were called green (class 2), which matches the overlap we saw in the picture.

2.3.2 The Last Layer

We want to point out something that surprises people. The thing we just built, a linear map into K logits followed by a softmax and trained with cross-entropy, is the final layer of essentially every modern classifier. An image network ends with it. A large language model ends with it too, where K is the size of the vocabulary (tens of thousands of tokens) and the "class" is the next word. The only difference is that the input $\mathbf{x}$ is no longer raw pixels or raw text. It is a learned representation produced by everything that comes before the last layer. Building that representation is what Chapters 3 and 4 are about. First by hand

(basis expansion), then automatically (neural networks).

Problem 2.5:

A common baseline for K classes is **one-vs-rest**. Train K separate logistic regressions, the k-th one separating class k from everything else, and predict the class whose classifier is most confident. Implement it on the three-cluster data using `logistic_regression_l2`. Compare its decision regions and accuracy with softmax regression. Can you find a point in the plane where none of the K classifiers says "yes", or where several do?

3

Basis Expansion And Regularization

"Basis (Domain) Expansion!"

– Gojo

Up until now, we have been working with data that could be fit by a line or classified by a straight boundary. Sometimes (honestly, most of the time) we need something nonlinear. How can we modify what we've built to handle that? The surprising answer is that we barely have to modify anything. We keep the linear machinery and change what we feed into it.

3.1 Choosing A Basis

Something to emphasize is that when we did linear regression, we *chose* to use a line. We could have chosen any function we wanted. Lines just happen to be well studied and easy to solve for. In the same vein, which functions to fit is a choice that we get to make. Suppose we have data that looks like the left panel of Figure 3.1.

When we can plot the data like this, we can guess that the curve should be quadratic. (In higher dimensions we won't be afforded this luxury, but let's enjoy it while it lasts.) So we expand in a basis of functions,

$$y = \theta_0 + \theta_1 x + \theta_2 x^2. \tag{3.1}$$

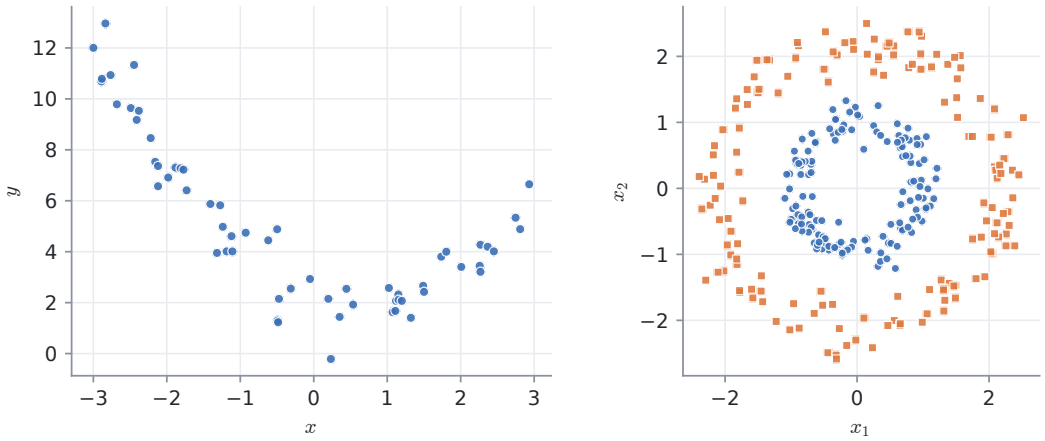


Figure 3.1: Two problems that a straight line can't solve. Left, regression data with an obvious bend. Right, two classes arranged in rings, where any straight boundary misclassifies about half the points.

Here is the key observation. This is quadratic in x, but it is still *linear in the parameters* $\boldsymbol{\theta}$. And linearity in $\boldsymbol{\theta}$ is the only thing least squares ever cared about. So we treat x^2 as just another feature, and the feature matrix becomes

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_N & x_N^2 \end{bmatrix}. \tag{3.2}$$

(Careful with the indices, the subscript now labels the data point, and the power is the power.) More generally, pick any functions $\phi_0(x) = 1, \phi_1(x), \ldots, \phi_P(x)$ that you like, called **basis functions**, and fit

$$\hat{y}(\mathbf{x}) = \sum_{j=0}^{P} \theta_j \, \phi_j(\mathbf{x}) = \boldsymbol{\theta}^T \boldsymbol{\phi}(\mathbf{x}). \tag{3.3}$$

The feature matrix has entries $\Phi_{ij} = \phi_j(\mathbf{x}_i)$, and every formula from Chapter 1 goes through with $\mathbf{X}$ replaced by $\boldsymbol{\Phi}$. Popular choices are polynomials, Gaussian bumps $\phi_j(x) = \exp\big(-(x - c_j)^2 / 2s^2\big)$ centered at points c_j, sines and cosines, and piecewise polynomials called splines, which Hastie, Tibshirani and Friedman spend a whole chapter on [6]. We can learn the coefficients with the normal equations or with gradient descent, whatever we like.

```
import numpy as np

rng = np.random.RandomState(1)
```

```
x = np.sort(rng.uniform(-3, 3, 60))
y = 0.8 * x**2 - x + 2 + rng.normal(0, 1, x.size)   # secretly quadratic

# feature matrix with columns 1, x, x^2
X = np.vstack((np.ones(len(x)), x, x**2)).T
theta, *_ = np.linalg.lstsq(X, y, rcond=None)
print("theta:", theta)

y_pred = X @ theta
```

```
theta: [ 2.22111804 -1.02450056  0.7794983 ]
```

The truth was $(2, -1, 0.8)$, so that's a good fit (left panel of Figure 3.2). In the case of curve fitting, this is really straightforward. All that changes is the feature matrix.

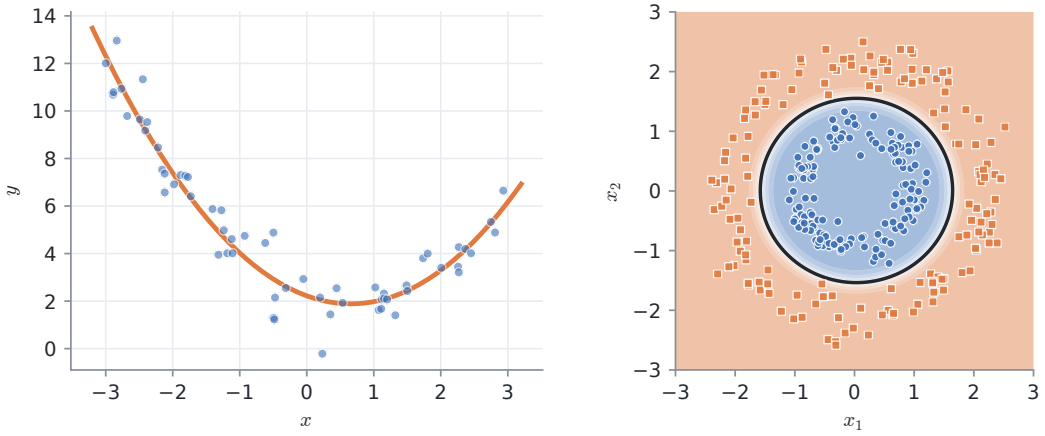


Figure 3.2: Left, least squares with features $(1, x, x^2)$. Right, logistic regression with features $(1, x_1, x_2, x_1^2, x_2^2)$. Both are linear models in the parameters, yet the curve and the decision boundary are nonlinear in the inputs.

What about logistic regression? For the ring data in Figure 3.1, we can deduce that the decision boundary should be a circle, whose equation is

$$x_1^2 + x_2^2 = c. \tag{3.4}$$

So the basis expansion would be

$$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2, \tag{3.5}$$

and the boundary $z = 0$ is now any axis-aligned ellipse (a circle if $\theta_3 = \theta_4$). If we also threw

in the cross term $x_1 x_2$, we would get every possible conic section. We do the exact same thing we did for least squares, treat the quadratic terms as extra features inside the logistic function. The code reuses `logistic_regression_l2` from Chapter 2.

```
from sklearn.preprocessing import StandardScaler

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def logistic_regression_l2(X, y, learning_rate=0.1, iterations=1000, lambda_=0.1):
    N, n = X.shape
    theta = np.zeros(n)
    for _ in range(iterations):
        grad = X.T @ (sigmoid(X @ theta) - y) / N
        grad[1:] += (lambda_ / N) * theta[1:]
        theta -= learning_rate * grad
    return theta

# two noisy rings
rng = np.random.RandomState(0)
n_points = 150
t1, t2 = rng.uniform(0, 2*np.pi, n_points), rng.uniform(0, 2*np.pi, n_points)
r1, r2 = 1 + rng.normal(0, 0.15, n_points), 2.2 + rng.normal(0, 0.2, n_points)
inner = np.c_[r1 * np.cos(t1), r1 * np.sin(t1)]
outer = np.c_[r2 * np.cos(t2), r2 * np.sin(t2)]
P = np.vstack((inner, outer))
y_ring = np.r_[np.zeros(n_points), np.ones(n_points)]

# basis expansion: x1, x2, x1^2, x2^2, then standardize and add intercept
Phi = np.hstack((P, P**2))
Phi = StandardScaler().fit_transform(Phi)
Phi = np.c_[np.ones(len(Phi)), Phi]

theta = logistic_regression_l2(Phi, y_ring, learning_rate=0.1, iterations=10000)
acc = np.mean((sigmoid(Phi @ theta) >= 0.5) == y_ring)
print("theta:", np.round(theta, 2), " training accuracy:", acc)
```

```
theta: [ 1.45 -0.27 -0.07  5.5   6.  ]  training accuracy: 1.0
```

Notice the coefficients on x_1 and x_2 come out near zero while the ones on x_1^2 and x_2^2 are large and nearly equal. The model figured out on its own that only the radius matters (right panel of Figure 3.2).

Problem 3.1:

How many monomials of total degree at most d are there in M variables? (Hint, stars and bars, the answer is $\binom{M+d}{d}$.) Evaluate it for $d = 3$ and $M = 10$, 100 and 784 (the number of pixels in an MNIST digit). What does this say about using full polynomial expansions on high-dimensional data?

3.2 Overfitting, The Burden Of Choice

Once we open the floodgates to all functions, we go from infinitely many possibilities to an even bigger infinity of possibilities. When the data is two-dimensional we have the privilege of plotting it and eyeballing the right function. In high dimensions that is impossible, and the question becomes, how do we know which functions to choose? How do we know we are fitting properly? There are a couple of answers, and none of them is exact. To see why this is a hard question, try fitting our quadratic data with a degree 15 polynomial.

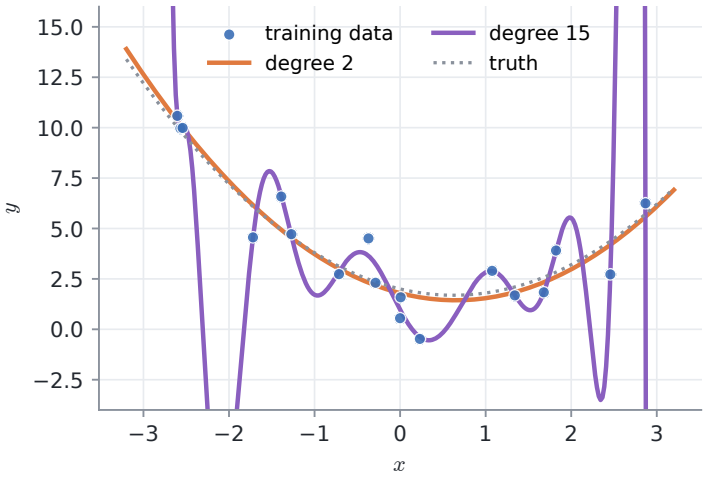


Figure 3.3: Eighteen points from the quadratic, fit with degree 2 and degree 15 polynomials. Degree 15 passes closer to every training point and is clearly worse.

The degree 15 curve has more degrees of freedom, so it can wiggle closer to every training point (Figure 3.3). The training error goes down. But the wiggles are fitting the *noise*, and between and beyond the data points the curve goes wild. On new data from the same source, it will predict horribly. To quantify this, we repeat the experiment hundreds of times. Each time we draw 20 fresh training points, fit every degree from 0 to 15, and measure the error on a big independent test set. Peep Figure 3.4.

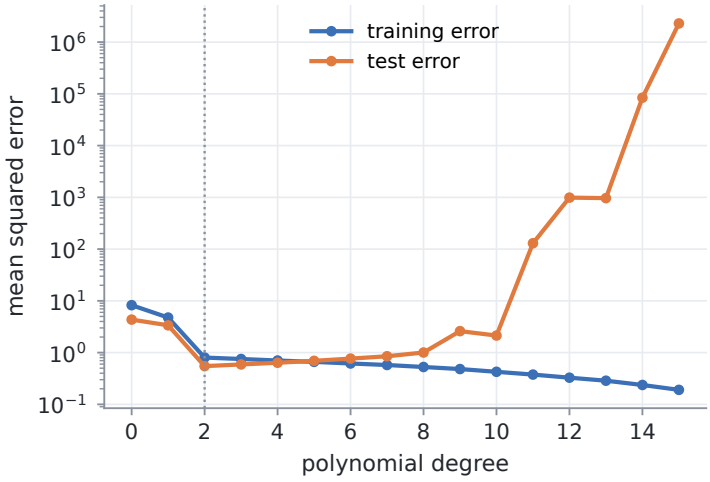


Figure 3.4: Training error (blue) always goes down as the degree increases. Test error (orange) is smallest near the true degree, 2, and explodes past degree 10. Note the log scale.

So what do we see? As we increase the model complexity, the error on the training data always decreases, but the error on the testing data eventually goes up, and it can go up a lot. When training error is low but test error is high, we say the model is **overfitted**. When both are high because the model is too simple to capture the pattern (fitting a line to our parabola, say), it is **underfitted**. We tend to worry more about overfitting, because flexible models are the ones people like to use. This tension has a name, the **bias-variance tradeoff**.

3.2.1 The Bias-Variance Tradeoff

In statistics we might ask "how far off is our estimator?" Our fitted function $\hat{f}$ is computed from the training data, which is random. So $\hat{f}$ itself is a random variable. Draw a different training set, get a different curve. Let's make this precise. Assume the data comes from

$$y = f(x) + \varepsilon, \qquad \mathbb{E}[\varepsilon] = 0, \quad \mathrm{Var}(\varepsilon) = \sigma^2, \tag{3.6}$$

where f is the true function and ε is noise independent of everything else. Fix a test input x and draw a fresh test output y there. We want the expected squared error of our prediction, where the expectation is over both the random training set (which determines $\hat{f}$) and the fresh noise,

$$\mathrm{MSE}(x) = \mathbb{E}\big[(y - \hat{f}(x))^2\big]. \tag{3.7}$$

A measure of how "wrong" we are could be the plain expectation $\mathbb{E}[y - \hat{f}(x)]$, but it's easy to be unbiased and still wrong every single time, with errors above and below canceling (the same issue as with least squares). So we square. To simplify notation, write $f = f(x)$, $\hat{f} = \hat{f}(x)$ and $\bar{f} = \mathbb{E}[\hat{f}(x)]$, the average prediction over all possible training sets. Now we insert zero in a clever way, twice,

$$y - \hat{f} = \underbrace{(y - f)}_{\varepsilon} + (f - \bar{f}) + (\bar{f} - \hat{f}). \tag{3.8}$$

Square this and take the expectation. There are three squared terms and three cross terms. Every cross term vanishes. The ones involving ε vanish because the fresh noise has mean zero and is independent of $\hat{f}$. The remaining one vanishes because $f - \bar{f}$ is a constant (no randomness in it) while $\mathbb{E}[\bar{f} - \hat{f}] = \bar{f} - \bar{f} = 0$. What survives is

$$\begin{aligned} \mathbb{E}\big[(y - \hat{f})^2\big] &= \mathbb{E}[\varepsilon^2] + (f - \bar{f})^2 + \mathbb{E}\big[(\bar{f} - \hat{f})^2\big] \\ &= \sigma^2 + \operatorname{bias}^2\big(\hat{f}(x)\big) + \operatorname{Var}\big(\hat{f}(x)\big). \end{aligned} \tag{3.9}$$

(The first edition left out the σ^2, because it compared $\hat{f}$ to the true f instead of to a noisy observation y. If you do that, you get the same formula without the noise term. Both are correct, they just answer slightly different questions.)

What does this equation mean? The expected test error splits into three pieces. There is the **irreducible noise** σ^2, which no model can beat. The **bias** measures how far our *average* prediction is from the truth. A rigid model (a line for a parabola) is biased no matter how much data we have. The **variance** measures how much our prediction jumps around from one training set to the next. A flexible model (degree 15) is unbiased-ish on average, but each individual fit is all over the place.

Making the model more flexible tends to decrease bias and increase variance. This is a tendency rather than a law of nature (you can always build a model that is worse in both ways). Still, it is the reason we can't eat for free in this world. The good news is that there usually is a sweet spot. We demonstrate this with Bishop's classic setup [3]. Take $f(x) = \sin(2\pi x)$, 25 noisy training points, and a model with 24 Gaussian basis functions, which is very flexible. We control its flexibility with a ridge penalty λ. For each λ, we fit 400 independent training sets and directly measure the squared bias and the variance of the predictions. The result is Figure 3.5, and the sum really does have a minimum in the middle.

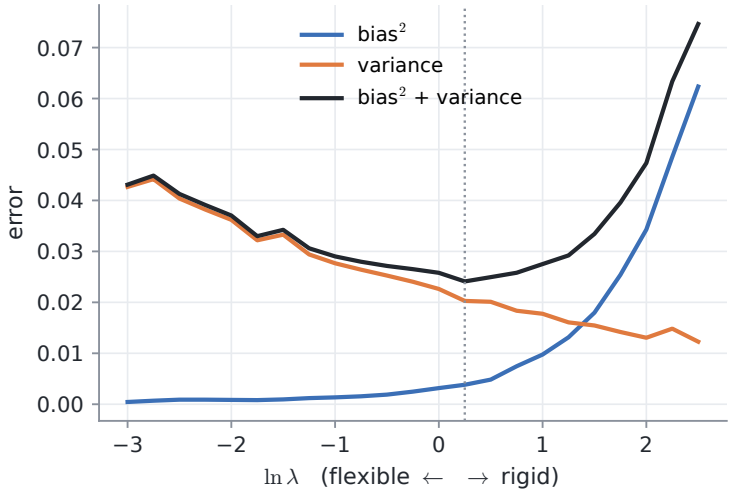


Figure 3.5: Bias-variance decomposition measured by simulation for a 24-basis-function model with ridge penalty λ. Small λ (left) is flexible, with low bias and high variance. Large λ (right) is rigid. The total is minimized around $\ln \lambda \approx 0$. The noise $\sigma^2 = 0.09$ would add a constant on top.

Problem 3.2:

(1) Fill in the cross terms in the derivation of (3.9) and check that each one vanishes. Which independence assumption does each step use?

(2) Consider "ordinary least squares with P features" when the true f is exactly linear in those features. Show the fit is unbiased, and that the variance averaged over the training inputs is $\sigma^2 P/N$. So in this case the average test error at the training inputs is $\sigma^2(1 + P/N)$. Interpret that as a function of P.

3.2.2 Cross-Validation, Or How To Actually Choose

The bias-variance curve is beautiful, but we can only draw it because we *knew* f and could draw as many training sets as we wanted. In real life we have one dataset. How do we pick the degree, or λ, or the basis? This is the human part of machine learning, and it used to be described as a matter of taste. Nowadays there is a standard workhorse, **k-fold cross-validation**. Split the data into k chunks (folds). For each fold, train on the other $k - 1$ folds and measure the error on the held-out one. Average the k errors. Do this for each candidate setting and pick the one with the smallest cross-validated error. Common choices are $k = 5$ or 10. The case $k = N$ is called leave-one-out.

```python
def kfold_cv_error(x, y, degree, k=5, seed=0):
    """Average held-out MSE of a polynomial fit, over k folds."""
    idx = np.random.default_rng(seed).permutation(len(x))
    folds = np.array_split(idx, k)
    errors = []
    for f in folds:
        train = np.setdiff1d(idx, f)
        coef = np.polynomial.Polynomial.fit(x[train], y[train], degree)
        errors.append(np.mean((coef(x[f]) - y[f]) ** 2))
    return np.mean(errors)

for d in [0, 1, 2, 3, 5, 8, 12]:
    print(f"degree {d:2d}: CV error {kfold_cv_error(x, y, d):8.3f}")
```

```
degree  0: CV error   10.159
degree  1: CV error    5.241
degree  2: CV error    0.757
degree  3: CV error    0.805
degree  5: CV error    0.852
degree  8: CV error    0.880
degree 12: CV error    1.169
```

Cross-validation picks degree 2 without ever seeing the true function. (We used `Polynomial.fit` from `np.polynomial` instead of building a raw x^d feature matrix because it rescales x internally. Raw powers like x^{12} make horribly ill-conditioned columns, which is the Chapter 1 feature scaling lesson coming back to bite.) One golden rule goes with this. Once you have used a dataset to choose settings, it is no longer an honest estimate of performance. Keep a final **test set** locked in a drawer and look at it exactly once, at the very end.

3.3 Regularization

We won't repeat everything from Chapter 1, but we now understand much better what regularization is *doing*. One way to tame the volatility of a flexible model is to regularize it, adding a penalty that discourages large coefficients,

$$\text{Cost} = \sum_{i=1}^{N} \left(y_i - \boldsymbol{\theta}^T \boldsymbol{\phi}(\mathbf{x}_i)\right)^2 + \lambda \sum_{j=1}^{P} \theta_j^2. \tag{3.10}$$

In the language we just built, the penalty deliberately *introduces bias*. To keep the cost small, coefficients that want to be huge are forced to shrink, so the model can't fit the

training data quite as well (more bias). In exchange, the fit stops whipping around from one dataset to the next (less variance). Figure 3.5 is exactly this trade, with λ as the knob. A nice consequence is that we can use a very flexible basis and let λ, chosen by cross-validation, decide how flexible the model really gets. That is usually much easier than choosing the right basis by hand.

3.3.1 The Lasso

Ridge isn't the only option. The **lasso** [16] uses absolute values instead of squares,

$$\text{Cost} = \sum_{i=1}^{N} \left(y_i - \boldsymbol{\theta}^T \boldsymbol{\phi}(\mathbf{x}_i)\right)^2 + \lambda \sum_{j=1}^{P} |\theta_j|. \tag{3.11}$$

It behaves like ridge, with one remarkable difference. Ridge shrinks every coefficient toward zero but essentially never makes any of them exactly zero. The lasso sets many coefficients *exactly* to zero, so it performs feature selection for free. Why? The picture in Figure 3.6 explains it. Minimizing the penalized cost is equivalent to minimizing the squared error subject to a constraint $\sum \theta_j^2 \le t$ or $\sum |\theta_j| \le t$ (the multiplier λ is a Lagrange multiplier). The elliptical contours of the squared error expand from the least squares solution until they first touch the constraint region. The ridge region is a round disk, so the first contact is generically somewhere with both coordinates nonzero. The lasso region is a diamond with corners sitting on the axes, and an expanding ellipse is very likely to hit a corner first. A corner means some coordinate is exactly zero. In high dimensions the diamond becomes a cross-polytope with tons of corners and edges, and sparse solutions become the rule.

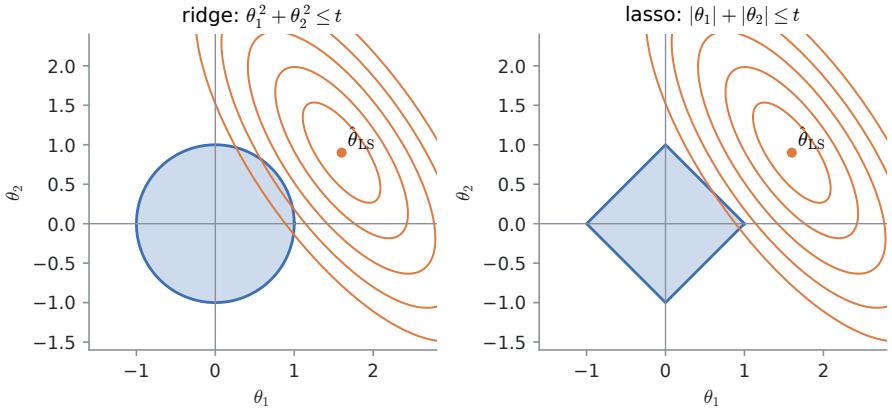


Figure 3.6: Contours of the squared error (orange) around the least squares solution, and the constraint regions (blue) of ridge and lasso. The lasso solution lands on a corner of the diamond, where $\theta_2 = 0$ exactly.

To implement the lasso, the first thing that comes to mind is gradient descent. The derivative of $|\theta|$ is $\operatorname{sgn}(\theta)$, except that it doesn't exist at 0, and 0 is exactly where we want to end up. Plain (sub)gradient descent just jitters around zero without ever landing on it. The modern fix is the **proximal gradient** method, also known as ISTA. Take a normal gradient step on the smooth squared-error part, then apply the **soft-thresholding** operator

$$S_\tau(\theta) = \operatorname{sgn}(\theta)\max(|\theta| - \tau, 0), \tag{3.12}$$

which shrinks every coefficient toward zero by τ and snaps the small ones to exactly zero. (Problem 3.3 shows where it comes from.) Here it is on a problem with 50 features where only 5 matter.

```python
def soft_threshold(theta, tau):
    return np.sign(theta) * np.maximum(np.abs(theta) - tau, 0.0)

def lasso_ista(X, y, lam, iterations=2000):
    """Minimize (1/2N)||y - X theta||^2 + lam * ||theta||_1 (no intercept)."""
    N = len(y)
    L = np.linalg.eigvalsh(X.T @ X / N).max()   # curvature, sets the step size
    alpha = 1.0 / L
    theta = np.zeros(X.shape[1])
    for _ in range(iterations):
        grad = X.T @ (X @ theta - y) / N
        theta = soft_threshold(theta - alpha * grad, alpha * lam)
    return theta

rng = np.random.default_rng(0)
N, M = 100, 50
Xs = rng.normal(size=(N, M))
true_theta = np.zeros(M)
true_theta[:5] = [3, -2, 1.5, 1, -1]            # only 5 features matter
ys = Xs @ true_theta + rng.normal(0, 0.5, N)

theta_lasso = lasso_ista(Xs, ys, lam=0.1)
theta_ridge = np.linalg.solve(Xs.T @ Xs + 10 * np.eye(M), Xs.T @ ys)
print("lasso, nonzero coefficients:", np.sum(theta_lasso != 0))
print("lasso, first 7:", np.round(theta_lasso[:7], 2))
print("ridge, nonzero coefficients:", np.sum(np.abs(theta_ridge) > 1e-8))
```

```
lasso, nonzero coefficients: 8
lasso, first 7: [ 2.8  -1.89  1.39  0.83 -0.84 -0.   -0.  ]
ridge, nonzero coefficients: 50
```

The lasso finds the five real features (shrunk a little toward zero, that's the bias) and zeros out all but three of the other forty-five. Ridge keeps all fifty, just smaller. If you need a model you can explain to a human, that difference matters a lot.

Problem 3.3:

(1) Solve the one-dimensional problem $\min_\theta \frac{1}{2}(\theta - z)^2 + \tau|\theta|$ by considering the cases $\theta > 0$, $\theta < 0$ and $\theta = 0$ separately. Show that the minimizer is $S_\tau(z)$.

(2) Suppose the features are orthonormal, $\mathbf{X}^T\mathbf{X} = \mathbf{I}$. Show that ridge gives $\hat{\theta}_j^{\text{ridge}} = \hat{\theta}_j^{\text{LS}}/(1+\lambda)$ while the lasso gives a soft-thresholded version of $\hat{\theta}_j^{\text{LS}}$ (with the scaling of (3.11), the threshold is $\lambda/2$). Sketch both as functions of $\hat{\theta}_j^{\text{LS}}$.

Are there other regularizers to consider? In general we could penalize any p-norm,

$$\|\boldsymbol{\theta}\|_p = \Big(\sum_j |\theta_j|^p\Big)^{1/p}, \tag{3.13}$$

with $p = 2$ giving ridge (we penalize the square of the norm, which is nicer) and $p = 1$ giving the lasso. For $p < 1$ the constraint region is even pointier and the solutions even sparser, but the problem stops being convex, which makes it much harder to optimize. A popular compromise is the **elastic net**, $\lambda_1\|\boldsymbol{\theta}\|_1 + \lambda_2\|\boldsymbol{\theta}\|_2^2$, which keeps lasso's sparsity but behaves better when features are correlated (pure lasso tends to pick one feature from a correlated group at random). To summarize, to make our models stable it's good practice to regularize. What kind of regularization we choose is up to us and depends on what we are trying to do, and cross-validation is how we choose how much of it to use.

3.4 A Modern Twist, Double Descent

So everything that we have done so far is what we would call the classical story, and for a long time people thought that was pretty much the whole story. Then deep learning came along and broke it. Modern neural networks have far more parameters than training points. By the classical story they should overfit catastrophically. Instead they generalize great. What gives?

Here is an experiment you can run in a few lines. We fit min-norm least squares with p random ReLU features (random basis functions $\phi_j(\mathbf{x}) = \max(0, \mathbf{w}_j^T\mathbf{x})$ with random $\mathbf{w}_j$) on $N = 40$ training points, and we keep increasing p way past N. When $p > N$ there are infinitely many $\boldsymbol{\theta}$ that fit the training data perfectly, and we pick the one with the smallest

norm, which is what `np.linalg.pinv` (and, it turns out, gradient descent started from zero) gives us. Figure 3.7 shows the test error.

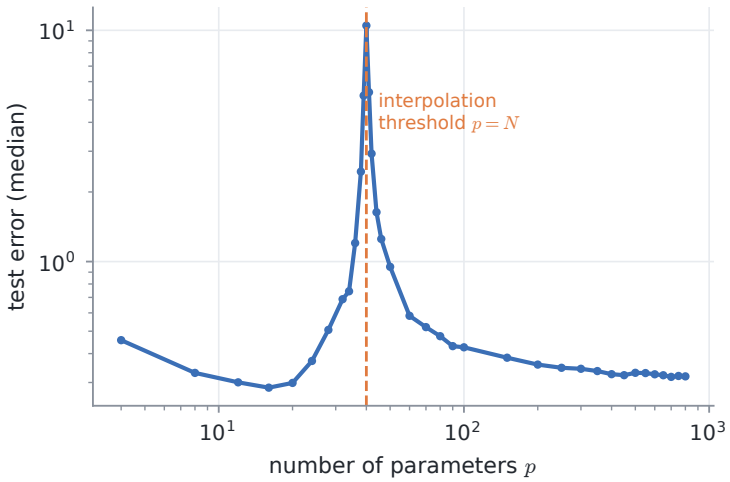


Figure 3.7: Double descent with random ReLU features and $N = 40$ training points. The classical U-shape on the left, a spike when the model can just barely interpolate the data ($p = N$), and then a second descent as p keeps growing.

On the left we see the classical U shape. Then the error explodes right at $p = N$, the **interpolation threshold**, where there is exactly one way to fit the data perfectly and it is a terrible, wiggly one (the variance blows up for the same reason as with nearly singular $\mathbf{X}^T\mathbf{X}$ in Chapter 1). And then, as p keeps growing, the error comes back *down*. This is **double descent** [2]. With lots of extra parameters, there are many interpolating solutions, and the min-norm one is smooth. The implicit preference for small norm acts like a hidden ridge penalty. So the bias-variance tradeoff still holds. The catch is that "number of parameters" turns out to be a bad measure of complexity. What actually controls variance is the norm of the solution, and the optimizer, not just the model, decides which solution we get. People are still working out the full theory, and it is one of the more fun open areas in the field.

Problem 3.4:

Reproduce Figure 3.7. Use $N = 40$, inputs $\mathbf{x} \in \mathbb{R}^6$ with standard normal entries, target $y = \tanh(2\boldsymbol{\beta}^T\mathbf{x}) + \varepsilon$ with $\boldsymbol{\beta}$ a random unit-ish vector, and features $\max(0, \mathbf{w}_j^T\mathbf{x})$. Average over many trials (use the median, the spike is heavy-tailed). Then replace the min-norm solution by ridge with a small $\lambda > 0$. What happens to the spike, and why?

4

Neural Networks

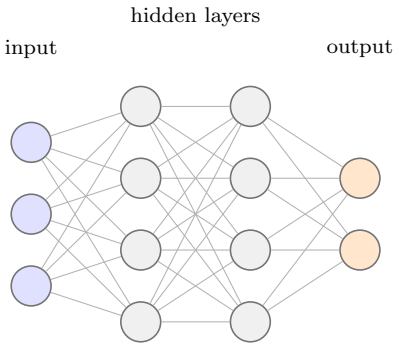
"Oh yeah, it's big brain time."

– Megamind

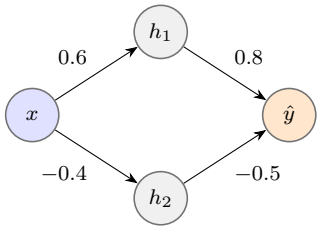
In a reductionist statement, machine learning is about function estimation. In the chapters prior, our goal was to estimate the function that generated (or separated) the data, and that meant one function with a bunch of parameters $\boldsymbol{\theta}$. Chapter 3 showed that we can get nonlinear behavior by expanding in basis functions, but *we* had to choose the basis. A neural network takes the obvious next step. It learns the basis functions too. Many simple functions are composed together, each one feeding the next, and all of their parameters are fit at once. The idea was inspired by the architecture of the brain, where neurons are interconnected and pass signals to one another, although modern networks are about as similar to brains as airplanes are to birds. We will walk through how a network computes a function (the feed forward pass) and how it learns its parameters (back propagation), and then we will build the modern training toolkit from scratch.

4.1 Neural Network Architecture

A neural network is built out of nodes arranged in layers, glued together by activation functions. Each circle in Figure 4.1 is a **node** (or neuron), and it stores a number. A column of nodes is a **layer**. The first layer holds the input, the last layer holds the output, and the layers in between are called **hidden layers**. Every node in one layer is connected to every node in the next layer, and each connection carries a weight.



(a) A general fully connected network.



(b) A tiny network with fixed weights.

Figure 4.1: Neural network diagrams. Each arrow multiplies by its weight, each hidden node adds up what flows in and applies an activation function.

What happens at a node? It adds up the weighted values flowing into it, adds a bias, and passes the result through an **activation function** g. We choose the activation function. All it does is transform its input in some nonlinear way, and the choice matters less than you might think, as long as it is not linear (Problem 4.1 explains why). Figure 4.2 shows the usual suspects. The sigmoid $\sigma(z)$ from logistic regression was the classic choice. The **rectified linear unit**, $\mathrm{ReLU}(z) = \max\{0, z\}$, took over in the 2010s because it is cheap and its gradient doesn't vanish for positive inputs. Smooth cousins of ReLU like GELU are what large language models use today.

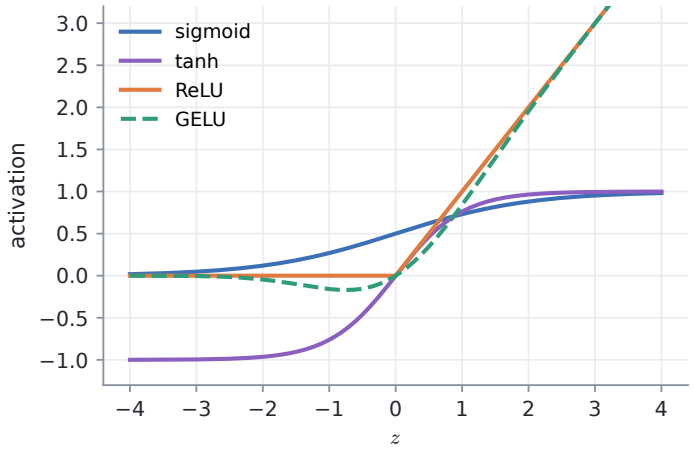


Figure 4.2: Common activation functions. The sigmoid and tanh saturate for large $|z|$, which makes their gradients tiny there. ReLU and GELU don't saturate for positive inputs.

Much like how a signal propagates through a network of neurons, you can imagine information

flowing left to right through the layers. You can treat the hidden layers as one giant series of transformations that turns the raw input into a representation that the last layer finds easy to work with. And the last layer is often just the linear or softmax regression we already know from Chapters 1 and 2.

> Problem 4.1:
>
> Suppose every activation function in a network is the identity, $g(z) = z$. Show that a network with any number of layers computes $\hat{\mathbf{y}} = \mathbf{A}\mathbf{x} + \mathbf{c}$ for some matrix $\mathbf{A}$ and vector $\mathbf{c}$. In other words, a deep linear network is just linear regression wearing a trench coat. Where exactly does the argument break when g is ReLU?

4.2 The Feed Forward Algorithm

Let's compute with the tiny network in Figure 4.1(b). Suppose the input is $x = 1$. Along the top connection we multiply by 0.6, so h_1 receives 0.6. Along the bottom connection h_2 receives -0.4. Now the hidden layer applies its activation function, and we choose ReLU. So h_1 stays 0.6, and h_2 becomes $\max\{0, -0.4\} = 0$. Finally the output node adds up what flows into it, $0.8 \times 0.6 + (-0.5) \times 0 = 0.48$. We use no activation on the output, since this is a regression network and we want any real number to be possible. So the network maps $1 \mapsto 0.48$. Feed it $x = -1$ and you get $0.8 \times 0 + (-0.5) \times 0.4 = -0.2$. Notice that a different hidden node "fires" depending on the sign of the input, and that switching on and off is where the nonlinearity lives.

In general the connections don't just multiply, they also add a bias, exactly like the intercept θ_0 in regression. Figure 4.3 shows the same network with general weights w and biases b.

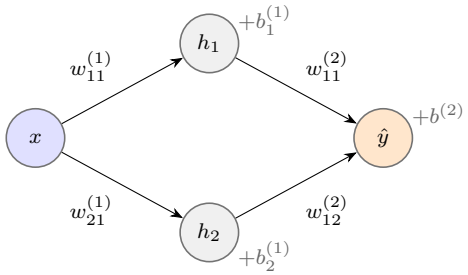


Figure 4.3: The network from Figure 4.1(b) with general weights and biases. The superscript labels the layer, and w_{jk} connects node k of one layer to node j of the next.

4.2.1 Vectorized Feed Forward

Like everything else in this book, we vectorize so the computation isn't costly. Since the input layer has one node, the input is a scalar x. The weights $w_{11}^{(1)}$ and $w_{21}^{(1)}$ form a 2×1 matrix $\mathbf{W}_1$ (just look at the indices, row j is the node it goes to and column k the node it comes from), and the biases form a 2×1 vector $\mathbf{b}_1$. The first computation is

$$\mathbf{z}_1 = \mathbf{W}_1 x + \mathbf{b}_1, \tag{4.1}$$

which is called the **pre-activation**. Then the activation function hits each entry,

$$\mathbf{h} = \mathrm{ReLU}(\mathbf{z}_1) = \max\{\mathbf{0}, \mathbf{W}_1 x + \mathbf{b}_1\}, \tag{4.2}$$

which is still a vector, one entry per hidden node. Going from the hidden layer to the output, $\mathbf{W}_2$ is a 1×2 matrix (one output node, two hidden nodes) and b_2 is a scalar,

$$\hat{y} = \mathbf{W}_2 \mathbf{h} + b_2. \tag{4.3}$$

(In the first edition we got a vector out of this step and then summed it with a vector of ones. That was just a 1×2 matrix with extra steps, so we cut the middleman.) Altogether,

$$x \;\longmapsto\; \mathbf{z}_1 = \mathbf{W}_1 x + \mathbf{b}_1 \;\longmapsto\; \mathbf{h} = \max\{\mathbf{0}, \mathbf{z}_1\} \;\longmapsto\; \hat{y} = \mathbf{W}_2 \mathbf{h} + b_2. \tag{4.4}$$

For a network with L layers, we just repeat the middle two steps,

$$\mathbf{a}_0 = \mathbf{x}, \qquad \mathbf{z}_\ell = \mathbf{W}_\ell \, \mathbf{a}_{\ell-1} + \mathbf{b}_\ell, \qquad \mathbf{a}_\ell = g(\mathbf{z}_\ell), \qquad \ell = 1, \ldots, L, \tag{4.5}$$

with no activation (or a sigmoid or softmax) on the last layer. If layer ℓ has n_ℓ nodes, then $\mathbf{W}_\ell$ is $n_\ell \times n_{\ell-1}$. It is a good exercise to check the dimensions in the code, they are not all the same.

In code, we almost always process a whole batch of inputs at once. The convention in NumPy and PyTorch is that each *row* is a data point, so a batch is an $N \times n_0$ matrix $\mathbf{X}$, and the layer becomes $\mathbf{Z} = \mathbf{A}\mathbf{W}^T + \mathbf{b}$. To save ourselves the transposes, we store each weight matrix in code already transposed, with shape $(n_{\ell-1}, n_\ell)$.

```python
import numpy as np

def relu(z):
    return np.maximum(0, z)

# the tiny network from the figure, stored as (n_in, n_out)
```

```python
W1 = np.array([[0.6, -0.4]])      # shape (1, 2)
b1 = np.zeros(2)
W2 = np.array([[0.8], [-0.5]])    # shape (2, 1)
b2 = np.zeros(1)

def forward(X):
    Z1 = X @ W1 + b1              # (N, 2)  pre-activations
    H = relu(Z1)                  # (N, 2)  hidden activations
    return H @ W2 + b2            # (N, 1)  output, no activation

X = np.array([[1.0], [-1.0], [2.0]])   # a batch of three inputs
print(forward(X).ravel())
```

```
[ 0.48 -0.2   0.96]
```

Same numbers as by hand.

4.3 Back Propagation

Now that we understand how networks compute, we need to discuss how they learn. The goal, per usual, is to minimize a cost function. For regression we keep least squares, and for a single data point

$$J = (y - \hat{y})^2. \tag{4.6}$$

We should be careful, though. This is a harder problem than linear regression, because $\hat{y}$ is a *composition* of functions of the weights rather than a simple linear function of them. To differentiate a composition we need the chain rule, and lots of it. The trick that makes it efficient is to start at the output and work backwards, layer by layer, reusing what we computed at the layer after. Hence the name **back propagation** [12].

Let's do the calculation for our two-layer network, being careful about the parentheses this time (the first edition dropped them and lost a sign). The forward pass gave us

$$\mathbf{z}_1 = \mathbf{W}_1 x + \mathbf{b}_1, \qquad \mathbf{h} = \mathrm{ReLU}(\mathbf{z}_1), \qquad \hat{y} = \mathbf{W}_2 \mathbf{h} + b_2. \tag{4.7}$$

The last layer. The derivative of the cost with respect to the output is

$$\delta_2 \equiv \frac{\partial J}{\partial \hat{y}} = -2\,(y - \hat{y}) = 2\,(\hat{y} - y). \tag{4.8}$$

We will see this number a lot, so we gave it a name, the **error signal** of the output layer. Since $\hat{y} = \mathbf{W}_2 \mathbf{h} + b_2$ is linear in b_2 with slope 1, the chain rule gives $\partial J / \partial b_2 = \delta_2$. For the

weight matrix, $\partial\hat{y}/\partial(W_2)_{1k} = h_k$, so

$$\frac{\partial J}{\partial \mathbf{W}_2} = \delta_2\, \mathbf{h}^T, \qquad \frac{\partial J}{\partial b_2} = \delta_2. \tag{4.9}$$

That has the right shape, 1×2, just like $\mathbf{W}_2$. And notice that it is our old friend from Chapter 2, error times input, where the "input" to this layer is $\mathbf{h}$.

The hidden layer. Here's where the chain rule really shines. To reach $\mathbf{W}_1$ we have to go through $\hat{y}$, then through $\mathbf{h}$, then through $\mathbf{z}_1$. First, how does the cost change with the hidden activations? Since $\hat{y} = \mathbf{W}_2\mathbf{h} + b_2$,

$$\frac{\partial J}{\partial \mathbf{h}} = \mathbf{W}_2^T \delta_2. \tag{4.10}$$

The error flows backward through the same weights it flowed forward through, just transposed. Next we go through the ReLU. Its derivative is the Heaviside step function $\Theta(z)$, which is 1 for $z > 0$ and 0 for $z < 0$. (At $z = 0$ it is undefined, we just pick 0 and nobody has ever been hurt by it.) The ReLU acts entry by entry, so its Jacobian is diagonal, and multiplying by it is an elementwise product, which we write as $\odot$,

$$\boldsymbol{\delta}_1 \equiv \frac{\partial J}{\partial \mathbf{z}_1} = \left(\mathbf{W}_2^T \delta_2\right) \odot \Theta(\mathbf{z}_1). \tag{4.11}$$

This is the error signal of the hidden layer. Hidden nodes that were switched off in the forward pass get zero error and so don't learn from this example. Finally, $\mathbf{z}_1 = \mathbf{W}_1 x + \mathbf{b}_1$ is linear, so exactly like the last layer,

$$\frac{\partial J}{\partial \mathbf{W}_1} = \boldsymbol{\delta}_1\, x^T, \qquad \frac{\partial J}{\partial \mathbf{b}_1} = \boldsymbol{\delta}_1. \tag{4.12}$$

And that's the whole algorithm. For a general network (4.5), it reads

$$\begin{aligned} \boldsymbol{\delta}_L &= \frac{\partial J}{\partial \mathbf{z}_L}, \\ \boldsymbol{\delta}_\ell &= \left(\mathbf{W}_{\ell+1}^T \boldsymbol{\delta}_{\ell+1}\right) \odot g'(\mathbf{z}_\ell), \qquad \ell = L-1, \ldots, 1, \\ \frac{\partial J}{\partial \mathbf{W}_\ell} &= \boldsymbol{\delta}_\ell\, \mathbf{a}_{\ell-1}^T, \qquad \frac{\partial J}{\partial \mathbf{b}_\ell} = \boldsymbol{\delta}_\ell. \end{aligned} \tag{4.13}$$

The cost of the backward pass is about twice the cost of the forward pass, no matter how many parameters there are. That is the miracle. A network with a billion weights gets all billion partial derivatives for the price of a couple of forward passes. Doing it naively, one weight at a time with finite differences, would cost a billion forward passes.

For a batch, we average the cost over the N examples, and the outer products $\boldsymbol{\delta}_\ell \mathbf{a}_{\ell-1}^T$ turn into matrix products summed over the batch. The gradient descent update is what you would expect. Subtract the learning rate α times the gradient from every weight and every bias.

> Problem 4.2:
>
> Suppose the output layer is a single sigmoid node, $\hat{p} = \sigma(z_L)$, and the cost is the binary cross-entropy $J = -y \log \hat{p} - (1-y)\log(1-\hat{p})$. Show that $\delta_L = \partial J / \partial z_L = \hat{p} - y$. Do the same for a softmax output with categorical cross-entropy and show $\boldsymbol{\delta}_L = \hat{\mathbf{p}} - \mathbf{y}$. (This is the "truth minus prediction" of Chapter 2 one more time, and it is why classification networks are always trained with cross-entropy on top of a sigmoid or softmax. The awkward σ' factor cancels.)

> Problem 4.3:
>
> (Gradient checking.) Backprop code is famously easy to get subtly wrong while still producing something that sort of trains. Write a function that perturbs a single weight by $\pm\epsilon$ and estimates $\partial J/\partial w \approx \big(J(w+\epsilon) - J(w-\epsilon)\big)/2\epsilon$. Compare it with your backprop gradient for every weight of a small random network. With $\epsilon = 10^{-5}$ in double precision, the relative error should be around 10^{-7} or smaller. Why is the centered difference so much better than the one-sided $\big(J(w+\epsilon) - J(w)\big)/\epsilon$?

4.4 An Example

Neural networks are extremely versatile, which is a big part of what makes them powerful. They can do regression and classification with the same machinery. We will start with the example from the first edition, a tiny classification problem. We generate 200 random inputs, and the target is the Heaviside step function of the input (so the label is 1 when $x > 0$). We train a network with 1 input node, 1 hidden layer of 5 ReLU nodes, and 1 output node, using the least squares cost and full-batch gradient descent, then threshold the output at 0.5. We split the data 80/20, train on the training part, and test on the rest.

```
def relu_derivative(z):
    return (z > 0).astype(float)            # the Heaviside step

rng = np.random.default_rng(1)
```

```
X = rng.normal(size=(200, 1))               # 200 samples, 1 feature
y = (X > 0).astype(float)                   # binary target

X_train, X_test = X[:160], X[160:]
y_train, y_test = y[:160], y[160:]

# weights stored as (n_in, n_out), small random init
n_in, n_hidden, n_out = 1, 5, 1
W1 = rng.normal(0, 1, (n_in, n_hidden));  b1 = np.zeros(n_hidden)
W2 = rng.normal(0, 1, (n_hidden, n_out)); b2 = np.zeros(n_out)

alpha, epochs, N = 0.1, 2000, len(X_train)
for epoch in range(epochs):
    # forward
    Z1 = X_train @ W1 + b1
    H = relu(Z1)
    y_hat = H @ W2 + b2
    cost = np.mean((y_hat - y_train) ** 2)

    # backward pass, the recursion above with rows = examples
    delta2 = 2 * (y_hat - y_train) / N           # (N, 1)
    dW2 = H.T @ delta2;  db2 = delta2.sum(0)
    delta1 = (delta2 @ W2.T) * relu_derivative(Z1) # (N, 5)
    dW1 = X_train.T @ delta1;  db1 = delta1.sum(0)

    # gradient descent step
    W2 -= alpha * dW2;  b2 -= alpha * db2
    W1 -= alpha * dW1;  b1 -= alpha * db1

    if epoch % 500 == 0:
        print(f"epoch {epoch:4d}  cost {cost:.4f}")

y_test_hat = relu(X_test @ W1 + b1) @ W2 + b2
accuracy = np.mean((y_test_hat > 0.5) == y_test) * 100
print(f"test accuracy: {accuracy:.2f}%")
```

```
epoch    0  cost 6.5562
epoch  500  cost 0.0187
epoch 1000  cost 0.0157
epoch 1500  cost 0.0139
test accuracy: 100.00%
```

Look at how directly the code mirrors (4.13). Each `delta` line is one line of the recursion, and each `dW` is "input to the layer, transposed, times the error signal". The cost doesn't go all the way to zero, though. A ReLU network is continuous, and the target jumps at $x = 0$,

so the outputs for inputs right around zero are always somewhere in between. With only 40 test points none of them happened to land close enough to the jump to be misclassified, but a bigger test set would find a few.

4.5 Training Networks The Modern Way

The loop above is honest, but it is not how anyone trains networks today. Over the last decade a handful of ingredients became standard, and together they are the difference between "trains on a toy" and "trains reliably on anything". We will add the most important ones and then train a network to learn $\sin x$ from noisy samples.

Initialization. Initialize every weight to zero and every hidden node computes the same thing and receives the same gradient, forever. The network never breaks the symmetry (Problem 4.4). So weights start random. But the scale matters a lot. If weights are too big, activations grow exponentially with depth. Too small, and they shrink to nothing. For ReLU networks, **He initialization** [7] draws $w \sim \mathcal{N}(0, 2/n_{\text{in}})$, which keeps the size of the activations roughly constant from layer to layer.

Mini-batches. Exactly as in Chapter 1, we estimate the gradient from a random batch of, say, 64 examples instead of the whole dataset.

Adam. Plain gradient descent uses one learning rate for every parameter. But in a network some parameters see huge gradients and others tiny ones, which is the ill-conditioned valley of Figure 1.5 on steroids. **Adam** [8] keeps a running average of each parameter's gradient (momentum, which smooths out the zig-zag) and of its squared gradient (which estimates the local scale), and divides one by the square root of the other,

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t, \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\,\mathbf{g}_t^2, \\ \boldsymbol{\theta}_t &= \boldsymbol{\theta}_{t-1} - \alpha\,\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}, \end{aligned} \tag{4.14}$$

where $\mathbf{g}_t$ is the current gradient, everything is elementwise, and the hats are bias corrections $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$ and $\hat{\mathbf{v}}_t = \mathbf{v}_t/(1-\beta_2^t)$ that fix the fact that $\mathbf{m}$ and $\mathbf{v}$ start at zero. Default values are $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. The effect is that every parameter takes steps of roughly size α, regardless of how big its raw gradient is. Adam (and its weight decay variant AdamW) is the default optimizer for practically everything trained today.

Learning rate schedules. Large steps are great early on and harmful near the end, when we want to settle into a minimum. So we decay α over training. A popular schedule is the

cosine, $\alpha_t = \frac{1}{2}\alpha_0\big(1 + \cos(\pi t/T)\big)$, which glides from α_0 down to zero.

Here is all of it together, in plain NumPy. The network has two hidden layers of 32 ReLU nodes.

```
rng = np.random.default_rng(0)
N = 256
x = rng.uniform(-np.pi, np.pi, (N, 1))
y = np.sin(x) + 0.1 * rng.normal(size=(N, 1))

sizes = [1, 32, 32, 1]
# He initialization, weights stored as (n_in, n_out)
params = [[rng.normal(0, np.sqrt(2 / a), (a, b)), np.zeros(b)]
          for a, b in zip(sizes[:-1], sizes[1:])]

def forward(X, params):
    acts, pre = [X], []
    for l, (W, b) in enumerate(params):
        z = acts[-1] @ W + b
        pre.append(z)
        last = (l == len(params) - 1)
        acts.append(z if last else np.maximum(0, z))
    return acts, pre

def backward(acts, pre, y, params):
    delta = 2 * (acts[-1] - y) / len(y)          # dJ/dz for the MSE output
    grads = [None] * len(params)
    for l in reversed(range(len(params))):
        grads[l] = [acts[l].T @ delta, delta.sum(0)]
        if l > 0:
            delta = (delta @ params[l][0].T) * (pre[l - 1] > 0)
    return grads

# Adam state, one (m, v) pair per parameter array
m = [[np.zeros_like(p) for p in layer] for layer in params]
v = [[np.zeros_like(p) for p in layer] for layer in params]
beta1, beta2, eps, step = 0.9, 0.999, 1e-8, 0

epochs, batch_size, lr0 = 2000, 64, 1e-2
for epoch in range(epochs):
    lr = 0.5 * lr0 * (1 + np.cos(np.pi * epoch / epochs))   # cosine schedule
    order = rng.permutation(N)
    for s in range(0, N, batch_size):
        idx = order[s:s + batch_size]
        acts, pre = forward(x[idx], params)
        grads = backward(acts, pre, y[idx], params)
        step += 1
```

```
        for l in range(len(params)):
            for k in range(2):                          # k=0 weights, k=1 biases
                g = grads[l][k]
                m[l][k] = beta1 * m[l][k] + (1 - beta1) * g
                v[l][k] = beta2 * v[l][k] + (1 - beta2) * g * g
                m_hat = m[l][k] / (1 - beta1 ** step)
                v_hat = v[l][k] / (1 - beta2 ** step)
                params[l][k] -= lr * m_hat / (np.sqrt(v_hat) + eps)

final_mse = np.mean((forward(x, params)[0][-1] - y) ** 2)
print(f"final training MSE: {final_mse:.4f}  (noise variance is 0.0100)")
```

```
final training MSE: 0.0089  (noise variance is 0.0100)
```

The training error ends up right around the noise level, which is about as low as it should go. It is actually a hair below 0.01, which means the network has started to memorize a little bit of the noise. With 256 points and over a thousand parameters, that's no surprise, and it is the overfitting story of Chapter 3 showing up again. Figure 4.4 shows the learned function and the training curve. The curve is jumpy early on, because each step only sees 64 points and the learning rate is large, and it calms down as the cosine schedule brings the steps down.

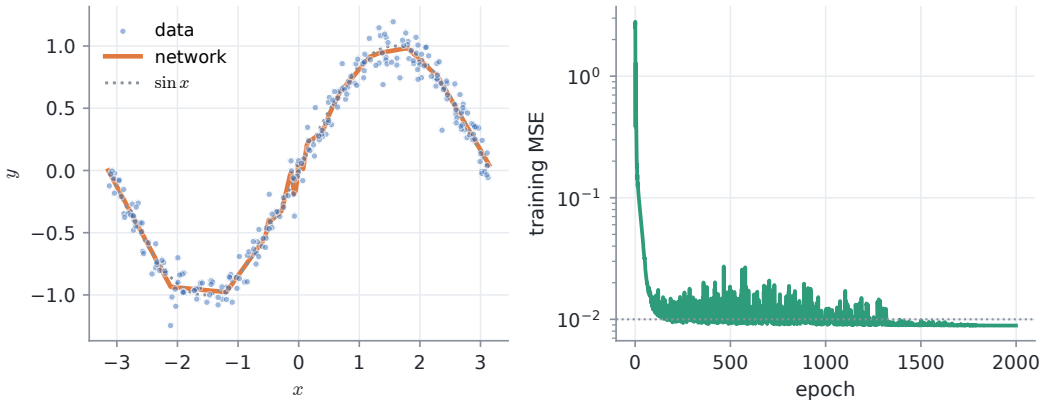


Figure 4.4: Left, a two-hidden-layer ReLU network trained with Adam recovers $\sin x$ from 256 noisy samples. The fit is piecewise linear (it's made of ReLUs) with lots of small pieces. Right, the training MSE per epoch, which settles at the noise variance 0.01 (dotted).

Problem 4.4:

(1) Initialize all weights in the sine network to the same constant. Show, using (4.13), that all hidden nodes in a layer receive identical gradients at every step, so they stay identical forever. Verify it numerically.

(2) (He initialization.) Let $z = \sum_{k=1}^{n} w_k a_k$ with independent $w_k \sim \mathcal{N}(0, s^2)$, and let $a_k = \max(0, u_k)$ where the u_k are independent and symmetric around zero with variance q. Show that $\mathbb{E}[a_k^2] = q/2$ and hence $\mathrm{Var}(z) = ns^2q/2$. What choice of s^2 keeps the variance the same from layer to layer?

(3) Swap Adam for plain mini-batch SGD with a constant learning rate. How small does the learning rate have to be to avoid divergence, and how many epochs does it take to reach the noise level?

4.6 Automatic Differentiation From Scratch

If we had to write `backward` by hand for every new architecture, it would get old really fast, and on top of that, it is usually where all the bugs hide. Modern frameworks never ask you to do it. You write the forward pass, and the framework computes all the gradients for you using **reverse-mode automatic differentiation**. This is not numerical differentiation (no ϵ's) and it is not symbolic differentiation (no giant formulas). It is back propagation, done automatically on any computation you write.

The idea is small enough to build in a page. Every number remembers which numbers it was computed from and how. Each operation knows its own local derivative. When we call `backward()` on the final result, we visit the computation graph in reverse order and apply the chain rule at each node, accumulating gradients. Here is a scalar version, in the spirit of Karpathy's *micrograd*.

```
class Value:
    """A scalar that remembers how it was computed."""
    def __init__(self, data, parents=(), local_grads=()):
        self.data = float(data)
        self.grad = 0.0
        self.parents = parents          # Values this one was computed from
        self.local_grads = local_grads  # d(self)/d(parent) for each parent

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data + other.data, (self, other), (1.0, 1.0))

```

```
    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        return Value(self.data * other.data, (self, other), (other.data, self.data))

    def relu(self):
        return Value(max(0.0, self.data), (self,), (float(self.data > 0),))

    def __neg__(self):  return self * -1
    def __sub__(self, other):  return self + (-other)
    __radd__ = __add__
    __rmul__ = __mul__

    def backward(self):
        # topological order: every node appears after its parents
        order, seen = [], set()
        def visit(v):
            if id(v) not in seen:
                seen.add(id(v))
                for p in v.parents:
                    visit(p)
                order.append(v)
        visit(self)
        self.grad = 1.0
        for v in reversed(order):                       # output to inputs
            for p, local in zip(v.parents, v.local_grads):
                p.grad += local * v.grad                # the chain rule

# the tiny network from Figure 4.1(b), now with gradients for free
x = Value(1.0)
w11, w21, v1, v2 = Value(0.6), Value(-0.4), Value(0.8), Value(-0.5)
y_hat = v1 * (w11 * x).relu() + v2 * (w21 * x).relu()
cost = (y_hat - 1.0) * (y_hat - 1.0)                  # target y = 1
cost.backward()
print(f"y_hat = {y_hat.data:.2f}, cost = {cost.data:.4f}")
print(f"dJ/dw11 = {w11.grad:.4f}, dJ/dw21 = {w21.grad:.4f}, dJ/dv1 = {v1.grad:.4f}")
```

```
y_hat = 0.48, cost = 0.2704
dJ/dw11 = -0.8320, dJ/dw21 = 0.0000, dJ/dv1 = -0.6240
```

Check these by hand with (4.13). We have $\delta_2 = 2(0.48 - 1) = -1.04$, so $\partial J/\partial v_1 = \delta_2 h_1 = -1.04 \times 0.6 = -0.624$. The hidden node h_1 is on, so $\partial J/\partial w_{11} = \delta_2 v_1 x = -0.832$. The node h_2 is off, so w_{21} gets exactly zero gradient. The tiny `Value` class did all of this by itself.

PyTorch and JAX do precisely this, with tensors instead of scalars and a few hundred operations instead of four. For comparison, here is the sine network in PyTorch. Every line

has a counterpart in our NumPy version, and `loss.backward()` replaces our hand-written `backward`.

```python
import torch
from torch import nn

model = nn.Sequential(nn.Linear(1, 32), nn.ReLU(),
                      nn.Linear(32, 32), nn.ReLU(),
                      nn.Linear(32, 1))
opt = torch.optim.AdamW(model.parameters(), lr=1e-2, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=2000)
xt, yt = torch.tensor(x, dtype=torch.float32), torch.tensor(y, dtype=torch.float32)

for epoch in range(2000):
    for idx in torch.randperm(len(xt)).split(64):
        loss = ((model(xt[idx]) - yt[idx]) ** 2).mean()
        opt.zero_grad()
        loss.backward()        # reverse-mode autodiff
        opt.step()             # AdamW update
    sched.step()
```

Problem 4.5:

Extend `Value` with `exp`, `log` and `tanh`, each with its correct local derivative. Then use it to build a 2-2-1 network with tanh hidden units and a sigmoid output (write $\sigma(z) = 1/(1+e^{-z})$ in terms of your new operations), and train it with plain gradient descent on the four XOR points $\{(0,0)\mapsto 0, (0,1)\mapsto 1, (1,0)\mapsto 1, (1,1)\mapsto 0\}$ using cross-entropy. Explain why no linear classifier from Chapter 2 can solve this problem, and why your network can.

4.7 Other Architectures

Everything in this chapter was about **fully connected** networks, also called multilayer perceptrons, where every node talks to every node in the next layer. They can approximate any continuous function on a bounded region if they are wide enough (the universal approximation theorem), but "can" is doing a lot of work in that sentence. The real progress of the last fifteen years came from building the structure of the data into the architecture, so that the network doesn't have to learn it from scratch.

Images, for example, have local structure, so a cat is still a cat no matter where it shows

up in the picture. **Convolutional networks** exploit this by using small weight matrices that slide across the image, sharing the same weights at every position. Sequences like text need to relate words that may be far apart. The **transformer** [17] does this with *attention*, where each position computes a softmax-weighted average of all the other positions, with the weights computed from the data itself. (Yes, it's the softmax from Chapter 2 again. It is everywhere.) Underneath, both are still compositions of linear maps and simple nonlinearities, trained by back propagation and Adam, exactly as we did here. If you understood this chapter, you understand the engine. The rest is the body kit, and Bishop and Bishop's newer book [4] is a great place to see it assembled.

Part II

Unsupervised Learning

5

K-Means, Gaussian Mixtures, And PCA

"The whole is greater than the sum of its parts."

– Aristotle, probably

Up to now we have been doing **supervised** learning, which means the machine learns by finding the relationship between inputs and outputs that we hand it. The word supervised is there because we tell the model what the correct answer is for every training point. There is a ground truth to cling onto. Now, what if there isn't? What if we just have a pile of inputs and nobody to tell us what they mean? Then we can't supervise the machine anymore, and it has to find patterns in the structure of the data itself. This is **unsupervised learning**. We will look at two flavors of it. The first is **clustering**, which finds groups, and we will do it two ways, with k-means (hard labels) and Gaussian mixture models (probabilities). The second is **dimensionality reduction**, which finds a smaller set of numbers that still describes the data, and the classic tool for it is principal component analysis.

5.1 Problem Set Up

Let us first set up the problem before we talk about methods. Consider the three clusters from Chapter 2 (Figure 2.6). The difference now is that we don't know the labels. There are 300 points in the plane and nothing else. We want to *cluster* the data, which means deciding on our own which points belong together.

This emphasizes the big difference between unsupervised and supervised learning. We don't know what the answer is, and often there isn't a single right one. You can look at Figure 2.6 and see three groups instantly, but the computer can't see the picture. And when we minimize a cost function for clustering, there are usually many local minima, so gradient-style methods can land in different answers depending on where they start. The data doesn't even need to be points in the plane. It could be customers, songs, pixels in an image or genes. The only thing all these problems need is a notion of **distance** between data points. For points in $\mathbb{R}^2$ the Euclidean distance is the natural choice, but the algorithms below work with any distance that makes sense for the data.

5.2 The K-Means Algorithm

Let's break down the terminology. What is the k in k-means? It is the number of clusters we want. It is the human telling the machine how many groups there should be. In our example we can see by eye that $k = 3$, but in practice we won't have that luxury. Before we can discuss what the "means" part means (ba dum tss), we need a **centroid**, which is the center of mass of a group of points, $\boldsymbol{\mu} = \frac{1}{n}\sum_i \mathbf{x}_i$. Each cluster gets a centroid, and each point belongs to the cluster whose centroid is nearest. With just the anatomy of the name, we can write down the algorithm.

Algorithm 2: k-means (Lloyd's algorithm)

Choose k, the number of clusters;
Initialize centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$ (for example, k random data points);
repeat
  Assign. Label each point with its nearest centroid, $c_i = \arg\min_j \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$;
  Update. Move each centroid to the mean of its points, $\boldsymbol{\mu}_j = \frac{1}{|C_j|}\sum_{i \in C_j} \mathbf{x}_i$;
until *the assignments stop changing*;

Why does this work? Because it is minimizing something. Define the total squared distance from each point to its centroid, called the **distortion** (or inertia),

$$J(\{c_i\}, \{\boldsymbol{\mu}_j\}) = \sum_{i=1}^{N} \|\mathbf{x}_i - \boldsymbol{\mu}_{c_i}\|^2. \tag{5.1}$$

The assignment step minimizes J over the labels with the centroids held fixed (each point picks its closest centroid, which is obviously the best it can do). The update step minimizes J over the centroids with the labels held fixed (the mean minimizes the sum of squared distances, which is the least squares fact from Chapter 1 in disguise). So each step can

only decrease J. Since there are finitely many ways to assign labels, the algorithm must stop. It stops at a *local* minimum though, not necessarily the global one. Finding the global minimum of (5.1) is NP-hard in general, so local is the best anyone can promise.

The k-means algorithm is surprisingly simple and surprisingly powerful. Here it is fully vectorized. The trick is to compute all $N \times k$ distances at once with broadcasting instead of looping over points.

```
import numpy as np

# the three clusters from Chapter 2, labels thrown away
np.random.seed(42)
centers = np.array([[1, 1], [3, 2], [2, 4]])
X = np.vstack([centers[0] + 0.4 * np.random.randn(100, 2),
               centers[1] + 0.5 * np.random.randn(100, 2),
               centers[2] + 0.3 * np.random.randn(100, 2)])

def k_means(X, init_centroids, max_iters=100):
    centroids = init_centroids.copy()
    for _ in range(max_iters):
        # (N, k) matrix of squared distances via broadcasting
        d2 = ((X[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2)
        labels = d2.argmin(axis=1)
        new_centroids = np.array([X[labels == j].mean(axis=0)
                                  for j in range(len(centroids))])
        if np.allclose(new_centroids, centroids):
            break
        centroids = new_centroids
    distortion = d2[np.arange(len(X)), labels].sum()
    return centroids, labels, distortion

rng = np.random.default_rng(3)
init = X[rng.choice(len(X), 3, replace=False)]
centroids, labels, J = k_means(X, init)
print("centroids:\n", np.round(centroids, 3))
print("distortion:", round(J, 2))
```

```
centroids:
 [[0.958 1.018]
 [3.092 1.987]
 [2.008 3.943]]
distortion: 88.24
```

We plot the result in Figure 5.1, along with the path that each centroid took from its random

starting point. The true centers were $(1, 1)$, $(3, 2)$ and $(2, 4)$, so that's a clean recovery in a handful of iterations.

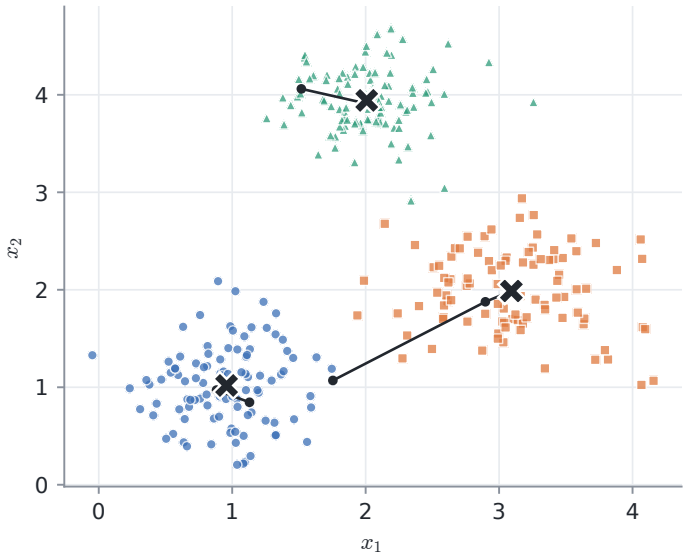


Figure 5.1: k-means with $k = 3$. Black lines trace each centroid from its random initial position to its final position (the crosses). Points are colored by their final cluster.

Problem 5.1:

(1) Prove that for fixed points $\mathbf{x}_1, \ldots, \mathbf{x}_n$, the vector $\boldsymbol{\mu}$ minimizing $\sum_i \|\mathbf{x}_i - \boldsymbol{\mu}\|^2$ is their mean. Use this to show that J never increases during k-means.

(2) Construct a one-dimensional dataset of four points and an initialization for $k = 2$ where k-means converges to a clustering that is clearly not the global minimum.

5.2.1 Better Starts With k-means++

Since k-means only finds a local minimum, the starting centroids matter. If two random starting points land in the same true cluster, the algorithm can happily split that cluster in two and merge two others. The standard fix today is **k-means++** [1], which is what sklearn uses by default. Pick the first centroid uniformly at random from the data. Then pick each next centroid at random with probability proportional to $D(\mathbf{x})^2$, the squared distance from $\mathbf{x}$ to the nearest centroid already chosen. Points far from every existing centroid are much more likely to be picked, so the starting centroids spread out. It is a tiny change, and it comes with a guarantee. The expected distortion is within a factor of $O(\log k)$ of the

optimum, before Lloyd's iterations even start.

```
def kmeans_plus_plus(X, k, rng):
    centroids = [X[rng.integers(len(X))]]
    for _ in range(k - 1):
        d2 = np.min([((X - c) ** 2).sum(axis=1) for c in centroids], axis=0)
        probs = d2 / d2.sum()
        centroids.append(X[rng.choice(len(X), p=probs)])
    return np.array(centroids)

init_pp = kmeans_plus_plus(X, 3, np.random.default_rng(0))
print("k-means++ distortion:", round(k_means(X, init_pp)[2], 2))
```

```
k-means++ distortion: 88.24
```

On this data both starts find the same answer, because the clusters are well separated and almost any start works. The difference shows up on messier data with more clusters, where purely random starts fail regularly. In practice people also just run k-means several times from different starts and keep the run with the smallest distortion. That's what the `n_init` argument in sklearn does.

5.2.2 Choosing k

We said that how many clusters to use is the hardest (and the easiest) step. How do we know we chose well if we can't plot the data? There are a couple of ways, and none of them is perfect (as the saying goes, if things were perfect, we wouldn't be doing machine learning).

The first is the **elbow method**. Run k-means for $k = 1, 2, \ldots, 10$ and plot the final distortion against k. The distortion always decreases as k grows (with $k = N$ it is zero, since every point is its own cluster). But typically there is some k where the curve drops sharply and then flattens, and each extra cluster only buys a small improvement. That kink looks like an elbow, and it is the k we pick. For our data, the left panel of Figure 5.2 has a very clear elbow at $k = 3$.

What if there is no elbow? Or two of them? Then the second major tool is the **silhouette score**. It is simple but expensive, since it needs all pairwise distances. Pick a point $\mathbf{x}_i$. Let $a(i)$ be the average distance from $\mathbf{x}_i$ to the other points in its own cluster, which measures **cohesion**. Let $b(i)$ be the average distance from $\mathbf{x}_i$ to the points of the nearest *other* cluster,

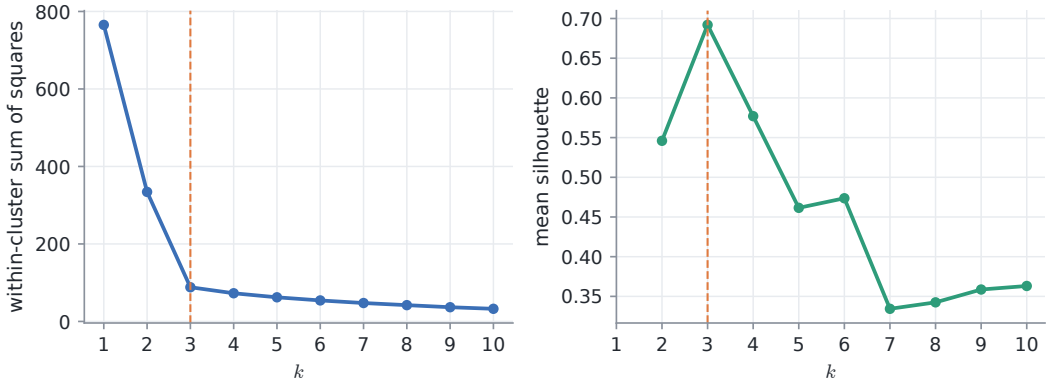


Figure 5.2: Choosing k for the three-cluster data. Left, the elbow method, the distortion stops dropping quickly after $k = 3$. Right, the mean silhouette score peaks at $k = 3$.

which measures **separation**. The silhouette of the point is

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}. \tag{5.2}$$

It lives between -1 and 1. A value near 1 means the point is snug in its own cluster and far from the next one. A value near 0 means it is on the fence between two clusters. A negative value means it is closer to another cluster than to its own and is probably misassigned. Average $s(i)$ over all points and pick the k with the highest average. In the right panel of Figure 5.2, it peaks at $k = 3$. Other approaches include stability tests (run k-means on random subsets and see whether the clusters stay the same) and, if you are willing to commit to a probabilistic model, the likelihood-based criteria we can use with Gaussian mixtures below. Just remember that every one of these means running the algorithm many times, which gets expensive with lots of data.

5.3 Gaussian Mixture Models

k-means makes hard decisions. Every point belongs to exactly one cluster, even a point sitting exactly halfway between two centroids. It also implicitly assumes clusters are round and similar in size, since it only looks at Euclidean distance to the center. We now discuss a softer, probabilistic version called **Gaussian mixture models** (GMMs). The premise is straightforward. We assume each cluster is a Gaussian with its own mean $\boldsymbol{\mu}_k$ and covariance $\boldsymbol{\Sigma}_k$, and that each point was generated by first picking a cluster at random and then sampling from that cluster's Gaussian.

To show how a computation goes, let us start in one dimension with a mixture of two

Gaussians. With probability $\pi_1 = 0.3$, choose cluster 1, which is $\mathcal{N}(1, 1)$. With probability $\pi_2 = 0.7$, choose cluster 2, which is $\mathcal{N}(4, 1.5^2)$. Call the chosen cluster z. So

$$\mathbb{P}(z = k) = \pi_k, \qquad x \mid z = k \sim \mathcal{N}(\mu_k, \sigma_k^2). \tag{5.3}$$

The numbers π_k are the **mixing coefficients**. They are nonnegative and sum to one, so z follows a categorical distribution (the first edition called this multi-Bernoulli, which is not wrong, just not what anybody calls it). The cluster label z is a **latent variable**, since it is part of the model but we never observe it. The data points x are the **observables**, the things we do get to see. Using the law of total probability, the distribution of a single observation is

$$p(x) = \sum_k \mathbb{P}(z = k)\, p(x \mid z = k) = \sum_k \pi_k \mathcal{N}(x; \mu_k, \sigma_k^2), \tag{5.4}$$

and in our example,

$$p(x) = 0.3\,\mathcal{N}(x; 1, 1) + 0.7\,\mathcal{N}(x; 4, 1.5^2). \tag{5.5}$$

The mixture is plotted in Figure 5.3. A weighted sum of two bells gives a lumpy distribution that no single Gaussian could represent. With enough components, a mixture can approximate pretty much any density, which is why GMMs are useful for density estimation too and not just clustering.

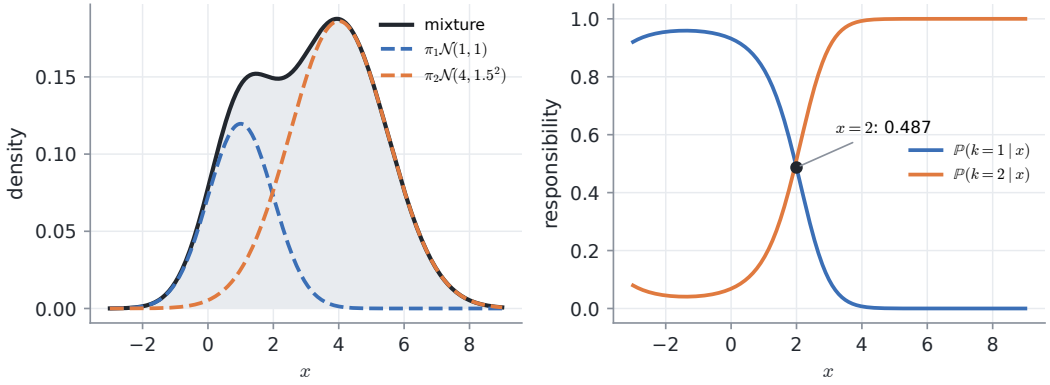


Figure 5.3: Left, the mixture $0.3\,\mathcal{N}(1, 1) + 0.7\,\mathcal{N}(4, 1.5^2)$ (black) and its two weighted components (dashed). Right, the responsibilities, the posterior probability that a point at x came from each component. At $x = 2$ it is essentially a coin flip.

Once we know the mixing coefficients, means and variances, what is the probability that a data point x belongs to cluster k? That is, what is $\mathbb{P}(z = k \mid x)$? This is a straightforward

use of Bayes' theorem,

$$\mathbb{P}(z = k \mid x) = \frac{\mathbb{P}(z = k)\, p(x \mid z = k)}{\sum_j \mathbb{P}(z = j)\, p(x \mid z = j)} = \frac{\pi_k \, \mathcal{N}(x; \mu_k, \sigma_k^2)}{\sum_j \pi_j \, \mathcal{N}(x; \mu_j, \sigma_j^2)}. \tag{5.6}$$

This quantity is called the **responsibility** of cluster k for the point x. For example, suppose we observe $x = 2$. The Gaussian densities are $\mathcal{N}(2; 1, 1) = 0.2420$ and $\mathcal{N}(2; 4, 1.5^2) = 0.1093$, so

$$\begin{aligned} \mathbb{P}(z = 1 \mid x = 2) &= \frac{0.3 \times 0.2420}{0.3 \times 0.2420 + 0.7 \times 0.1093} \\ &= \frac{0.0726}{0.0726 + 0.0765} \\ &\approx 0.487. \end{aligned} \tag{5.7}$$

The probability for the second cluster is the complement, about 0.513. (In the first edition we plugged the mixing weights into the wrong slots and got 0.824. Notice how the bigger weight of cluster 2 almost exactly cancels the fact that $x = 2$ is closer to the center of cluster 1.) This is why Gaussian mixture models are called soft clustering. They don't tell us which cluster each observation belongs to, they give us the probability of each. If we want hard labels like k-means, we pick the cluster with the highest responsibility.

5.3.1 How Do Gaussian Mixture Models Learn?

The calculation above assumed we knew the parameters $\boldsymbol{\theta} = \{\pi_k, \mu_k, \sigma_k\}$. Of course we have to learn them from data, by maximum likelihood as usual. The log-likelihood of the data is

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{N} \log p(x_i) = \sum_{i=1}^{N} \log \sum_k \pi_k \, \mathcal{N}(x_i; \mu_k, \sigma_k^2). \tag{5.8}$$

The log of a sum is annoying. It doesn't split up nicely, and setting the derivative to zero gives no closed form. There are two ways forward, gradient-based or expectation maximization. Either way we need the derivative of the log-likelihood, and there is a neat trick for it. For a single point,

$$\begin{aligned} \frac{\partial}{\partial \theta} \log p(x) &= \frac{1}{p(x)} \frac{\partial}{\partial \theta} \sum_z p(z, x) \\ &= \sum_z \frac{p(z, x)}{p(x)} \frac{\partial}{\partial \theta} \log p(z, x) \\ &= \sum_z \mathbb{P}(z \mid x) \frac{\partial}{\partial \theta} \log p(z, x) \end{aligned}$$

$$= \mathbb{E}_{z \sim \mathbb{P}(z|x)} \left[\frac{\partial}{\partial \theta} \log p(z, x) \right]. \tag{5.9}$$

In the second line we used the log-derivative trick, $\partial_\theta p = p \, \partial_\theta \log p$, which is just multiplying and dividing by p. In the third line we used Bayes' theorem, $p(z, x)/p(x) = \mathbb{P}(z \mid x)$. What this tells us is that the gradient of the log-likelihood of the observed data is the *expected* gradient of the log-likelihood we would have if we could see the labels, with the expectation taken over the posterior of the labels. And $\log p(z, x)$ is easy, since it has no sum inside. It's just $\log \pi_z + \log \mathcal{N}(x; \mu_z, \sigma_z^2)$.

For example, the gradient with respect to μ_1. The mixing coefficient term doesn't depend on μ_1, so its derivative gets killed. Among the Gaussian terms, only $z = 1$ involves μ_1, so only that term survives the expectation, and it comes with weight $\mathbb{P}(z = 1 \mid x)$. Using $\partial_\mu \log \mathcal{N}(x; \mu, \sigma^2) = (x - \mu)/\sigma^2$, and summing over all the data, we get

$$\frac{\partial \ell}{\partial \mu_1} = \sum_{i=1}^{N} r_{i1} \, \frac{x_i - \mu_1}{\sigma_1^2}, \qquad r_{ik} \equiv \mathbb{P}(z_i = k \mid x_i). \tag{5.10}$$

We could plug this into gradient ascent and call it a day. It works, but it is slow and fiddly (the variances have to stay positive, and the mixing weights have to stay on the simplex). Is there an alternative? Yes!

5.3.2 The Expectation Maximization Algorithm

Set (5.10) equal to zero and solve for μ_1. We get

$$\mu_1 = \frac{\sum_{i=1}^{N} r_{i1} \, x_i}{\sum_{i=1}^{N} r_{i1}}, \tag{5.11}$$

which looks like a center of mass where each point is weighted by how much cluster 1 is responsible for it. We might call it a day here and say we found the closed form for μ_1, but there is an issue. The responsibilities r_{i1} *depend on* μ_1. So what we actually have is a fixed point equation rather than a solution. Change μ_1 and the responsibilities change, which changes the best μ_1, and so on. That's ok, because iterating that loop is exactly the **expectation maximization** (EM) algorithm. Repeat until converged.

E-step. With the current parameters, compute the responsibilities,

$$r_{ik} \leftarrow \mathbb{P}(z_i = k \mid \mathbf{x}_i) = \frac{\pi_k \, \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \, \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}. \tag{5.12}$$

M-step. With the responsibilities held fixed, maximize the *expected* complete-data log-likelihood,

$$\boldsymbol{\theta} \leftarrow \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{N} \sum_{k=1}^{K} r_{ik} \Big[\log \pi_k + \log \mathcal{N}(\mathbf{x}_i; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \Big]. \tag{5.13}$$

The M-step is easy because the log is now on the inside. Each parameter only shows up in one of the two terms, so we can maximize them separately. For the mixing proportions, we need a Lagrange multiplier to enforce $\sum_k \pi_k = 1$. In the first edition we skipped this, but it's three lines, so let's do it. Maximize $\sum_{i,k} r_{ik} \log \pi_k + \lambda(1 - \sum_k \pi_k)$. The derivative with respect to π_k is $\sum_i r_{ik}/\pi_k - \lambda = 0$, so $\pi_k = \sum_i r_{ik}/\lambda$. Summing over k and using $\sum_k r_{ik} = 1$ gives $\lambda = N$. Defining $N_k = \sum_i r_{ik}$, the effective number of points in cluster k,

$$\pi_k \leftarrow \frac{N_k}{N}. \tag{5.14}$$

This is exactly the intuitive estimate for a proportion, the number of times the event happens over the total, just with soft counts. For the Gaussian parameters, the problem for cluster k is a weighted Gaussian maximum likelihood, and we have taken maximum likelihoods of Gaussians before. The answers are the weighted mean and the weighted covariance,

$$\boldsymbol{\mu}_k \leftarrow \frac{1}{N_k} \sum_{i=1}^{N} r_{ik}\, \mathbf{x}_i, \qquad \boldsymbol{\Sigma}_k \leftarrow \frac{1}{N_k} \sum_{i=1}^{N} r_{ik}\, (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^T. \tag{5.15}$$

(In one dimension the second formula is the variance σ_k^2, not the standard deviation, which the first edition mixed up.) Notice how much this looks like k-means. The E-step is a soft version of the assignment step, and the M-step is a soft version of the update step. In fact k-means is a limiting case of EM (Problem 5.3).

```
def log_gaussian(X, mu, Sigma):
    """Log density of N(mu, Sigma) at each row of X."""
    d = X.shape[1]
    L = np.linalg.cholesky(Sigma)
    sol = np.linalg.solve(L, (X - mu).T)            # whitened residuals
    return (-0.5 * np.sum(sol ** 2, axis=0)
            - np.log(np.diag(L)).sum() - 0.5 * d * np.log(2 * np.pi))

def gmm_em(X, K, n_iter=200, tol=1e-8, seed=0):
    N, d = X.shape
    rng = np.random.default_rng(seed)
    mu = kmeans_plus_plus(X, K, rng)                # good starting means
    Sigma = np.array([np.cov(X.T) for _ in range(K)])
    pi = np.ones(K) / K
    prev = -np.inf
```

```
    for it in range(n_iter):
        # E-step, in log space for stability
        log_r = np.stack([np.log(pi[k]) + log_gaussian(X, mu[k], Sigma[k])
                          for k in range(K)], axis=1)          # (N, K)
        log_norm = np.logaddexp.reduce(log_r, axis=1, keepdims=True)
        r = np.exp(log_r - log_norm)
        loglik = log_norm.sum()
        # M-step
        Nk = r.sum(axis=0)
        pi = Nk / N
        mu = (r.T @ X) / Nk[:, None]
        for k in range(K):
            D = X - mu[k]
            Sigma[k] = (r[:, k, None] * D).T @ D / Nk[k] + 1e-6 * np.eye(d)
        if loglik - prev < tol:
            break
        prev = loglik
    return pi, mu, Sigma, r, loglik, it

pi, mu, Sigma, r, loglik, it = gmm_em(X, 3)
print(f"converged after {it} iterations, log-likelihood {loglik:.2f}")
print("mixing weights:", np.round(pi, 3))
print("means:\n", np.round(mu, 3))
```

```
converged after 27 iterations, log-likelihood -586.08
mixing weights: [0.337 0.326 0.337]
means:
 [[1.999 3.961]
 [3.081 2.008]
 [0.96  1.019]]
```

The means match k-means almost exactly (in a different order, cluster labels are arbitrary), and the mixing weights come out close to the true $1/3$ each. Two modern details snuck into this code. We compute the E-step in log space with `logaddexp` (the log-sum-exp trick from Chapter 2), since in high dimensions the Gaussian densities underflow to zero long before the responsibilities become meaningless. And we evaluate the Gaussian through a Cholesky factor instead of inverting $\mathbf{\Sigma}$, for the same reason we avoided inverting $\mathbf{X}^T\mathbf{X}$ in Chapter 1.

Where GMMs really pull ahead of k-means is when the clusters aren't round. Peep Figure 5.4. Two long, tilted clusters lie side by side. k-means only measures distance to the centroid, so it can't help cutting them in a way that makes each piece roughly round, and it gets them wrong. The mixture learns a full covariance for each cluster and gets them exactly right.

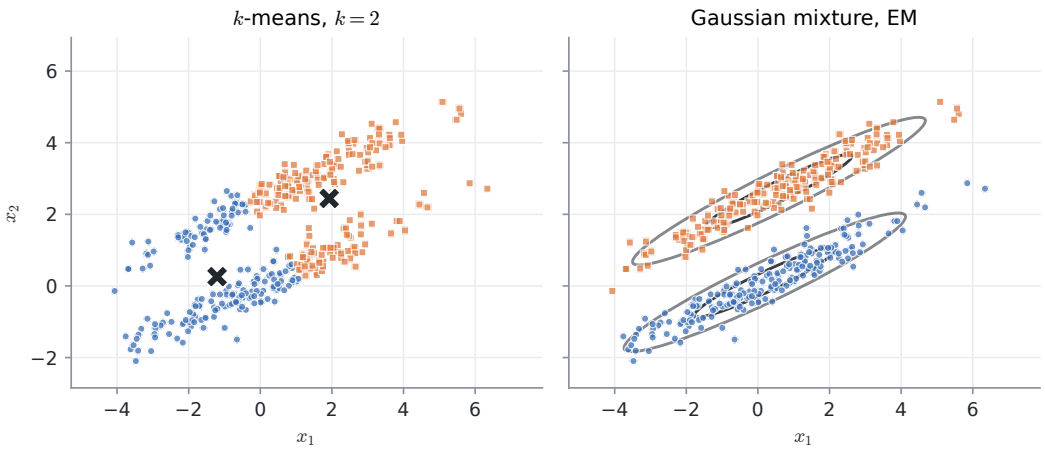


Figure 5.4: Two elongated, tilted clusters. Left, k-means assumes round clusters and slices the data across the long axis, agreeing with the true labels only about 60% of the time. Right, a Gaussian mixture fit by EM learns the shape of each cluster (ellipses at one and two standard deviations) and recovers the true labels exactly.

5.3.3 Why EM Works

We motivated EM as "set the gradient to zero and iterate", but there's a slicker argument, due to Dempster, Laird and Rubin and explained beautifully in Section 9.4 of Bishop [3], that also proves it always goes uphill. For any distribution $q(z)$ over the latent variable,

$$\log p(x) = \log \sum_z q(z) \frac{p(x,z)}{q(z)} \;\geq\; \sum_z q(z) \log \frac{p(x,z)}{q(z)} \equiv \mathcal{L}(q, \boldsymbol{\theta}), \tag{5.16}$$

by Jensen's inequality (log is concave, so the log of an average is at least the average of the logs). The right side is called the **evidence lower bound** (ELBO). The gap between the two sides is exactly the KL divergence between $q(z)$ and the posterior $\mathbb{P}(z \mid x)$, so the bound is tight when q *is* the posterior. Now EM is just coordinate ascent on $\mathcal{L}$. The E-step maximizes over q (set it to the posterior, which is the responsibilities, and the gap closes). The M-step maximizes over $\boldsymbol{\theta}$ with q fixed. Neither step can decrease $\mathcal{L}$, and after each E-step $\mathcal{L}$ equals the log-likelihood, so the log-likelihood never decreases. This same ELBO, with q replaced by a neural network, is the loss function of variational autoencoders, and a close relative of it trains diffusion models. Old tricks never die.

Problem 5.2:

Starting from the M-step objective, derive the update for $\boldsymbol{\mu}_k$ and, in one dimension, for σ_k^2. Then show that $\sum_z q(z) \log \frac{p(x,z)}{q(z)} = \log p(x) - \mathrm{KL}\big(q(z) \,\|\, \mathbb{P}(z \mid x)\big)$, which justifies the claim that the ELBO is tight exactly when q is the posterior.

Problem 5.3:

Consider a Gaussian mixture where every component has the same fixed covariance $\boldsymbol{\Sigma}_k = \epsilon \mathbf{I}$ and only the means are learned. Show that as $\epsilon \to 0$ the responsibilities become hard assignments to the nearest mean, and that EM turns into exactly the k-means algorithm. What does this say about the implicit assumptions behind k-means?

5.4 Dimensionality Reduction And Principal Component Analysis

This next section is going to seem a bit out of nowhere, and maybe it is. In the last two sections our data lived in a two-dimensional space. In real life, as we have said many times, data lives in ridiculously high-dimensional spaces. A tiny 8×8 grayscale image is already a point in $\mathbb{R}^{64}$. Is there a way to reduce the dimension of the problem but keep the information? It sounds like too much to ask, but that is exactly what **principal component analysis** (PCA) does. Consider the following toy dataset of people rating foods from 1 to 10.

	Kale	**Taco Bell**	**Sashimi**	**Pop Tarts**
Alice	10	1	2	7
Bob	7	2	1	10
Carolyn	2	9	7	3
Dave	3	6	10	2

Table 5.1: Preferences of four people for four foods.

Obviously Bob is a psychopath who likes kale and Pop Tarts. Each person is a point in a four-dimensional space, which we can't plot. The goal of PCA is to approximate the data in a lower dimensional space, so that we can visualize what's going on and keep only the essential information. Here is an approximation for the table. Let $\overline{\mathbf{x}} = (5.5, 4.5, 5, 5.5)$ be the average rating of each food, and define

$$\mathbf{v}_1 = (3, -3, -3, 3), \qquad \mathbf{v}_2 = (1, -1, 1, -1). \tag{5.17}$$

Then each person is approximately $\overline{\mathbf{x}} + a_1\mathbf{v}_1 + a_2\mathbf{v}_2$, with (a_1, a_2) equal to $(1, 1)$ for Alice, $(1, -1)$ for Bob, $(-1, -1)$ for Carolyn and $(-1, 1)$ for Dave. For example, Alice is approximated by $\overline{\mathbf{x}} + \mathbf{v}_1 + \mathbf{v}_2 = (9.5, 0.5, 3, 7.5)$, pretty close to her real $(10, 1, 2, 7)$. The vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ describe patterns of correlation between the columns. The first one says that liking kale goes with liking Pop Tarts and *not* liking Taco Bell or sashimi. So it is something like a "kale and Pop Tarts versus Taco Bell and sashimi" axis, which makes no sense as a diet but that is the data we have. The coefficients a_1, a_2 say where each person sits along those axes. How did we reduce the dimension? We took a four-dimensional space of foods and compressed each person down to two numbers (a_1, a_2) that hold most of the information. Now let's see how to find such vectors systematically for large datasets.

5.4.1 PCA As Best Approximation

We haven't yet said what a principal component *is*, so let's do that now. We want to write each data point as the mean plus a combination of orthonormal basis vectors,

$$\mathbf{x}_n = \overline{\mathbf{x}} + \sum_{d=1}^{D} \alpha_{nd}\, \mathbf{u}_d, \tag{5.18}$$

where D is the dimension of the data and $\mathbf{u}_d^T \mathbf{u}_{d'} = \delta_{dd'}$ is a constraint that we impose. If we use all D vectors this is exact for *any* orthonormal basis. Since the basis is orthonormal, the coefficients are just projections,

$$\alpha_{nd} = (\mathbf{x}_n - \overline{\mathbf{x}})^T\, \mathbf{u}_d. \tag{5.19}$$

You can think of α_{nd} as how much of $\mathbf{x}_n$ is explained by the $\mathbf{u}_d$ direction. The interesting question is what happens if we keep only the first $K < D$ directions. How accurate is the approximation

$$\hat{\mathbf{x}}_n = \overline{\mathbf{x}} + \sum_{d=1}^{K} \alpha_{nd}\, \mathbf{u}_d, \tag{5.20}$$

and which directions should we keep to make it as accurate as possible? The squared error for point n is

$$\begin{aligned} \|\mathbf{x}_n - \hat{\mathbf{x}}_n\|^2 &= \Big\| \sum_{d=K+1}^{D} \alpha_{nd}\, \mathbf{u}_d \Big\|^2 \\ &= \sum_{d=K+1}^{D} \sum_{d'=K+1}^{D} \alpha_{nd}\, \alpha_{nd'}\, \mathbf{u}_d^T \mathbf{u}_{d'} \end{aligned}$$

$$= \sum_{d=K+1}^{D} \alpha_{nd}^2, \tag{5.21}$$

since the mean cancels and the orthonormality kills the cross terms. But we know what the α's are! Substituting them back and summing over all data points,

$$\begin{aligned} L &= \sum_{n=1}^{N} \sum_{d=K+1}^{D} \left((\mathbf{x}_n - \overline{\mathbf{x}})^T \mathbf{u}_d \right)^2 \\ &= \sum_{d=K+1}^{D} \mathbf{u}_d^T \left(\sum_{n=1}^{N} (\mathbf{x}_n - \overline{\mathbf{x}}) (\mathbf{x}_n - \overline{\mathbf{x}})^T \right) \mathbf{u}_d \\ &= N \sum_{d=K+1}^{D} \mathbf{u}_d^T \, \mathbf{\Sigma} \, \mathbf{u}_d, \end{aligned} \tag{5.22}$$

where in the second line we wrote the square $(\mathbf{a}^T\mathbf{u})^2$ as $\mathbf{u}^T\mathbf{a}\mathbf{a}^T\mathbf{u}$. Ah, but we recognize the thing in the parenthesis! It is N times the **sample covariance matrix**,

$$\mathbf{\Sigma} = \frac{1}{N} \sum_{n=1}^{N} (\mathbf{x}_n - \overline{\mathbf{x}}) (\mathbf{x}_n - \overline{\mathbf{x}})^T . \tag{5.23}$$

So our objective is to minimize $\sum_{d=K+1}^{D} \mathbf{u}_d^T \mathbf{\Sigma} \mathbf{u}_d$ subject to $\mathbf{u}_d^T \mathbf{u}_d = 1$. We do the constrained optimization with Lagrange multipliers, one per direction,

$$\sum_{d=K+1}^{D} \Big[\mathbf{u}_d^T \mathbf{\Sigma} \, \mathbf{u}_d + \lambda_d \left(1 - \mathbf{u}_d^T \mathbf{u}_d \right) \Big]. \tag{5.24}$$

Differentiating with respect to $\mathbf{u}_d$ and setting it equal to zero gives $2\mathbf{\Sigma}\mathbf{u}_d - 2\lambda_d \mathbf{u}_d = 0$, that is,

$$\mathbf{\Sigma} \, \mathbf{u}_d = \lambda_d \, \mathbf{u}_d. \tag{5.25}$$

The Lagrange multipliers are eigenvalues of the covariance matrix, and the directions are its eigenvectors! Substituting back, the error we are trying to minimize becomes

$$\sum_{d=K+1}^{D} \mathbf{u}_d^T \mathbf{\Sigma} \, \mathbf{u}_d = \sum_{d=K+1}^{D} \mathbf{u}_d^T \lambda_d \, \mathbf{u}_d = \sum_{d=K+1}^{D} \lambda_d. \tag{5.26}$$

To make this small, the directions we throw away should be the ones with the *smallest* eigenvalues. Equivalently, the directions we keep, the **principal components**, are the eigenvectors of $\mathbf{\Sigma}$ with the largest eigenvalues. And since $\lambda_d = \mathbf{u}_d^T \mathbf{\Sigma} \mathbf{u}_d$ is the variance of the data along $\mathbf{u}_d$, the principal components are the directions in which the data varies the most.

Minimizing reconstruction error and maximizing retained variance are the same problem.

5.4.2 PCA Is Not Regression

A common confusion is to think the first principal component is the same as the least squares line through the data. They're different, and Figure 5.5 shows why. Least squares picks one variable as the output and minimizes the *vertical* distances to the line. PCA treats all coordinates symmetrically and minimizes the *perpendicular* distances. The least squares line is always a little flatter, because it only cares about errors in x_2. If you swap the roles of x_1 and x_2 you get a different least squares line, but PCA doesn't change at all.

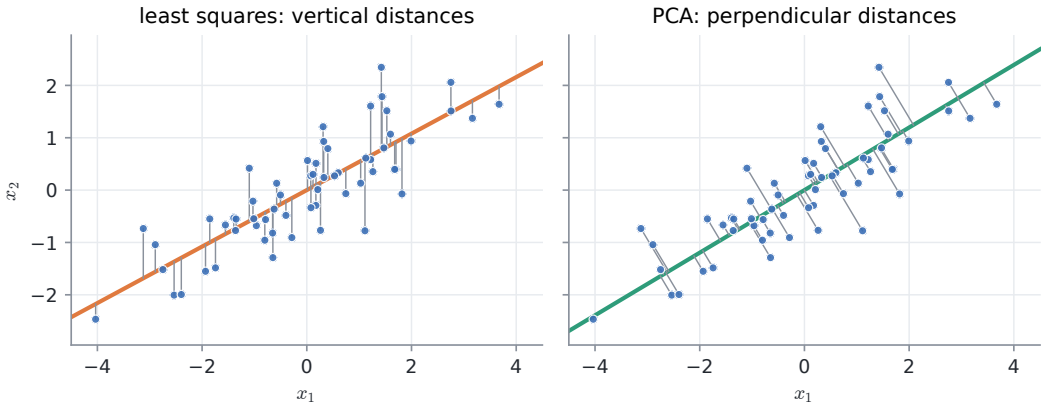


Figure 5.5: The same data fit two ways. Least squares (left) minimizes vertical distances, PCA (right) minimizes perpendicular distances. The grey segments show the errors each one is minimizing.

5.4.3 PCA Through The SVD

All of this relies on computing the covariance matrix and its eigenvectors. As it turns out, we don't even need to form the covariance matrix. Stack the centered data as rows of $\tilde{\mathbf{X}}$ (each row is $\mathbf{x}_n - \overline{\mathbf{x}}$), and take its singular value decomposition, $\tilde{\mathbf{X}} = \mathbf{U}\mathbf{S}\mathbf{V}^T$. Then

$$\mathbf{\Sigma} = \frac{1}{N}\tilde{\mathbf{X}}^T\tilde{\mathbf{X}} = \frac{1}{N}\mathbf{V}\mathbf{S}^2\mathbf{V}^T, \tag{5.27}$$

which is an eigendecomposition. So the principal directions are the right singular vectors (the columns of $\mathbf{V}$), and the eigenvalues are $\lambda_d = s_d^2/N$. The coordinates of the data in the new basis are $\tilde{\mathbf{X}}\mathbf{V} = \mathbf{U}\mathbf{S}$. Using the SVD is more accurate (same conditioning story as in Chapter 1), and it is how every library does it. Let's do the food table from scratch.

```
data = np.array([[10, 1, 2, 7],      # Alice
```

```
                  [7, 2, 1, 10],      # Bob
                  [2, 9, 7, 3],       # Carolyn
                  [3, 6, 10, 2]],     # Dave
                 dtype=float)

mean = data.mean(axis=0)
Xc = data - mean
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)

print("variance explained:", np.round(s**2 / np.sum(s**2), 3))
print("first direction:", np.round(Vt[0], 3))

K = 2
scores = Xc @ Vt[:K].T                  # each person as K numbers
reconstruction = mean + scores @ Vt[:K]
print("Alice approximated:", np.round(reconstruction[0], 2))
```

```
variance explained: [0.887 0.09  0.023 0.   ]
first direction: [-0.477  0.476  0.561 -0.48 ]
Alice approximated: [9.52 0.48 2.47 7.51]
```

The first component alone explains 89% of the variance, and it points roughly along $\pm(1, -1, -1, 1)/2$, the kale-and-Pop-Tarts axis we guessed. (The last variance is exactly zero because four centered points can span at most three dimensions.) Keeping two components reconstructs Alice to within a rating point, which to a good degree is the approximation we made at the beginning of this section. (sklearn's `PCA` gives the same numbers, possibly with flipped signs. The sign of an eigenvector is arbitrary.)

5.4.4 A Real Example, Handwritten Digits

The food table was cute, but PCA earns its keep on data we can't look at. sklearn ships a dataset of 1797 handwritten digits, each an 8×8 image, so each is a point in $\mathbb{R}^{64}$. Running the exact same SVD code and projecting onto the first two components gives the left panel of Figure 5.6. Even though PCA never saw the labels, digits of the same kind land near each other, and 0, 4 and 6 get their own territories. The right panel shows how much variance each component explains. The first two only capture about 29%, but ten components capture about 74% and twenty capture about 89%. So a 64-dimensional dataset is "really" something like 20-dimensional.

PCA is linear. It can only find flat subspaces. When the data lies on a curved surface inside the high-dimensional space, people reach for nonlinear methods, such as t-SNE and UMAP for visualization, or **autoencoders**, neural networks trained to squeeze the input through a

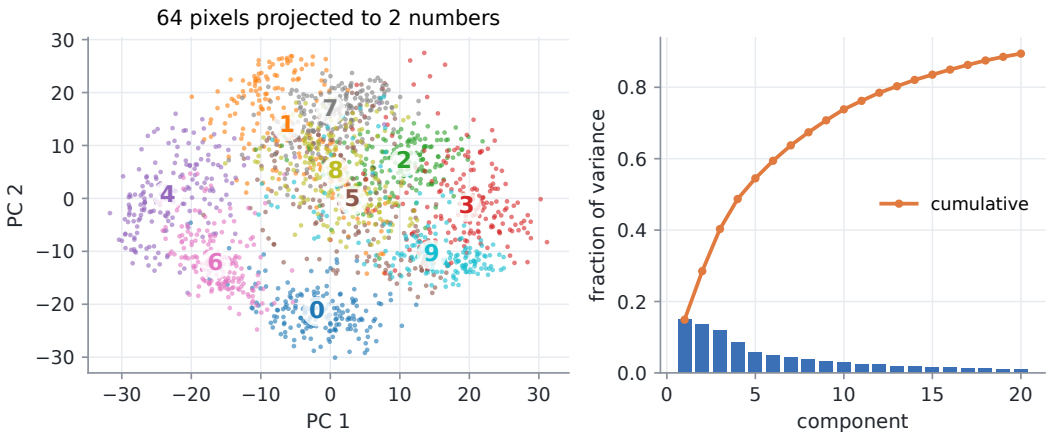


Figure 5.6: PCA on the sklearn digits. Left, every image projected onto the first two principal components and colored by its true label (which PCA never saw), with each digit drawn at the median of its class. Right, the fraction of variance explained by each component and the running total.

narrow hidden layer and reconstruct it on the other side. A linear autoencoder with squared error loss learns exactly the PCA subspace, which is a fun thing to prove once (it is the least squares story again). So PCA is still the first thing to try, and the baseline everything else is compared against.

Problem 5.4:

(1) Show directly that the unit vector $\mathbf{u}$ maximizing the variance of the projections, $\frac{1}{N}\sum_n \left((\mathbf{x}_n - \overline{\mathbf{x}})^T \mathbf{u}\right)^2$, is the top eigenvector of $\mathbf{\Sigma}$.

(2) Load the digits with `sklearn.datasets.load_digits`, run PCA with the SVD, and reconstruct a few images using $K = 2, 5, 10, 20$ components. Display them as 8×8 images next to the originals. At what K does each digit become recognizable?

(3) Run k-means with $k = 10$ on the 64-dimensional digits and on their 20-dimensional PCA projection. Compare how well the clusters match the true labels. Is anything lost by clustering in the reduced space?

Part III

More Tools for the Toolbox

6

Support Vector Machines

"Good fences make good neighbors."

– Robert Frost

Support vector machines had massive hype around them for a long time as maybe the best off-the-shelf supervised learning method. From the late 1990s until deep learning took over image recognition around 2012, that was arguably true. They are still excellent on small and medium datasets, and the ideas they introduced, margins and kernels especially, show up all over modern theory. The problem we are going to solve is one we have seen already, binary classification. Suppose we have data labeled positive or negative (circles and squares, up and down, etc etc), and for now assume it is **linearly separable**, meaning some hyperplane splits the two classes perfectly.

Logistic regression would find *a* separating hyperplane. Actually, as we saw at the end of Chapter 2, on separable data it has a hard time deciding which one, because there are infinitely many. Support vector machines take a different approach. Instead of maximizing the likelihood of the data, we look at the points of each class that are closest to the boundary, and we put the boundary as far from them as possible. Figure 6.1 shows the idea. The boundary sits in the middle of the widest possible empty street between the classes. The question we are going to answer is, what hyperplane maximizes the width of that street?

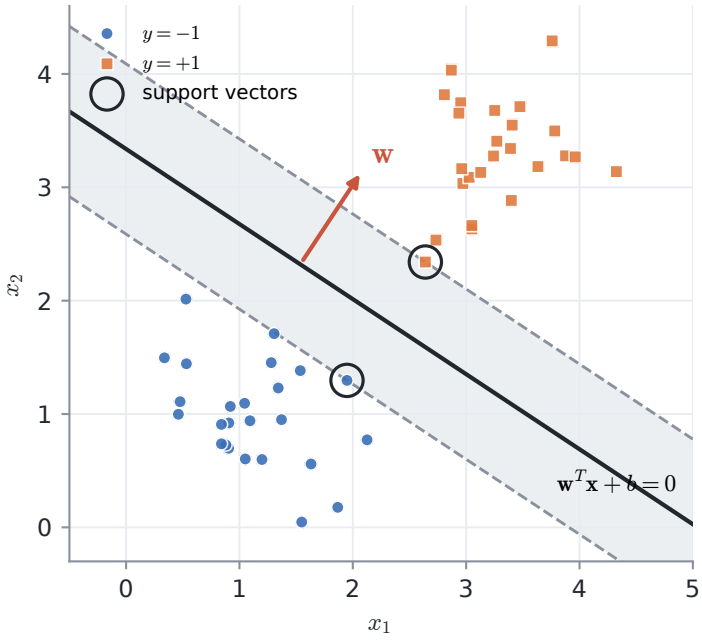


Figure 6.1: The maximum margin classifier. The solid line is the decision boundary, the dashed lines are the edges of the margin (the "gutters"), and the circled points sitting on the gutters are the support vectors. The vector $\mathbf{w}$ is perpendicular to the boundary.

6.1 Finding The Margin

A hyperplane in $\mathbb{R}^M$ can be written as the set of points with $\mathbf{w}^T\mathbf{x} + b = 0$. The vector $\mathbf{w}$ is perpendicular to the hyperplane (if $\mathbf{x}$ and $\mathbf{x}'$ are both on it, then $\mathbf{w}^T(\mathbf{x} - \mathbf{x}') = 0$), and with respect to Figure 6.1, $\mathbf{w}$ points across the street. The number b is the bias, which shifts the hyperplane away from the origin, like the intercept of a line. Taking the dot product $\mathbf{w}^T\mathbf{x}$ projects $\mathbf{x}$ onto the direction of $\mathbf{w}$, so the sign of $\mathbf{w}^T\mathbf{x} + b$ tells us which side of the boundary we're on. We use the decision rule

$$\mathbf{w}^T\mathbf{x} + b \geq 0 \implies \text{label } +1, \qquad \mathbf{w}^T\mathbf{x} + b < 0 \implies \text{label } -1. \tag{6.1}$$

The equality case is exactly the decision boundary (think back to logistic regression, where the boundary was $\boldsymbol{\theta}^T\mathbf{x} = 0$). The trouble is we don't know $\mathbf{w}$ and we don't know b.

Here's a subtlety. If we multiply both $\mathbf{w}$ and b by any positive number, the hyperplane doesn't change. So we have a free scale, and we fix it in the most convenient way. Without loss of generality, require that the closest points to the boundary satisfy $\mathbf{w}^T\mathbf{x} + b = +1$ on the positive side and $\mathbf{w}^T\mathbf{x} + b = -1$ on the negative side. Every other point is further out,

so $\mathbf{w}^T\mathbf{x} + b \geq 1$ for positive points and ≤ -1 for negative ones. If $\mathbf{w}^T\mathbf{x} + b$ is very large, we are deep inside the positive region and very confident. Introducing labels $y_i \in \{-1, +1\}$ (note, not $\{0, 1\}$ this time), both conditions compact into one,

$$y_i \left(\mathbf{w}^T\mathbf{x}_i + b\right) \geq 1 \qquad \text{for all } i, \tag{6.2}$$

with equality exactly for the points in the gutters. Those points are the **support vectors**.

Now the width. Let $\mathbf{x}_+$ be a support vector on the positive gutter and $\mathbf{x}_-$ one on the negative gutter. The vector $\mathbf{x}_+ - \mathbf{x}_-$ goes across the street, and dotting it with the unit normal $\mathbf{w}/\|\mathbf{w}\|$ picks out the part perpendicular to the boundary, which is the width,

$$\begin{aligned} \text{width} &= (\mathbf{x}_+ - \mathbf{x}_-)^T \frac{\mathbf{w}}{\|\mathbf{w}\|} \\ &= \frac{(1-b) - (-1-b)}{\|\mathbf{w}\|} \\ &= \frac{2}{\|\mathbf{w}\|}, \end{aligned} \tag{6.3}$$

where in the second line we used $\mathbf{w}^T\mathbf{x}_\pm = \pm 1 - b$ from the gutter conditions. (The first edition had $\|\mathbf{w}\|^2$ in the denominator. It didn't change where the maximum was, but it was wrong.) So maximizing the width means minimizing $\|\mathbf{w}\|$. For convenience we minimize $\frac{1}{2}\|\mathbf{w}\|^2$ instead, which has the same minimizer (the square root is monotone) and nicer derivatives. The full problem is

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i \left(\mathbf{w}^T\mathbf{x}_i + b\right) \geq 1, \quad i = 1, \ldots, N. \tag{6.4}$$

What we have now is a convex quadratic objective with linear inequality constraints, which is called a **quadratic program**, and the nice thing about it is that it has a unique solution.

Problem 6.1:

Show that the distance from a point $\mathbf{x}_0$ to the hyperplane $\mathbf{w}^T\mathbf{x}+b = 0$ is $|\mathbf{w}^T\mathbf{x}_0+b|/\|\mathbf{w}\|$. Use it to give a second derivation of the width $2/\|\mathbf{w}\|$, and to explain why fixing the closest points at ± 1 loses no generality.

6.1.1 The Dual Problem

In true physics fashion, we define a **Lagrangian**, and we enforce each constraint with its own Lagrange multiplier $\lambda_i \geq 0$,

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\lambda}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{N} \lambda_i \left[y_i \left(\mathbf{w}^T \mathbf{x}_i + b \right) - 1 \right]. \tag{6.5}$$

(Sadly there is no action to minimize, so the physics analogy stops here.) Because the constraints are inequalities, the multipliers have to be nonnegative, and the theory of constrained optimization (the Karush–Kuhn–Tucker or KKT conditions, see [5]) tells us that at the optimum

$$\lambda_i \left[y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 \right] = 0 \quad \text{for every } i. \tag{6.6}$$

This is called **complementary slackness**. For each point, either the constraint is tight (the point is in the gutter) or its multiplier is zero. Hold that thought, it is the most important equation in the chapter. Now take derivatives and set them equal to zero. With respect to $\mathbf{w}$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{N} \lambda_i y_i \mathbf{x}_i = 0 \quad \Longrightarrow \quad \mathbf{w} = \sum_{i=1}^{N} \lambda_i y_i \mathbf{x}_i, \tag{6.7}$$

and with respect to b,

$$\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^{N} \lambda_i y_i = 0. \tag{6.8}$$

Now we substitute these back into the Lagrangian. Using (6.7) for $\mathbf{w}$ in both places it appears,

$$\begin{aligned} \mathcal{L} &= \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i,j} \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - b \sum_i \lambda_i y_i + \sum_i \lambda_i \\ &= \sum_{i=1}^{N} \lambda_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \lambda_i \lambda_j y_i y_j \, \mathbf{x}_i^T \mathbf{x}_j, \end{aligned} \tag{6.9}$$

where the b term vanished because of (6.8). This is the **dual problem**. We maximize (6.9) over $\boldsymbol{\lambda}$ subject to $\lambda_i \geq 0$ and $\sum_i \lambda_i y_i = 0$. It is another quadratic program, but now with one variable per data point instead of one per feature.

Once we have the optimal λ's, (6.7) gives us $\mathbf{w}$. And here's where complementary slackness pays off. Every point that is *not* in a gutter has $\lambda_i = 0$, so it contributes nothing to $\mathbf{w}$. The solution depends only on the support vectors! You could delete every other training point, retrain, and get the exact same classifier. To get b, pick any support vector $\mathbf{x}_s$ (one with

$\lambda_s > 0$), and solve $y_s(\mathbf{w}^T\mathbf{x}_s + b) = 1$ for $b = y_s - \mathbf{w}^T\mathbf{x}_s$ (using $1/y_s = y_s$). In practice we average this over all support vectors to reduce round-off error.

6.2 An Illustrative Example

Let's go back to the two clusters from Chapter 2 (Figure 2.1) and solve the dual directly with a quadratic program solver. Generic QP solvers like `cvxopt` want the problem in the standard form

$$\min_{\boldsymbol{\lambda}} \ \frac{1}{2}\boldsymbol{\lambda}^T\mathbf{P}\boldsymbol{\lambda} + \mathbf{q}^T\boldsymbol{\lambda} \quad \text{subject to} \quad \mathbf{G}\boldsymbol{\lambda} \le \mathbf{h}, \quad \mathbf{A}\boldsymbol{\lambda} = \mathbf{b}_{\text{eq}}. \tag{6.10}$$

Maximizing (6.9) is minimizing its negative, so $P_{ij} = y_i y_j \mathbf{x}_i^T\mathbf{x}_j$ and $\mathbf{q} = -\mathbf{1}$. The constraint $\lambda_i \ge 0$ becomes $-\mathbf{I}\boldsymbol{\lambda} \le \mathbf{0}$, and $\sum_i \lambda_i y_i = 0$ becomes $\mathbf{y}^T\boldsymbol{\lambda} = 0$.

```
import numpy as np
from cvxopt import matrix, solvers
solvers.options["show_progress"] = False

np.random.seed(0)
x1_a = np.random.normal(1, 0.3, 50); x2_a = np.random.normal(1, 0.3, 50)
x1_b = np.random.normal(3, 0.5, 50); x2_b = np.random.normal(5, 0.5, 50)
X = np.c_[np.r_[x1_a, x1_b], np.r_[x2_a, x2_b]]
y = np.r_[-np.ones(50), np.ones(50)]        # labels must be -1 and +1
N = len(y)

# P_ij = y_i y_j x_i . x_j, built with one matrix product
Gram = X @ X.T
P = matrix(np.outer(y, y) * Gram)
q = matrix(-np.ones(N))
G = matrix(-np.eye(N))                      # -lambda_i <= 0
h = matrix(np.zeros(N))
A = matrix(y.reshape(1, -1))                # sum_i lambda_i y_i = 0
b_eq = matrix(0.0)

lambdas = np.ravel(solvers.qp(P, q, G, h, A, b_eq)["x"])

sv = lambdas > 1e-5                         # support vectors
w = ((lambdas * y)[sv, None] * X[sv]).sum(axis=0)
b = np.mean(y[sv] - X[sv] @ w)

print("support vector indices:", np.where(sv)[0])
print("their multipliers:", np.round(lambdas[sv], 4))
print("w =", w, " b =", b)
print("margin width:", 2 / np.linalg.norm(w))
```

```
support vector indices: [47 83 90]
their multipliers: [0.2319 0.1553 0.0766]
w = [0.35503642 0.58111213]  b = -2.330297116682462
margin width: 2.9369171866013364
```

These are the same $\mathbf{w}$ and b as in the first edition, so at least that part was right. Out of 100 multipliers, only three are nonzero (the solver returns tiny values like 10^{-10} for the rest, which are zero up to numerical precision). Those three points pin down the entire classifier. Peep Figure 6.2. (In the first edition we printed all 100 multipliers. We have spared you.)

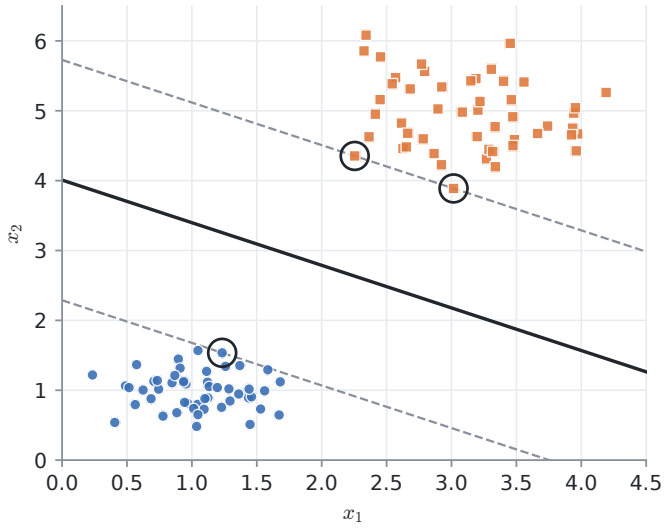


Figure 6.2: The maximum margin boundary for the Chapter 2 clusters, computed by solving the dual. Only the three circled points have nonzero multipliers.

Problem 6.2:

Using the λ's from the code, verify numerically that $\sum_i \lambda_i y_i = 0$, that $y_i(\mathbf{w}^T\mathbf{x}_i + b) = 1$ for each support vector, and that the optimal value of the dual equals $\frac{1}{2}\|\mathbf{w}\|^2$ (strong duality). Then delete all non-support vectors from the data and solve again. Do you get the same $\mathbf{w}$ and b?

6.3 Soft Margins

Real data is rarely separable. One mislabeled point on the wrong side, and (6.4) has no solution at all, since no hyperplane satisfies all the constraints. Even when the data is

separable, a single outlier near the boundary can drag the hyperplane far from where it should be. The fix is to let points violate the margin, but make them pay for it. Give each point a **slack variable** $\xi_i \geq 0$ and relax the constraint to $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$. A point with $0 < \xi_i \leq 1$ is inside the street but on the correct side, and one with $\xi_i > 1$ is misclassified. Then add the total slack to the objective,

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i \quad \text{subject to} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \tag{6.11}$$

The constant $C > 0$ sets the trade-off. Large C makes violations expensive, so we get a narrow margin that tries hard to classify every training point (low bias, high variance). Small C tolerates violations in exchange for a wide, stable margin (higher bias, lower variance). Chapter 3 strikes again, and C is chosen by cross-validation like any other regularization knob. Figure 6.3 shows both extremes on overlapping data.

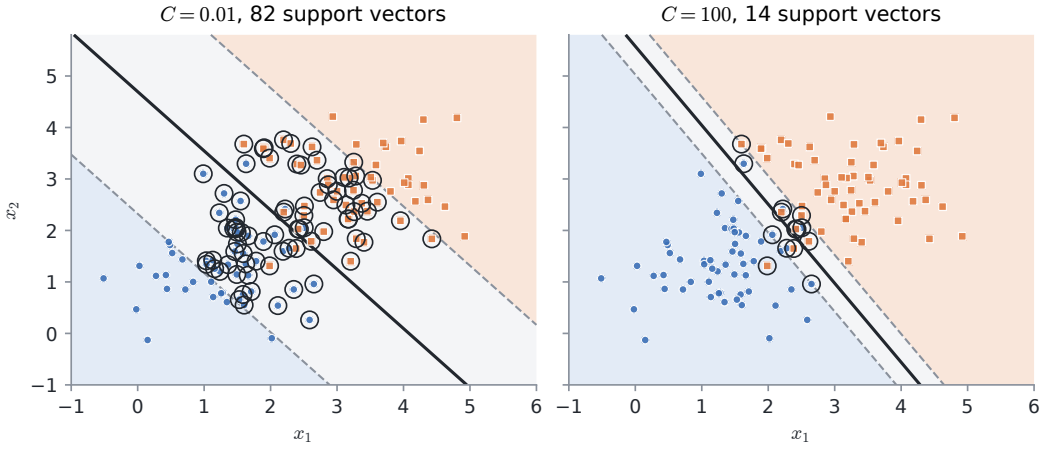


Figure 6.3: Soft margin SVMs on overlapping classes. With small C (left), the margin is wide and many points are support vectors. With large C (right), the margin shrinks to accommodate the training points as well as it can.

Going through the same dual derivation with the slacks gives a surprisingly small change. The dual objective (6.9) is identical, and the only difference is that the multipliers now live in a box, $0 \leq \lambda_i \leq C$ (Problem 6.3). Points strictly outside the margin still have $\lambda_i = 0$, points exactly on the gutters have $0 < \lambda_i < C$, and points inside the margin or misclassified have $\lambda_i = C$.

6.3.1 The Hinge Loss, Or SVMs Without A QP Solver

There's another way to read (6.11). At the optimum each slack is as small as it's allowed to be, $\xi_i = \max\{0,\, 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b)\}$. Substituting this in, the constraints disappear and we get an unconstrained problem,

$$\min_{\mathbf{w},b}\ \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{N}\sum_{i=1}^{N}\max\left\{0,\ 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b)\right\}, \tag{6.12}$$

with $\lambda = 1/(NC)$. The function $\max\{0, 1-m\}$ of the margin $m = y\,(\mathbf{w}^T\mathbf{x} + b)$ is called the **hinge loss**. So a soft margin SVM is "hinge loss plus ridge penalty". Compare it with logistic regression, which in the same ± 1 notation is "$\log(1 + e^{-m})$ plus ridge penalty". The two losses look very similar for misclassified points. The difference is that the hinge loss is *exactly zero* once a point is past the margin, so those points stop influencing the solution at all, which is where the sparsity in support vectors comes from. Logistic loss never quite reaches zero, so every point always has a tiny say.

Equation (6.12) also gives us a modern, scalable way to train SVMs. The hinge loss isn't differentiable at $m = 1$, but it has a **subgradient**, which is $-y_i\mathbf{x}_i$ when $m_i < 1$ and $\mathbf{0}$ otherwise, and stochastic subgradient descent works fine with that. This is the **Pegasos** algorithm [14], and it is the same mini-batch loop we have been writing since Chapter 1. It scales to millions of points, where building the $N \times N$ matrix for the dual would be hopeless.

```
def pegasos(X, y, lam=0.05, epochs=300, batch_size=10, seed=0):
    """Linear soft-margin SVM by stochastic subgradient descent."""
    rng = np.random.default_rng(seed)
    mu = X.mean(axis=0)
    Xc = X - mu                      # centering keeps the bias small and tame
    N, d = Xc.shape
    w, b, t = np.zeros(d), 0.0, 0
    for _ in range(epochs):
        for idx in np.array_split(rng.permutation(N), N // batch_size):
            t += 1
            eta = 1.0 / (lam * t)                    # Pegasos step size
            margins = y[idx] * (Xc[idx] @ w + b)
            active = margins < 1                     # points with nonzero hinge loss
            grad_w = lam * w - (y[idx, None] * Xc[idx])[active].sum(0) / len(idx)
            grad_b = -y[idx][active].sum() / len(idx)
            w -= eta * grad_w
            b -= eta * grad_b
    return w, b - w @ mu             # undo the centering

w_sgd, b_sgd = pegasos(X, y)
```

```
cos = w_sgd @ w / (np.linalg.norm(w_sgd) * np.linalg.norm(w))
print("Pegasos w =", w_sgd, " b =", b_sgd)
print("angle to the QP direction (degrees):", np.degrees(np.arccos(cos)))
print("training accuracy:", np.mean(np.sign(X @ w_sgd + b_sgd) == y))
```

```
Pegasos w = [0.35379122 0.68565563]  b = -2.4481739418067883
angle to the QP direction (degrees): 4.130005876218975
training accuracy: 1.0
```

It classifies every training point correctly, and its $\mathbf{w}$ points within about four degrees of the exact hard margin solution. They aren't identical because they solve different problems. With a finite λ the soft margin happily lets a few points into the street in exchange for a wider one, so it leans on more points than the three hard margin support vectors.

Problem 6.3:

Write the Lagrangian of (6.11) with multipliers $\lambda_i \geq 0$ for the margin constraints and $\mu_i \geq 0$ for $\xi_i \geq 0$. Show that setting $\partial \mathcal{L}/\partial \xi_i = 0$ gives $\lambda_i = C - \mu_i$, and conclude that the dual is (6.9) with the box constraint $0 \leq \lambda_i \leq C$. Then plot the hinge loss $\max\{0, 1-m\}$ and the logistic loss $\log_2(1+e^{-m})$ on the same axes as functions of m.

6.4 Kernels

Everything so far only works for data that is (nearly) linearly separable. What do we do when it isn't, as in Figure 6.4? We use the trick from Chapter 3, a basis function expansion. Instead of $\mathbf{x}_i$, we feed the SVM a feature vector $\mathbf{\Phi}(\mathbf{x}_i)$. In Figure 6.4 the extra feature $x_1^2 + x_2^2$ is enough to make the classes separable by a plane. Our dual (6.9) becomes

$$\max_{\boldsymbol{\lambda}} \sum_{i=1}^{N} \lambda_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \lambda_i \lambda_j y_i y_j \, \mathbf{\Phi}(\mathbf{x}_i)^T \mathbf{\Phi}(\mathbf{x}_j) \quad \text{s.t.} \quad \sum_{i=1}^{N} y_i \lambda_i = 0, \quad 0 \leq \lambda_i \leq C. \tag{6.13}$$

Now notice something. The data only appears through inner products $\mathbf{\Phi}(\mathbf{x}_i)^T \mathbf{\Phi}(\mathbf{x}_j)$. The same is true of predictions, since by (6.7)

$$\mathbf{w}^T \mathbf{\Phi}(\mathbf{x}) + b = \sum_{i=1}^{N} \lambda_i y_i \, \mathbf{\Phi}(\mathbf{x}_i)^T \mathbf{\Phi}(\mathbf{x}) + b. \tag{6.14}$$

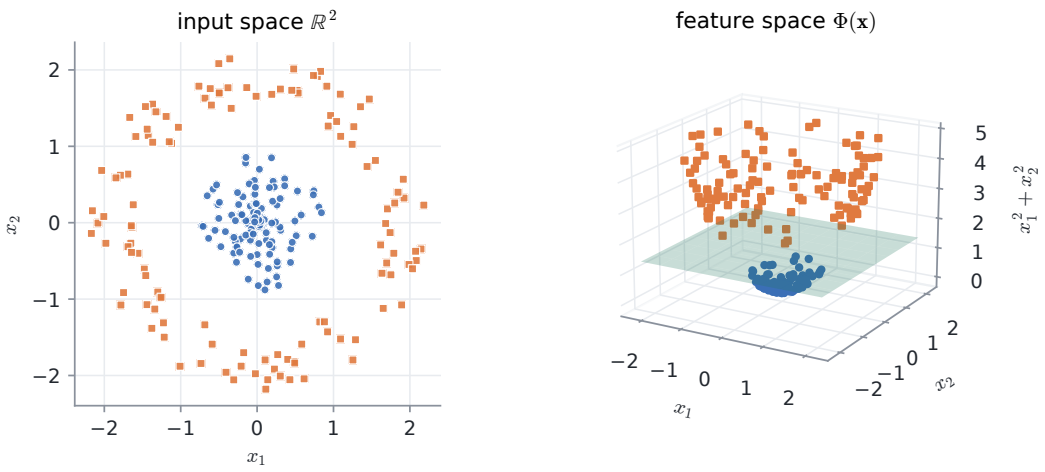


Figure 6.4: Left, two classes that no line can separate. Right, lifting each point to $(x_1, x_2, x_1^2 + x_2^2)$ makes them separable by a flat plane (green). A plane in the lifted space is a circle back in the original space.

So we never need the features themselves, only a function that computes their inner products. A **kernel** is a function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ for which there exists a feature map $\mathbf{\Phi}$ with $K(\mathbf{x}, \mathbf{x}') = \mathbf{\Phi}(\mathbf{x})^T \mathbf{\Phi}(\mathbf{x}')$. Replacing every inner product with a kernel evaluation is the **kernel trick**, and it turns the linear SVM into a nonlinear one at no extra cost,

$$\max_{\boldsymbol{\lambda}} \sum_{i=1}^{N} \lambda_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \lambda_i \lambda_j y_i y_j \, K(\mathbf{x}_i, \mathbf{x}_j), \qquad f(\mathbf{x}) = \sum_{i=1}^{N} \lambda_i y_i K(\mathbf{x}_i, \mathbf{x}) + b. \tag{6.15}$$

Why is this such a big deal? Because some kernels are cheap to compute even though their feature maps are enormous. Take the **polynomial kernel** $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z})^2$ in two dimensions. Expanding,

$$\begin{aligned} (\mathbf{x}^T \mathbf{z})^2 &= (x_1 z_1 + x_2 z_2)^2 \\ &= x_1^2 z_1^2 + 2 x_1 x_2 z_1 z_2 + x_2^2 z_2^2 \\ &= \left(x_1^2,\, \sqrt{2} x_1 x_2,\, x_2^2\right)^T \left(z_1^2,\, \sqrt{2} z_1 z_2,\, z_2^2\right), \end{aligned} \tag{6.16}$$

so it is an inner product of three-dimensional feature vectors, computed with one two-dimensional dot product and a square. For $(\mathbf{x}^T \mathbf{z} + 1)^d$ in M dimensions the feature space has $\binom{M+d}{d}$ dimensions (Problem 3.1!), and the kernel still costs one dot product. The most popular kernel of all is the **radial basis function** (RBF) or Gaussian kernel,

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\gamma \|\mathbf{x} - \mathbf{z}\|^2\right), \tag{6.17}$$

whose feature space is *infinite-dimensional* (Problem 6.4). With the RBF kernel, $f(\mathbf{x})$ is a sum of Gaussian bumps centered at the support vectors, and γ sets how wide the bumps are. Not every function of two arguments is a valid kernel. The condition (Mercer's theorem) is that the matrix $K(\mathbf{x}_i, \mathbf{x}_j)$ is positive semi-definite for every possible set of points, which is exactly what makes the dual a convex problem.

Here is the QP solver from before, generalized to any kernel and any C. The only changes are the Gram matrix and the extra upper bound on the λ's.

```
def rbf_kernel(A, B, gamma=1.0):
    d2 = (A**2).sum(1)[:, None] + (B**2).sum(1)[None, :] - 2 * A @ B.T
    return np.exp(-gamma * d2)

def kernel_svm(X, y, kernel, C=1.0):
    N = len(y)
    K = kernel(X, X)
    P = matrix(np.outer(y, y) * K)
    q = matrix(-np.ones(N))
    G = matrix(np.vstack([-np.eye(N), np.eye(N)]))     # 0 <= lambda <= C
    h = matrix(np.r_[np.zeros(N), C * np.ones(N)])
    A = matrix(y.reshape(1, -1)); b_eq = matrix(0.0)
    lam = np.ravel(solvers.qp(P, q, G, h, A, b_eq)["x"])
    sv = lam > 1e-6
    on_margin = sv & (lam < C - 1e-6)                  # 0 < lambda < C
    b = np.mean(y[on_margin] - (lam * y) @ K[:, on_margin])
    def decision(Xnew):
        return (lam[sv] * y[sv]) @ kernel(X[sv], Xnew) + b
    return decision, sv

from sklearn.datasets import make_moons
Xm, ym = make_moons(200, noise=0.2, random_state=0)
ym = 2.0 * ym - 1                                      # to -1/+1

f, sv = kernel_svm(Xm, ym, lambda A, B: rbf_kernel(A, B, gamma=1.0), C=1.0)
print("support vectors:", sv.sum(), "of", len(ym))
print("training accuracy:", np.mean(np.sign(f(Xm)) == ym))
```

```
support vectors: 54 of 200
training accuracy: 0.95
```

Figure 6.5 shows the resulting boundary and what happens when we crank γ up. Narrow bumps let the model draw a little island around every point. We now have two knobs, C and γ, and they are tuned together by cross-validation over a grid.

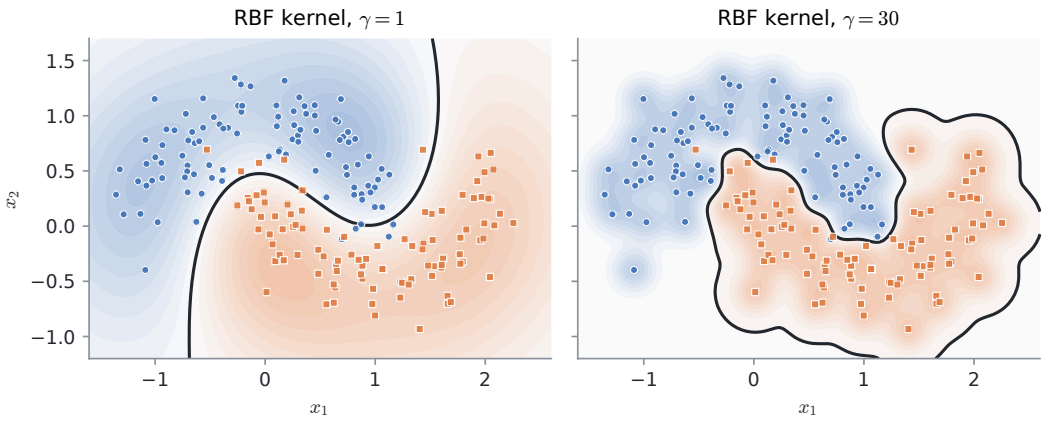


Figure 6.5: RBF kernel SVMs on the two moons. With $\gamma = 1$ (left) the boundary is smooth. With $\gamma = 30$ (right) each support vector gets a narrow bump, and the boundary wraps tightly around the training points, a textbook case of overfitting.

6.4.1 Kernels Today

If kernel SVMs are so elegant, why did deep learning win? The main reason is scale. The dual needs the $N \times N$ kernel matrix, which is fine for 10^4 points and impossible for 10^8. There's also a deeper reason. A kernel is a *fixed* feature map that we choose up front, just like the hand-picked bases of Chapter 3, while a neural network *learns* its features from data. But kernels never really left. Gaussian processes, which are Bayesian linear regression (Chapter 7) with a kernel, are the standard tool for uncertainty-aware regression on small data. And one of the more surprising results of recent theory is that very wide neural networks trained by gradient descent behave like kernel machines, with a specific kernel called the neural tangent kernel. The two camps turned out to be closer than anyone thought.

Problem 6.4:

(1) In one dimension, write $e^{-\gamma(x-z)^2} = e^{-\gamma x^2} e^{-\gamma z^2} e^{2\gamma xz}$ and Taylor expand the last factor to show that the RBF kernel is an inner product of feature vectors with infinitely many components $\phi_n(x) = e^{-\gamma x^2} \sqrt{(2\gamma)^n / n!}\, x^n$.

(2) Show that if K_1 and K_2 are kernels then so are $K_1 + K_2$ and cK_1 for $c > 0$. (The product $K_1 K_2$ is too, but that one is harder.)

(3) Use `kernel_svm` with 5-fold cross-validation (Chapter 3) to choose $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{0.1, 1, 10, 30\}$ on the moons. Which pair wins, and

how does its test accuracy on a fresh `make_moons` sample compare with $\gamma = 30$?

7

Bayesian Methods

"When the facts change, I change my mind. What do you do, sir?"

– attributed to John Maynard Keynes

There is no doubt in our minds that Bayesian statistics is one of the most important developments in statistics. It is a different philosophy from its frequentist counterpart, and it has quietly been sitting underneath several things we already did. In this chapter we develop it from the ground up, use it to finally explain where ridge regression comes from, build a regression model that knows when it doesn't know, and then use Bayes' theorem to build a classifier, linear discriminant analysis.

7.1 Bayesian Statistics

Let's begin by comparing and contrasting the Bayesian view with the classical frequentist interpretation of probability. In classical probability and statistics, a probability is a limiting frequency. If we flip a coin 1000 times and 500 land heads, we suspect that the probability of heads is 1/2, and the true probability is what that fraction would converge to if we flipped forever. Because of this, unknown parameters like the p of a Bernoulli trial are *fixed* numbers. There's no probability distribution over them, since there's nothing random about them. We just don't know them. (There's also the whole business of interpreting confidence intervals, which is not too important for machine learning.)

How is this contrasted with the Bayesian philosophy? In Bayesian statistics, a probability is

a degree of belief. For example, we believe you will drink coffee tomorrow with probability 0.5. There is no limiting frequency here, since we can't rerun tomorrow a thousand times. Under this view we are perfectly happy to put a probability distribution on an unknown parameter, because that distribution describes what we believe about it. We start with a guess, called the **prior**, and as we observe data we update it. The updated belief is called the **posterior**. So we can make probability statements about p even though p itself is a fixed number. The randomness is in our knowledge, not in the coin.

7.1.1 The Posterior Distribution

We begin with a prior distribution $f(\theta)$ on the parameter. What we want is the posterior $f(\theta \mid x_1, \dots, x_n)$, our belief about the parameter after seeing the data. Bayes' theorem gives it to us. For a parameter taking discrete values,

$$\mathbb{P}(\Theta = \theta \mid x_1, \dots, x_n) = \frac{\mathbb{P}(x_1, \dots, x_n \mid \Theta = \theta)\, \mathbb{P}(\Theta = \theta)}{\sum_{\theta'} \mathbb{P}(x_1, \dots, x_n \mid \Theta = \theta')\, \mathbb{P}(\Theta = \theta')}, \tag{7.1}$$

where we used the law of total probability on the denominator. For continuous parameters the sum becomes an integral,

$$f(\theta \mid x) = \frac{f(x \mid \theta)\, f(\theta)}{\int f(x \mid \theta')\, f(\theta')\, d\theta'}. \tag{7.2}$$

The function $f(x \mid \theta)$, viewed as a function of θ, is our old friend the **likelihood**. In most applications we won't care about the denominator, which is just a normalization constant that doesn't depend on θ. It is more concise to write

$$\text{posterior} \propto \text{likelihood} \times \text{prior}. \tag{7.3}$$

(The denominator, called the **evidence** or marginal likelihood, does matter when comparing different models, and it is often the hardest thing in Bayesian statistics to compute. For now, we can ignore it.)

Let us see an example. Consider i.i.d. coin flips $X_1, \dots, X_n \sim \text{Bernoulli}(p)$, and suppose we want to estimate p. Before seeing anything, our prior is uniform on $[0, 1]$, which is a way of saying we're ignorant. If s of the n flips are heads, the likelihood is $p^s(1-p)^{n-s}$, and plugging into (7.2),

$$f(p \mid x_1, \dots, x_n) \propto p^s(1-p)^{n-s} = p^{(s+1)-1}(1-p)^{(n-s+1)-1}. \tag{7.4}$$

The reason we included the funny $+1-1$ is that we recognize the **beta distribution**,

$$f(p) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\,\Gamma(\beta)}\, p^{\alpha-1}(1-p)^{\beta-1}, \qquad 0 \le p \le 1. \tag{7.5}$$

So if we take $\alpha = s+1$ and $\beta = n-s+1$, we get the normalization constant for free, and the posterior is

$$f(p \mid x_1, \dots, x_n) = \frac{\Gamma(n+2)}{\Gamma(s+1)\,\Gamma(n-s+1)}\, p^{s}(1-p)^{n-s}. \tag{7.6}$$

The mean of a $\mathrm{Beta}(\alpha,\beta)$ is $\alpha/(\alpha+\beta)$, so the posterior mean of p is

$$\bar{p} = \frac{s+1}{n+2}. \tag{7.7}$$

This is Laplace's famous **rule of succession**. Compare it with the maximum likelihood estimate s/n. If we flip once and get heads, the MLE says $p = 1$, which is absurdly overconfident. The Bayesian estimate says $2/3$. It is as if we had seen one imaginary head and one imaginary tail before starting, and those pseudo-counts keep us from jumping to conclusions. As n grows the pseudo-counts stop mattering and the two estimates agree. Figure 7.1 shows the posterior sharpening as data comes in.

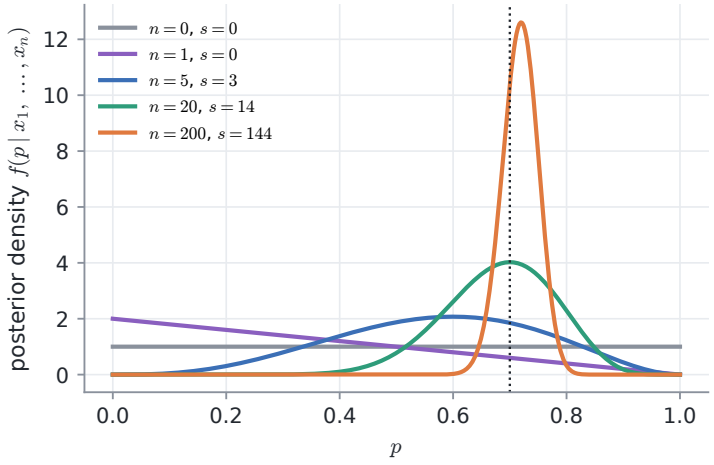


Figure 7.1: Posterior over the probability of heads p for a coin with true $p = 0.7$ (dotted), starting from a uniform prior ($n = 0$). As flips accumulate, the posterior concentrates around the truth and its width shrinks like $1/\sqrt{n}$.

Notice that we started with a beta prior (the uniform distribution is just $\mathrm{Beta}(1,1)$) and then we ended up with a beta posterior too. When the prior and posterior are in the same family, we say the prior is **conjugate** to the likelihood. Conjugate priors are what make

Bayesian calculations doable with pen and paper. When we don't have one, we reach for approximations (Laplace approximations, variational inference, or Markov chain Monte Carlo), which is a whole other book.

Problem 7.1:

Redo the coin example with a general Beta(α, β) prior. Show that the posterior is Beta$(\alpha + s, \beta + n - s)$ and that its mean can be written as a weighted average

$$\bar{p} = w\,\frac{\alpha}{\alpha+\beta} + (1-w)\,\frac{s}{n}, \qquad w = \frac{\alpha+\beta}{\alpha+\beta+n},$$

of the prior mean and the MLE. Interpret $\alpha + \beta$ as a number of imaginary prior observations.

7.1.2 MAP Estimation, And Where Ridge Came From

A full posterior is the most complete answer, but sometimes we just want one number. One option is the posterior mean, as above. Another is the most probable value, the **maximum a posteriori** (MAP) estimate,

$$\hat{\boldsymbol{\theta}}_{\text{MAP}} = \arg\max_{\boldsymbol{\theta}}\; f(\boldsymbol{\theta} \mid \mathcal{D}) = \arg\max_{\boldsymbol{\theta}}\; \Big[\log f(\mathcal{D} \mid \boldsymbol{\theta}) + \log f(\boldsymbol{\theta})\Big], \tag{7.8}$$

where $\mathcal{D}$ is the data. It is maximum likelihood plus an extra term from the prior. Now we can finally cash the check we wrote in Chapter 1. Take linear regression with Gaussian noise, $y_i = \boldsymbol{\theta}^T \mathbf{x}_i + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, and put an independent Gaussian prior on each weight, $\theta_j \sim \mathcal{N}(0, \tau^2)$. Then

$$\begin{aligned}
\log f(\boldsymbol{\theta} \mid \mathcal{D}) &= \log f(\mathcal{D} \mid \boldsymbol{\theta}) + \log f(\boldsymbol{\theta}) + \text{const} \\
&= -\frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - \boldsymbol{\theta}^T\mathbf{x}_i)^2 - \frac{1}{2\tau^2}\sum_j \theta_j^2 + \text{const} \\
&= -\frac{1}{2\sigma^2}\left[\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 + \frac{\sigma^2}{\tau^2}\|\boldsymbol{\theta}\|^2\right] + \text{const}.
\end{aligned} \tag{7.9}$$

Maximizing this is exactly ridge regression with $\lambda = \sigma^2/\tau^2$! The penalty is a prior belief that the weights are small, and λ is the ratio of how noisy we think the data is to how big we think the weights are. A very confident prior (small τ) means a big λ. If you use a Laplace prior $f(\theta_j) \propto e^{-|\theta_j|/b}$ instead, you get the lasso (Problem 7.2). Almost every regularizer in machine learning can be read as a prior in disguise.

Problem 7.2:

Show that the MAP estimate for linear regression with Gaussian noise and an i.i.d. Laplace prior on the weights is the lasso, and find λ in terms of σ^2 and b. Then argue that the lasso is *not* the posterior mean under this prior. (Hint, is the posterior mean ever exactly zero?)

7.2 Bayesian Linear Regression

MAP throws away most of the posterior and keeps only its peak. What do we get if we keep the whole thing? Let's do it for regression, following Bishop's Section 3.3 [3], because the answer is both beautiful and useful. Use a basis expansion like Chapter 3, $\hat{y}(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$, Gaussian noise with precision $\beta = 1/\sigma^2$, and a Gaussian prior $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I})$. (Bishop's α and β are precisions, inverse variances, so here α is not a learning rate.) The log posterior is, up to a constant,

$$-\frac{\beta}{2}\|\mathbf{y} - \boldsymbol{\Phi}\mathbf{w}\|^2 - \frac{\alpha}{2}\mathbf{w}^T\mathbf{w} = -\frac{1}{2}\mathbf{w}^T\big(\alpha\mathbf{I} + \beta\boldsymbol{\Phi}^T\boldsymbol{\Phi}\big)\mathbf{w} + \beta\,\mathbf{w}^T\boldsymbol{\Phi}^T\mathbf{y} + \text{const.} \tag{7.10}$$

It is quadratic in $\mathbf{w}$, so the posterior is Gaussian, and completing the square (Problem 7.3) reads off its covariance and mean,

$$f(\mathbf{w} \mid \mathcal{D}) = \mathcal{N}(\mathbf{w}; \mathbf{m}_N, \mathbf{S}_N), \qquad \mathbf{S}_N = \big(\alpha\mathbf{I} + \beta\boldsymbol{\Phi}^T\boldsymbol{\Phi}\big)^{-1}, \qquad \mathbf{m}_N = \beta\,\mathbf{S}_N\boldsymbol{\Phi}^T\mathbf{y}. \tag{7.11}$$

The mean $\mathbf{m}_N$ is the ridge solution with $\lambda = \alpha/\beta$, as promised. The new ingredient is the covariance $\mathbf{S}_N$, which tells us how uncertain we are about the weights. To make a prediction at a new input $\mathbf{x}$, we average the model over all the weights the posterior still considers plausible. This gives the **posterior predictive distribution**, which is again Gaussian,

$$y \mid \mathbf{x}, \mathcal{D} \;\sim\; \mathcal{N}\Big(\mathbf{m}_N^T\boldsymbol{\phi}(\mathbf{x}),\; \underbrace{\tfrac{1}{\beta}}_{\text{noise}} + \underbrace{\boldsymbol{\phi}(\mathbf{x})^T\mathbf{S}_N\,\boldsymbol{\phi}(\mathbf{x})}_{\text{uncertainty in } \mathbf{w}}\Big). \tag{7.12}$$

The predictive variance has two pieces. One is the irreducible noise from Chapter 3. The other comes from not knowing the weights exactly, and it is small where we have lots of data and large where we have none. So the model knows when it doesn't know.

```
import numpy as np

rng = np.random.default_rng(1)
f_true = lambda t: np.sin(2 * np.pi * t)
```

```

centers = np.linspace(0, 1, 9)
def phi(t, s=0.1):
    """Constant plus 9 Gaussian bumps."""
    return np.c_[np.ones(len(t)), np.exp(-(t[:, None] - centers) ** 2 / (2 * s ** 2))]

alpha, beta = 2.0, 25.0                  # prior precision, noise precision
x_all = rng.uniform(0, 1, 25)
y_all = f_true(x_all) + rng.normal(0, 0.2, 25)

def posterior(x, y):
    P = phi(x)
    S_N = np.linalg.inv(alpha * np.eye(P.shape[1]) + beta * P.T @ P)
    m_N = beta * S_N @ P.T @ y
    return m_N, S_N

def predictive(x_new, m_N, S_N):
    P = phi(x_new)
    mean = P @ m_N
    var = 1 / beta + np.einsum("ij,jk,ik->i", P, S_N, P)
    return mean, np.sqrt(var)

x_query = np.array([0.25, 0.5, 0.95])
for n in [2, 6, 25]:
    m_N, S_N = posterior(x_all[:n], y_all[:n])
    mean, sd = predictive(x_query, m_N, S_N)
    print(f"N={n:2d}  mean {np.round(mean, 2)}  sd {np.round(sd, 2)}")
```

```
N= 2  mean [-0.27 -0.44 -0.34]  sd [0.94 0.29 0.28]
N= 6  mean [ 0.86 -0.15 -0.3 ]  sd [0.31 0.26 0.24]
N=25  mean [ 0.9  -0.06 -0.26]  sd [0.22 0.22 0.22]
```

Look at $x = 0.25$ with only two data points, neither of them nearby. The model guesses -0.27, which is badly wrong (the truth is 1), but it also reports a standard deviation of 0.94, so it is telling us not to trust that guess. As N grows, the predictions approach $\sin(2\pi x)$ (which is 1, 0 and -0.31 at these three inputs) and the standard deviations shrink toward the noise level $1/\sqrt{\beta} = 0.2$, but never below it. Figure 7.2 shows the whole picture, including a few functions sampled from the posterior. With two data points the posterior is wide and the sampled curves disagree wildly away from the data. With 25 they all agree.

This is a big deal for anything that has to make decisions. A self-driving car, a doctor or an agent exploring an environment (Chapter 8) all need to know how much to trust a prediction, and a single point estimate can't tell them. Replace the basis functions with

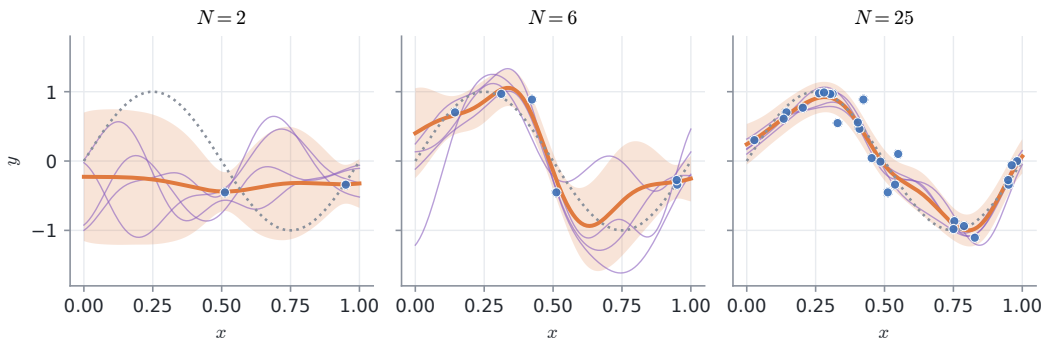


Figure 7.2: Bayesian linear regression with 9 Gaussian basis functions as data arrives. Orange is the predictive mean with a one standard deviation band, purple are functions drawn from the posterior over weights, and dotted grey is the truth $\sin(2\pi x)$.

a kernel (Chapter 6) and you get **Gaussian processes**, which are still the standard tool for small-data regression with honest error bars, for example in Bayesian optimization of hyperparameters.

Problem 7.3:

(1) Complete the square to derive $\mathbf{S}_N$ and $\mathbf{m}_N$. (Hint, a Gaussian $\mathcal{N}(\mathbf{m}, \mathbf{S})$ has log density $-\frac{1}{2}\mathbf{w}^T\mathbf{S}^{-1}\mathbf{w} + \mathbf{w}^T\mathbf{S}^{-1}\mathbf{m} + \text{const}$, so match the quadratic and linear terms.)

(2) Show that if we process the data one point at a time, using the posterior after each point as the prior for the next, we end up with the same $\mathbf{m}_N$ and $\mathbf{S}_N$ as processing it all at once. This sequential updating is one of the nicest features of Bayesian learning.

7.3 Linear Discriminant Analysis

Having had our discussion on Bayesian statistics, we turn to a classifier built directly out of Bayes' theorem, **linear discriminant analysis** (LDA). The problem is the same one as in Chapter 2, assigning points to one of K classes. The difference is in *what we model*. Logistic regression models $\mathbb{P}(\text{class} \mid \mathbf{x})$ directly. It asks "given this point, how likely is each class?" and never bothers to describe what the points themselves look like. A model of that type is called **discriminative**. LDA turns things around. It models $p(\mathbf{x} \mid \text{class})$, a description of what the data from each class looks like, plus the prior probability of each class, and then uses Bayes' theorem to flip it into $\mathbb{P}(\text{class} \mid \mathbf{x})$. A model of that type is called **generative**,

because we could use it to generate new fake data points from each class. (The first edition had these two descriptions swapped. They are related by Bayes' theorem, so it's an easy mix-up.)

We change notation a bit to blend in with the Hastie–Tibshirani crowd [6]. Let G be the class label, $\pi_k = \mathbb{P}(G = k)$ the prior probability of class k, and $f_k(\mathbf{x})$ the density of $\mathbf{x}$ within class k. Then Bayes' theorem gives

$$\mathbb{P}(G = k \mid \mathbf{x}) = \frac{f_k(\mathbf{x})\,\pi_k}{\sum_{\ell=1}^{K} f_\ell(\mathbf{x})\,\pi_\ell}. \tag{7.13}$$

This is the foundation of the method. Now we make two assumptions about the data. Each class density f_k is a multivariate Gaussian, and all the classes share the same covariance matrix $\boldsymbol{\Sigma}$,

$$f_k(\mathbf{x}) = \frac{1}{(2\pi)^{M/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)\right). \tag{7.14}$$

So each class is a blob with the same shape, just centered at a different place. With both the prior and the likelihood in hand, we could compute the posterior. More instructive is to find the decision boundary between two classes k and ℓ, which is where $\mathbb{P}(G = k \mid \mathbf{x}) = \mathbb{P}(G = \ell \mid \mathbf{x})$, that is, where the log of their ratio is zero,

$$\log \frac{\mathbb{P}(G = k \mid \mathbf{x})}{\mathbb{P}(G = \ell \mid \mathbf{x})} = \log \frac{f_k(\mathbf{x})}{f_\ell(\mathbf{x})} + \log \frac{\pi_k}{\pi_\ell}. \tag{7.15}$$

The normalizing constants of the two Gaussians are identical (same $\boldsymbol{\Sigma}$!) and cancel. Expanding the quadratic forms, the $\mathbf{x}^T\boldsymbol{\Sigma}^{-1}\mathbf{x}$ pieces also cancel, which is the whole magic of the shared covariance,

$$\begin{aligned}
\log \frac{f_k(\mathbf{x})}{f_\ell(\mathbf{x})} &= -\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_k) + \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_\ell)^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}_\ell) \\
&= \mathbf{x}^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_k - \boldsymbol{\mu}_\ell) - \frac{1}{2}\boldsymbol{\mu}_k^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k + \frac{1}{2}\boldsymbol{\mu}_\ell^T\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_\ell \\
&= \mathbf{x}^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_k - \boldsymbol{\mu}_\ell) - \frac{1}{2}(\boldsymbol{\mu}_k + \boldsymbol{\mu}_\ell)^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_k - \boldsymbol{\mu}_\ell).
\end{aligned} \tag{7.16}$$

So the boundary is where

$$\mathbf{x}^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_k - \boldsymbol{\mu}_\ell) = \frac{1}{2}(\boldsymbol{\mu}_k + \boldsymbol{\mu}_\ell)^T\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_k - \boldsymbol{\mu}_\ell) - \log \frac{\pi_k}{\pi_\ell} \equiv c. \tag{7.17}$$

The right hand side is just a number, and the only thing depending on the data is the left hand side, which is linear in $\mathbf{x}$. How does this give us a decision boundary? Let's break it down. We are dotting $\mathbf{x}$ with the fixed vector $\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_k - \boldsymbol{\mu}_\ell)$. This should ring some bells

from support vector machines. That vector is **perpendicular to the decision boundary**, playing exactly the role of $\mathbf{w}$, and the boundary is a hyperplane. If the projection of $\mathbf{x}$ onto it is bigger than c, the log ratio is positive and we pick class k, otherwise class ℓ. It may seem like we are doing binary classification, but the labels k and ℓ are generic, so this works for every pair. A cleaner way to handle many classes at once is to compute the **linear discriminant function** for each class,

$$\delta_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k - \frac{1}{2} \boldsymbol{\mu}_k^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_k + \log \pi_k, \tag{7.18}$$

and pick the class with the largest one. (The boundary between k and ℓ is exactly $\delta_k = \delta_\ell$, which is (7.17).)

7.3.1 Estimating The Parameters

This is great if we knew the true means and the true covariance. But if we knew those, we wouldn't need machine learning. So we estimate them from the data. The class densities are Gaussian by assumption, and we know the maximum likelihood estimators for Gaussians. The prior is just a proportion, so we use the proportion of training points in each class,

$$\hat{\pi}_k = \frac{N_k}{N}, \qquad \hat{\boldsymbol{\mu}}_k = \frac{1}{N_k} \sum_{g_i = k} \mathbf{x}_i, \qquad \hat{\boldsymbol{\Sigma}} = \frac{1}{N - K} \sum_{k=1}^{K} \sum_{g_i = k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^T, \tag{7.19}$$

where N_k is the number of class k observations and g_i is the label of point i. The covariance estimate is called the **pooled** covariance. Each point is measured relative to its own class mean, and everything is pooled together. Dividing by $N - K$ instead of N makes it unbiased, since we used up K degrees of freedom estimating the means. Notice that there's no iterative optimization at all! LDA is trained in closed form in one pass over the data, which is part of why it is still a popular baseline. Let us do it on the three clusters of Figure 2.6.

```
np.random.seed(42)
centers = np.array([[1, 1], [3, 2], [2, 4]])
X = np.vstack([centers[0] + 0.4 * np.random.randn(100, 2),
               centers[1] + 0.5 * np.random.randn(100, 2),
               centers[2] + 0.3 * np.random.randn(100, 2)])
g = np.repeat([0, 1, 2], 100)

def lda_fit(X, g):
    classes = np.unique(g)
    N, K = len(X), len(classes)
    pi = np.array([np.mean(g == k) for k in classes])
    mu = np.array([X[g == k].mean(axis=0) for k in classes])
    resid = X - mu[g]                          # each point minus its class mean
```

```
    Sigma = resid.T @ resid / (N - K)          # pooled covariance
    return pi, mu, Sigma

def lda_predict(X, pi, mu, Sigma):
    Sinv_mu = np.linalg.solve(Sigma, mu.T)      # columns are Sigma^{-1} mu_k
    delta = X @ Sinv_mu - 0.5 * np.sum(mu.T * Sinv_mu, axis=0) + np.log(pi)
    return delta.argmax(axis=1)

pi, mu, Sigma = lda_fit(X, g)
print("priors:", pi)
print("means:\n", np.round(mu, 3))
print("pooled covariance:\n", np.round(Sigma, 4))
print("training accuracy:", np.mean(lda_predict(X, pi, mu, Sigma) == g))
```

```
priors: [0.33333333 0.33333333 0.33333333]
means:
 [[0.954 1.014]
 [3.064 2.022]
 [1.986 3.962]]
pooled covariance:
 [[ 0.1607 -0.0027]
 [-0.0027  0.154 ]]
training accuracy: 0.9866666666666667
```

The pooled covariance matches the first edition's output to four digits (it computed the same thing as a weighted average of the three class covariances, which is equivalent here since the classes have equal size). The results are in Figure 7.3. Three straight boundaries, very similar to softmax regression in Figure 2.7, but obtained with no gradient descent at all.

7.3.2 Generative Versus Discriminative

Here is something cute. Divide the numerator and denominator of the two-class LDA posterior by the numerator, and you get

$$\mathbb{P}(G = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp\big(-\log \frac{f_1(\mathbf{x})\pi_1}{f_2(\mathbf{x})\pi_2}\big)} = \sigma(\mathbf{w}^T\mathbf{x} + w_0), \tag{7.20}$$

since we just showed that the log ratio is linear in $\mathbf{x}$. The posterior of LDA has *exactly* the form of logistic regression! So which is better? They estimate the same kind of boundary in different ways. LDA makes a strong assumption (Gaussian classes, shared covariance). When that assumption is right, it uses the data more efficiently and needs fewer samples. Logistic regression assumes much less and is more robust when the assumption is wrong, for

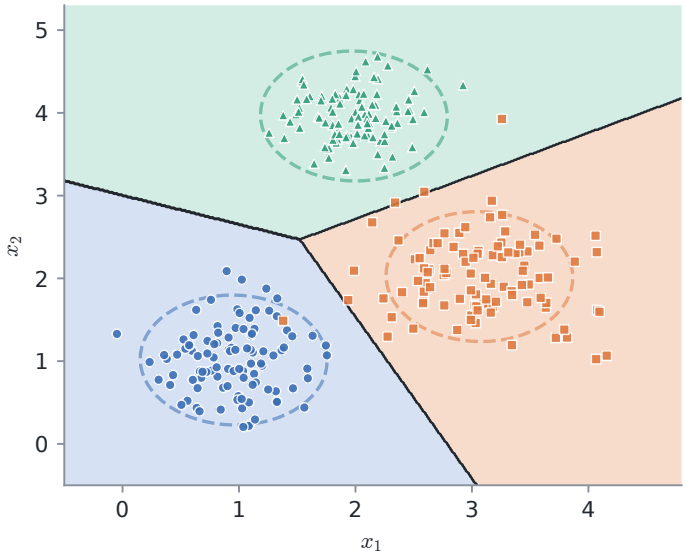


Figure 7.3: LDA on the three clusters. The dashed ellipses show the single pooled covariance placed at each class mean. Because the covariance is shared, every boundary is a straight line.

example with heavy tails or outliers, but it needs more data to get there. A classic result of Ng and Jordan made this precise, generative models approach their (worse) limit faster, discriminative models approach a better limit more slowly. Today the word "generative" mostly makes people think of image and text generators, and those really are generative models in this same sense, just with neural networks for f_k instead of Gaussians.

Problem 7.4:

Find $\mathbf{w}$ and w_0 in the expression above in terms of $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_2$, $\boldsymbol{\Sigma}$, π_1 and π_2. Then generate two-class data from Gaussians with a shared covariance, fit both LDA and logistic regression (Chapter 2) on training sets of size $N = 10, 20, 50, 200, 1000$, and plot the test error of each against N. Repeat with data where one class has a heavy-tailed distribution (say, a t distribution with 2 degrees of freedom). Which method wins in each case?

7.4 Quadratic Discriminant Analysis

The difference between quadratic discriminant analysis (QDA) and LDA is that we relax the assumption that all classes share the same covariance. Each class gets its own $\boldsymbol{\Sigma}_k$. When we take the log ratio now, the covariances don't cancel. The discriminant function, which is just the log of the numerator of the posterior with constants dropped, becomes

$$\delta_k(\mathbf{x}) = -\frac{1}{2}\log|\boldsymbol{\Sigma}_k| - \frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^T\boldsymbol{\Sigma}_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k) + \log\pi_k. \tag{7.21}$$

If the covariances are different, then when we set $\delta_k = \delta_\ell$, the $\mathbf{x}^T\boldsymbol{\Sigma}_k^{-1}\mathbf{x}$ terms no longer cancel, and the boundary is a **quadratic form** in $\mathbf{x}$, a conic section in two dimensions. Peep Figure 7.4, where one class is a tight round blob and the other is wide and tilted. LDA forces a straight line between them. QDA curves around the tight class, which is the right thing to do.

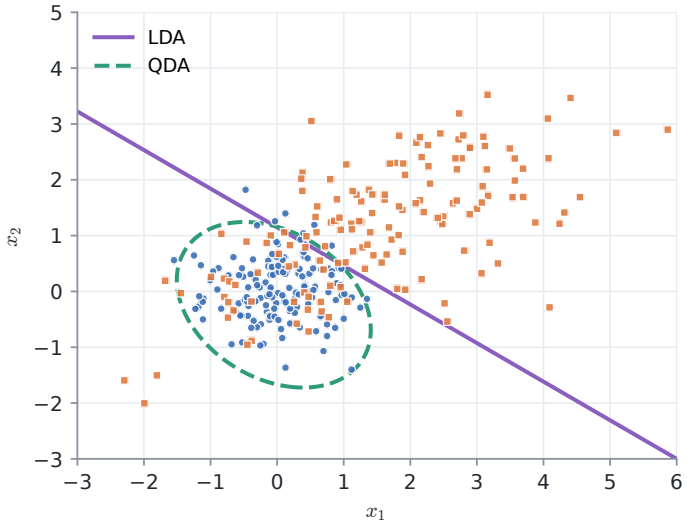


Figure 7.4: Two classes with very different covariances. LDA (purple) is stuck with a straight line. QDA (green, dashed) uses a separate covariance per class and draws an ellipse around the compact class.

The price is parameters. In M dimensions, each covariance matrix has $M(M+1)/2$ free entries. LDA estimates one of them, QDA estimates K of them, and with many features and few points per class the QDA estimates become noisy (bias-variance again). This is why LDA often wins in practice even when the covariances clearly differ. A common compromise, **regularized discriminant analysis**, shrinks each class covariance toward the pooled one, $\hat{\boldsymbol{\Sigma}}_k(a) = a\hat{\boldsymbol{\Sigma}}_k + (1-a)\hat{\boldsymbol{\Sigma}}$, with a chosen by cross-validation. LDA and QDA are the two

ends of that dial.

Problem 7.5:

Modify `lda_fit` and `lda_predict` to implement QDA. Count the number of parameters each model estimates for $M = 2$, 10 and 100 features with $K = 3$ classes. Then fit both to the data behind Figure 7.4 (two Gaussians, $\mathbf{\Sigma}_1 = 0.3\mathbf{I}$ and $\mathbf{\Sigma}_2 = \begin{bmatrix} 2.5 & 1.2 \\ 1.2 & 1.4 \end{bmatrix}$) and compare their test accuracies.

8

Reinforcement Learning

"The rat is always right."

– B. F. Skinner, more or less

Reinforcement learning is a paradigm of machine learning that is different from both supervised and unsupervised learning. The inspiration comes from psychology, where we train animals to behave a certain way by rewarding good actions and punishing bad ones. Nobody tells the dog the correct answer for every situation. It tries things, and the treats tell it how it's doing. The field is massive and deserves an entire book of its own (that book is Sutton and Barto [15], and you should read it). What we will do here is build the foundations carefully, implement the two classic algorithms from scratch on a small problem, and then sketch how the same ideas scale up to the systems that play Go and fine-tune language models. In the first edition we said there would be no code in this chapter because reinforcement learning "is not something we can write in a few lines". We were wrong. The core algorithms fit on a page.

8.1 Markov Decision Processes

Before any algorithm, we need terminology and a setup. Suppose we have an **agent** (a player, a driver, a robot) that lives in an **environment**. Sometimes it's very hard to explain to the agent exactly how to perform its task, because the details are too complicated or because we don't know how to solve the problem ourselves. Imagine trying to explain to a robot how to walk. How much force at each joint, at what angle, how wide a step, how to

recover from a stumble? There are so many things that we do without thinking. So instead of explaining, we let the agent attempt the task itself. If it does something good, we reward it. If it does something bad, we punish it, and over time it should make the good actions more likely and the bad ones less likely.

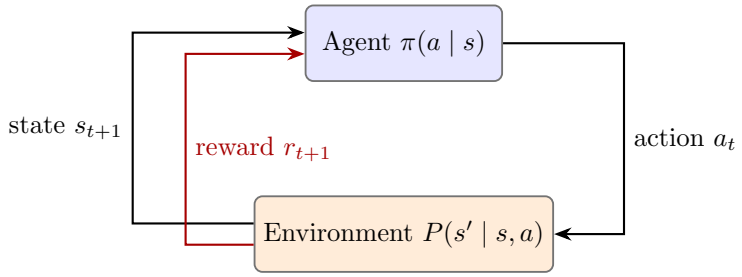


Figure 8.1: The agent–environment loop. At each step the agent sees a state, picks an action, and the environment responds with a reward and a new state.

The interaction goes in a loop (Figure 8.1). The mathematical framework for it is the **Markov decision process** (MDP), which consists of

- a set $\mathcal{S}$ of possible **states**,
- a set $\mathcal{A}$ of possible **actions**,
- a **transition model** $P(s' \mid s, a)$, the probability of landing in state s' after taking action a in state s (so $\sum_{s'} P(s' \mid s, a) = 1$ for all s, a),
- a **reward** $R(s, a, s')$ for each transition,

plus a **discount factor** $\gamma \in [0, 1)$ that we will explain in a moment. The agent starts in some state s_0, picks an action, collects a reward, lands somewhere new, and repeats. The reason it's called a *Markov* process is the Markov property from stochastic processes. The future depends only on the current state and action, not on the history of how we got here. What happens tomorrow only depends on today and not on yesterday. If your problem doesn't satisfy this, the usual fix is to put more information into the state until it does.

The last ingredient is the agent's behavior, the **policy** $\pi(a \mid s)$, which is the probability of taking action a in state s. A policy can be deterministic (always do the same thing in the same state), in which case we just write $a = \pi(s)$. The goal of reinforcement learning is to find a good policy. But good in what sense?

8.1.1 Returns And Discounting

We want the agent to collect as much reward as possible, but rewards arrive over time, so we have to decide how to add them up. One option is a **finite horizon**, where the agent

only cares about the next T steps,

$$G_t = \sum_{k=0}^{T-1} r_{t+k+1}. \tag{8.1}$$

The other extreme is an **infinite horizon**, letting $T \to \infty$. The trouble is that this sum can diverge, and then every policy is "infinitely good" and we can't compare them. The middle ground is the **discounted return**,

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k+1} = r_{t+1} + \gamma \, G_{t+1}. \tag{8.2}$$

Think of it like money. We would love to make 100 dollars in 10 years, but it matters more to us to make 100 dollars now. A reward k steps in the future is worth γ^k of a reward today. If rewards are bounded by $R_{\max}$, the geometric series guarantees $|G_t| \le R_{\max}/(1-\gamma)$, so everything is finite. A γ close to 1 makes the agent patient and far-sighted, and a small γ makes it greedy. The recursive form on the right of (8.2) is going to do a lot of work for us.

8.2 Value Functions And The Bellman Equations

To compare policies, we ask how much return they collect on average. The **state value function** of a policy is the expected return when starting in state s and following π forever after,

$$V^\pi(s) = \mathbb{E}_\pi \left[G_t \mid s_t = s \right], \tag{8.3}$$

and the **action value function**, or **Q-function**, is the expected return when we start in s, take action a first, and *then* follow π,

$$Q^\pi(s,a) = \mathbb{E}_\pi \left[G_t \mid s_t = s, a_t = a \right]. \tag{8.4}$$

The expectation is over everything random, the agent's own choices and the environment's transitions. Now use the recursion $G_t = r_{t+1} + \gamma G_{t+1}$. Conditioning on the first action and the first transition,

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi \left[r_{t+1} + \gamma \, G_{t+1} \mid s_t = s \right] \\ &= \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \Big[R(s,a,s') + \gamma \, \mathbb{E}_\pi [G_{t+1} \mid s_{t+1} = s'] \Big] \\ &= \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \Big[R(s,a,s') + \gamma \, V^\pi(s') \Big], \end{aligned} \tag{8.5}$$

where the Markov property let us replace the expected future return from s' by $V^\pi(s')$. This is the **Bellman expectation equation**. It says the value of a state is the immediate reward plus the discounted value of wherever we land next, averaged over everything that can happen. It's a system of $|\mathcal{S}|$ linear equations for the $|\mathcal{S}|$ unknowns $V^\pi(s)$, so for a small problem we could just solve it.

What we really want is the best policy. Define the **optimal value function** $V^*(s) = \max_\pi V^\pi(s)$. An optimal agent picks the best action at every step instead of averaging over π, which turns the sum over a into a max,

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a)\Big[R(s, a, s') + \gamma\, V^*(s')\Big], \tag{8.6}$$

and similarly for the optimal Q-function,

$$Q^*(s, a) = \sum_{s'} P(s' \mid s, a)\Big[R(s, a, s') + \gamma \max_{a'} Q^*(s', a')\Big]. \tag{8.7}$$

These are the **Bellman optimality equations**. Once we have Q^*, acting optimally is trivial, just be greedy,

$$\pi^*(s) = \arg\max_a Q^*(s, a). \tag{8.8}$$

So the whole game reduces to finding V^* or Q^*.

Problem 8.1:

(1) Fill in the conditioning steps in (8.5), using the law of total expectation over a_t and s_{t+1}, and point to exactly where the Markov property is needed.
(2) Write (8.5) in matrix form, $\mathbf{v} = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}$, and show that the solution $\mathbf{v} = (\mathbf{I} - \gamma \mathbf{P}_\pi)^{-1}\mathbf{r}_\pi$ exists for every $\gamma < 1$. (Hint, the rows of $\mathbf{P}_\pi$ sum to one, so its eigenvalues have modulus at most one.)

8.3 Value Iteration

The Bellman optimality equation (8.6) is a fixed point equation, $V^* = \mathcal{T}V^*$, where $\mathcal{T}$ is the operator on the right hand side. The simplest thing to do with a fixed point equation is to iterate it. Start from any guess, say $V = 0$, and repeatedly replace V by $\mathcal{T}V$. This is **value iteration**, and it is guaranteed to converge because $\mathcal{T}$ is a contraction (Problem 8.2). Every sweep shrinks the distance to V^* by at least a factor of γ.

Let's build an environment to try it on. Our **gridworld** is a 5×5 grid with a few walls. The agent starts in the bottom left corner. The top right cell is a goal worth $+1$ and the cell right below it is a pit worth -1, and both end the episode. Every other step costs 0.04, so dawdling is bad. To make it interesting the floor is slippery. The agent moves in the intended direction with probability 0.8, and slips to each of the two perpendicular directions with probability 0.1. Bumping into a wall or the edge means staying put.

```
import numpy as np

ROWS, COLS = 5, 5
WALLS = {(1, 1), (2, 1), (2, 3), (3, 3)}
TERMINAL = {(0, 4): 1.0, (1, 4): -1.0}      # goal and pit
START = (4, 0)
STEP_REWARD = -0.04
ACTIONS = [(-1, 0), (0, 1), (1, 0), (0, -1)]  # up, right, down, left
ARROWS = "^>v<"
STATES = [(r, c) for r in range(ROWS) for c in range(COLS) if (r, c) not in WALLS]

def move(s, a):
    r, c = s[0] + ACTIONS[a][0], s[1] + ACTIONS[a][1]
    if 0 <= r < ROWS and 0 <= c < COLS and (r, c) not in WALLS:
        return (r, c)
    return s                                    # bumped into something

def transitions(s, a, slip=0.1):
    """All outcomes of action a in state s, as (prob, next_state, reward)."""
    if s in TERMINAL:
        return [(1.0, s, 0.0)]                  # absorbing, nothing more happens
    outcomes = []
    for prob, a_actual in [(1 - 2 * slip, a), (slip, (a + 1) % 4), (slip, (a + 3) % 4)]:
        s_next = move(s, a_actual)
        outcomes.append((prob, s_next, TERMINAL.get(s_next, STEP_REWARD)))
    return outcomes
```

Value iteration is then a direct transcription of (8.6). We sweep over the states, back up each value with the best one-step lookahead, and stop when nothing changes much.

```
def q_from_v(s, V, gamma):
    """One-step lookahead: expected reward + discounted next value, per action."""
    return np.array([sum(p * (r + gamma * V[s2]) for p, s2, r in transitions(s, a))
                     for a in range(4)])

def value_iteration(gamma=0.95, tol=1e-10):
    V = {s: 0.0 for s in STATES}
    sweeps = 0
```

```
    while True:
        delta = 0.0
        for s in STATES:
            if s in TERMINAL:
                continue
            v_new = q_from_v(s, V, gamma).max()        # Bellman optimality backup
            delta = max(delta, abs(v_new - V[s]))
            V[s] = v_new
        sweeps += 1
        if delta < tol:
            break
    policy = {s: int(np.argmax(q_from_v(s, V, gamma)))
              for s in STATES if s not in TERMINAL}
    return V, policy, sweeps

def show(V, policy):
    for r in range(ROWS):
        row = []
        for c in range(COLS):
            s = (r, c)
            if s in WALLS:       row.append("  ###  ")
            elif s in TERMINAL:  row.append(f" {TERMINAL[s]:+.2f} ")
            else:                row.append(f"{V[s]:+.2f} {ARROWS[policy[s]]}")
        print(" ".join(row))

V_star, pi_star, sweeps = value_iteration()
print("converged in", sweeps, "sweeps")
show(V_star, pi_star)
```

```
converged in 34 sweeps
+0.62 > +0.72 > +0.82 > +0.94 >  +1.00
+0.53 ^   ###   +0.72 ^ +0.66 <  -1.00
+0.45 ^   ###   +0.62 ^   ###   +0.14 v
+0.38 ^ +0.43 > +0.52 ^   ###   +0.21 v
+0.31 ^ +0.36 ^ +0.43 ^ +0.35 < +0.27 <
```

The values and the policy are drawn in Figure 8.2. Values are highest near the goal and decay with distance, and the arrows always point toward higher values. There is a subtle thing to notice next to the pit. In the cell just left of it, $(1, 3)$, the agent does *not* go straight up toward the goal row. Going up risks slipping right, into the pit, with probability 0.1. So the policy points left, away from the pit, and accepts that it will sometimes slip up into the safe row instead. No one programmed caution into it. It falls out of the expectation in the Bellman equation.

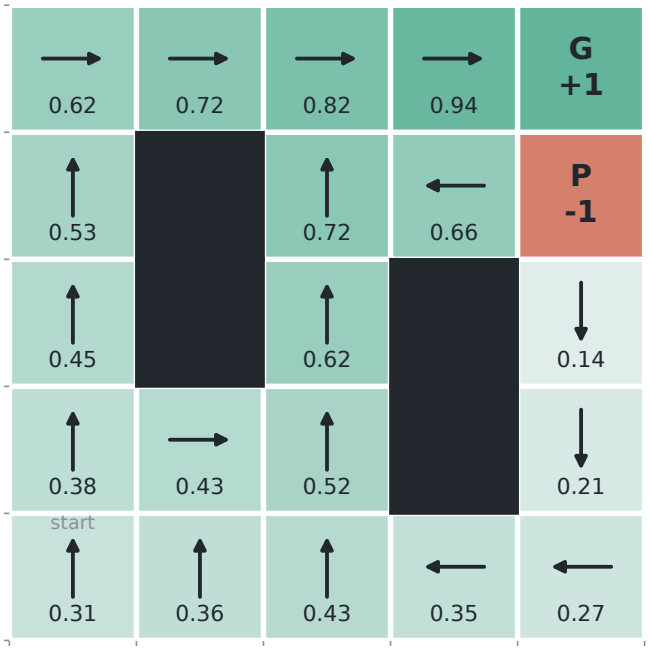


Figure 8.2: The optimal value function (color and numbers) and optimal policy (arrows) of the slippery gridworld, computed by value iteration with $\gamma = 0.95$. Dark cells are walls, G is the goal and P is the pit.

Problem 8.2:

(1) Let $\|V\|_\infty = \max_s |V(s)|$. Show that for any two value functions, $\|\mathcal{T}V - \mathcal{T}U\|_\infty \leq \gamma\|V - U\|_\infty$. (Hint, $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$.) By the Banach fixed point theorem, value iteration converges to a unique V^* from any start.

(2) Rerun value iteration with $\gamma = 0.5$ and with `slip=0.0`. How does the policy change in each case, especially near the pit? Then make the step reward -2 instead of -0.04. What does the agent do now, and why is it rational?

8.4 Q-Learning

Value iteration needs the transition model $P(s' \mid s, a)$. We had it because we wrote the environment. But a robot learning to walk doesn't get a formula for physics, and a game-playing agent doesn't get the source code of the game. It just gets to act and see what happens. Methods that learn from experience without a model are called **model-free**, and the most famous one is **Q-learning**.

The idea is to turn (8.7) into a stochastic update. We can't compute the expectation over s', but every time we take action a in state s and observe a reward r and next state s', we get one sample of the quantity inside it. So we nudge our estimate of $Q(s,a)$ toward that sample,

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{target}} - \underbrace{Q(s,a)}_{\text{prediction}}\Big]. \tag{8.9}$$

The bracket is called the **temporal difference (TD) error**, the difference between what we predicted and a slightly better informed guess that uses one real step of experience. Now look closely. This is a gradient descent step on the squared error between prediction and target, $\frac{1}{2}(\text{target} - Q(s,a))^2$, where we treat the target as a constant. It is least squares from Chapter 1 all over again, with the twist that the targets are built from our own current estimates (people call this "bootstrapping"). That the procedure converges at all is a theorem, under conditions like visiting every state–action pair infinitely often and decaying α appropriately.

There's one more ingredient, the **exploration–exploitation** dilemma. If the agent always takes the action that currently looks best, it might never try the action that is actually best. The simplest fix is **ϵ-greedy**, where we take a random action with probability ϵ and the greedy one otherwise. Here's tabular Q-learning on our gridworld. Notice that it never calls `transitions` to plan. It only uses `step`, which samples one outcome, exactly like interacting with the real world.

```
def step(s, a, rng, slip=0.1):
    """Sample ONE outcome; this is all a model-free agent gets to see."""
    outcomes = transitions(s, a, slip)
    i = rng.choice(len(outcomes), p=[o[0] for o in outcomes])
    _, s_next, r = outcomes[i]
    return s_next, r, s_next in TERMINAL

def q_learning(episodes=3000, alpha=0.1, gamma=0.95, eps=0.1, seed=0):
    rng = np.random.default_rng(seed)
    Q = {s: np.zeros(4) for s in STATES}
    returns = []
    for ep in range(episodes):
        s, G, t, done = START, 0.0, 0, False
        while not done and t < 200:
            if rng.random() < eps:
                a = int(rng.integers(4))                        # explore
            else:
                a = int(rng.choice(np.flatnonzero(Q[s] == Q[s].max())))  # exploit,
    ↪ random tie-break
            s_next, r, done = step(s, a, rng)
```

```
            target = r if done else r + gamma * Q[s_next].max()
            Q[s][a] += alpha * (target - Q[s][a])              # TD update
            G += gamma ** t * r
            s, t = s_next, t + 1
        returns.append(G)
    return Q, np.array(returns)

Q, returns = q_learning()
pi_q = {s: int(np.argmax(Q[s])) for s in STATES if s not in TERMINAL}
agree = np.mean([pi_q[s] == pi_star[s] for s in pi_q])
print(f"mean return, first 100 episodes: {returns[:100].mean():+.3f}")
print(f"mean return, last 500 episodes:  {returns[-500:].mean():+.3f}")
print(f"V*(start) from value iteration:  {V_star[START]:+.3f}")
print(f"policy agrees with value iteration in {agree:.0%} of states")
```

```
mean return, first 100 episodes: -0.003
mean return, last 500 episodes:  +0.209
V*(start) from value iteration:  +0.312
policy agrees with value iteration in 79% of states
```

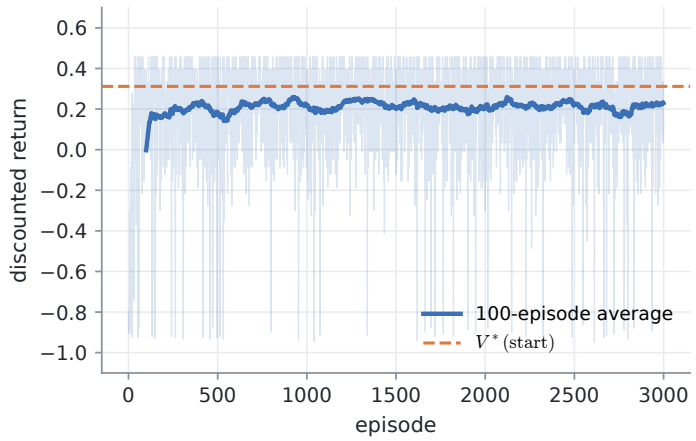


Figure 8.3: Discounted return per episode while Q-learning trains on the gridworld (light, with a 100-episode moving average in bold). The dashed line is $V^*(\text{start})$ from value iteration. The learning agent climbs quickly but stays below it, because it keeps exploring with $\epsilon = 0.1$.

The learning curve is in Figure 8.3. Early on the agent wanders and sometimes falls into the pit. Within a couple of hundred episodes it has found a good route, and its average return levels off around 0.21, below the optimal 0.31. It won't reach V^* while training, because ϵ-greedy keeps taking random actions 10% of the time, and on a slippery floor next to a pit a random action can be very expensive. (Run the greedy policy with $\epsilon = 0$ afterwards

and the gap shrinks a lot.) The states where its greedy policy still disagrees with value iteration are mostly states it rarely visits, which is exactly what you'd expect from learning by experience.

Problem 8.3:

(1) Run `q_learning` with $\epsilon \in \{0, 0.05, 0.1, 0.3\}$ and plot the moving-average return for each. Why does $\epsilon = 0$ sometimes do fine and sometimes get stuck? (Try several seeds.)
(2) Replace the target $r + \gamma \max_{a'} Q(s', a')$ by $r + \gamma Q(s', a')$, where a' is the action the agent actually takes next. This is **SARSA**. Compare the learned policies next to the pit. Which algorithm learns the safer route while exploring, and why?

8.5 Scaling Up, Deep Q-Networks

A table with one entry per state works for 21 states. It doesn't work for chess ($\sim 10^{44}$ positions) or for an Atari game, where the state is a screen of pixels. The fix is the one this whole book has been building toward. Replace the table with a function approximator, a neural network $Q_{\boldsymbol{\theta}}(s, a)$ that takes the state as input and outputs one Q-value per action, and fit it by gradient descent on the TD error,

$$\mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}\left[\Big(\underbrace{r + \gamma \max_{a'} Q_{\boldsymbol{\theta}^-}(s', a')}_{\text{target}} - \underbrace{Q_{\boldsymbol{\theta}}(s, a)}_{\text{predicted}}\Big)^2\right]. \tag{8.10}$$

(The first edition's version of this equation forgot the Q inside the max, which made the target just $r + \gamma$ max of nothing.) This is the deep Q-network (DQN) of Mnih and colleagues [10], which learned to play dozens of Atari games from raw pixels at human level. Two tricks made it stable. First, **experience replay**. Transitions (s, a, r, s') go into a big buffer, and each gradient step uses a random mini-batch from it, which breaks the strong correlation between consecutive frames (consecutive samples violate the i.i.d. assumption that SGD quietly relies on). Second, a **target network**. The target uses an older copy of the parameters $\boldsymbol{\theta}^-$, refreshed only every few thousand steps, so the network isn't chasing a target that moves every time it updates. Without these, the bootstrapped targets feed back on themselves and training tends to blow up.

8.6 Policy Gradients

Q-learning learns values and derives a policy from them. Another family of methods skips the middleman and optimizes the policy directly. Parametrize a stochastic policy $\pi_{\boldsymbol{\theta}}(a \mid s)$, for example as a softmax over action scores from a neural network (Chapter 2!), and maximize the expected return $J(\boldsymbol{\theta}) = \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\theta}}}[G(\tau)]$, where $\tau = (s_0, a_0, r_1, s_1, \dots)$ is a whole trajectory. The problem is that the expectation is over trajectories whose distribution depends on $\boldsymbol{\theta}$, and the environment's dynamics are unknown, so how can we differentiate it? With the same log-derivative trick we used for Gaussian mixtures in Chapter 5,

$$\begin{aligned} \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) &= \nabla_{\boldsymbol{\theta}} \sum_{\tau} p_{\boldsymbol{\theta}}(\tau)\, G(\tau) \\ &= \sum_{\tau} p_{\boldsymbol{\theta}}(\tau)\, \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\tau)\, G(\tau) \\ &= \mathbb{E}_{\tau}\left[G(\tau) \sum_{t} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t \mid s_t) \right]. \end{aligned} \tag{8.11}$$

The last step is the beautiful part. The probability of a trajectory is a product of policy probabilities $\pi_{\boldsymbol{\theta}}(a_t \mid s_t)$ and transition probabilities $P(s_{t+1} \mid s_t, a_t)$. Taking the log turns the product into a sum, and the transition terms don't depend on $\boldsymbol{\theta}$, so their gradients vanish. We never need the model! We can estimate the gradient just by running the policy and averaging. This is the **REINFORCE** algorithm [18]. Intuitively, it pushes up the log-probability of every action in proportion to the return that followed it. Good trajectories get reinforced, hence the name.

In practice we subtract a **baseline** b from the return, using $(G - b)$ instead of G, which doesn't change the expected gradient (Problem 8.4) but reduces its variance enormously. Here's REINFORCE on the simplest possible problem, a three-armed bandit (one state, three slot machines with different average payouts), with a softmax policy over three preferences. For a softmax, $\nabla_{\boldsymbol{\theta}} \log \pi(a) = \mathbf{e}_a - \boldsymbol{\pi}$, the one-hot vector of the chosen action minus the probabilities. Yes, truth minus prediction one more time.

```
rng = np.random.default_rng(0)
true_means = np.array([0.2, 0.5, 0.8])          # arm 2 is best
theta = np.zeros(3)                             # action preferences
baseline, lr = 0.0, 0.1

for t in range(2000):
    pi = np.exp(theta - theta.max()); pi /= pi.sum()     # softmax policy
    a = rng.choice(3, p=pi)
    reward = rng.normal(true_means[a], 0.5)
```

```
    grad_log_pi = np.eye(3)[a] - pi                    # e_a - pi
    theta += lr * (reward - baseline) * grad_log_pi    # REINFORCE step
    baseline += 0.05 * (reward - baseline)             # running average reward

pi = np.exp(theta - theta.max()); pi /= pi.sum()
print("final policy:", np.round(pi, 3))
```

```
final policy: [0.003 0.004 0.993]
```

The policy has learned to pull the best arm almost every time.

8.6.1 From Games To Language Models

Policy gradients with a learned baseline (an "actor" for the policy and a "critic" estimating values) are the backbone of most modern deep reinforcement learning. The most widely used variant, **proximal policy optimization** (PPO) [13], adds one idea. It clips each update so that the new policy can't move too far from the old one in a single step, which makes training dramatically more stable. PPO is also what made reinforcement learning a household name, because it is the algorithm behind **reinforcement learning from human feedback** (RLHF) [11]. There, the "agent" is a language model, an "action" is the next token, an episode is a whole response, and the reward comes from a neural network trained to predict which of two responses a human would prefer. Every concept in this chapter shows up in that pipeline, including the discount factor, the policy gradient, the baseline and the exploration problem. So when we said reinforcement learning takes a different approach to machine learning, that's true, but the engine underneath is the same one we have been building since Chapter 1. A model, a loss, a gradient, and a loop.

Problem 8.4:

(1) Show that $\mathbb{E}_{a\sim\pi_{\boldsymbol{\theta}}}\left[\nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a)\right] = \mathbf{0}$. Conclude that subtracting any constant baseline b from the reward leaves the expected REINFORCE gradient unchanged.

(2) Run the bandit code with `baseline` fixed at 0 and with the running average, for 20 different seeds each. Plot the probability of choosing the best arm against t, averaged over seeds, and comment on the difference.

(3) For a softmax policy, verify $\nabla_{\boldsymbol{\theta}} \log \pi(a) = \mathbf{e}_a - \boldsymbol{\pi}$. Where have you seen this expression before?

Bibliography

[1] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.

[2] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.

[3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[4] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024.

[5] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[6] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2nd edition, 2009.

[7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision*, 2015.

[8] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations*, 2015.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations*, 2019.

[10] Volodymyr Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[11] Long Ouyang et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 2022.

[12] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.

[13] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[14] Shai Shalev-Shwartz, Yoram Singer, Nathan Srebro, and Andrew Cotter. Pegasos: Primal estimated sub-gradient solver for SVM. *Mathematical Programming*, 127(1):3–30, 2011.

[15] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

[16] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1):267–288, 1996.

[17] Ashish Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[18] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.